

Python: lộ trình chính và học phần nâng cao

92 bài học dành cho đội phát triển ProgrammingEdu. Dùng cùng lesson_bank.json; đáp án và tiêu chí chấm chi tiết nằm trong gói đồng hành.

Cách đi qua mỗi bài

S1: hiểu tình huống, xem ví dụ và dự đoán. S2: trả lời câu hỏi, nhận phản hồi theo lựa chọn. S3: hoàn thiện có hỗ trợ, thay dữ liệu và giải thích. S4: tự giải nhiệm vụ mới, mở gợi ý theo nhu cầu.

S4 mặc định ẩn lời giải. Ghi riêng lần làm có gợi ý. Chấp nhận lời giải tương đương đúng hợp đồng. Lỗi môi trường chạy không tính là sai kiến thức.

Sau mỗi bài: giải thích một quyết định trong lời giải, dự đoán điều gì đổi khi đầu vào đổi và nêu một trường hợp khiến cách giải sai thường gặp thất bại.

Rubric giải thích chung 0-2: 0 sai cơ chế hoặc chép output; 1 đúng thao tác nhưng thiếu liên hệ; 2 nói đúng đầu vào, xử lý, đầu ra và trường hợp biên. Hoàn thành khi challenge đạt mọi tiêu chí bắt buộc và giải thích đạt 2. Mã dài trong PDF có thể xuống dòng để đọc; lấy mã nguyên vẹn từ JSON hoặc file đáp án.

Hai tầng học

Lộ trình chính: nền tảng Python, dữ liệu/file, OOP, Tkinter và ứng dụng nhỏ. Học phần nâng cao: hệ điều hành, đồng bộ, dữ liệu khoa học, MySQL, wxPython, OpenGL và chuỗi dự án tích hợp.

Mục lục sắp theo điều kiện học trước. P01-P24 giữ mã của bản cũ; C, T, A, X là bài bổ sung. Theo prerequisite từng bài để chuyển giữa các nhóm. Thời lượng là ước tính thiết kế; bài dài chia nhiều phiên 20-40 phút, không buộc làm một lượt trên điện thoại.

Quy ước tương tác dành cho dev

Giữ nhiệm vụ và fixture hiện trong lúc luyện. Chạy mã chỉ khi bấm Chạy; highlight dòng và ô dữ liệu thay đổi trong 250ms. Có thao tác chọn-đặt bằng bàn phím; reduced motion đổi trạng thái tức thì. Reset khôi phục fixture, giữ tiến độ. Quiz sai giữ nguyên câu và giải thích, không tự chuyển bước.

Bài native GUI chạy trên desktop lab. Phần xem trước trên mobile không thay thể nghiệm thu native. Dữ liệu fixture, lời giải và mã kiểm thử là thiết kế mới dựa trên khái niệm trong các trang nguồn được dẫn.

Mục lục bài học

C01 - Chọn công cụ và dựng môi trường học	6
C02 - REPL, script và notebook: trạng thái đến từ đâu?	7
C03 - Git: lưu một thay đổi và gửi vào fork	8
P01 - Tên biến và kết quả chương trình	9
P02 - Chuỗi nhập vào và phép tính số	10
C04 - Tên biến, kiểu động và ghi chú có ích	11
C05 - Toán tử, thứ tự tính và làm tròn xuống	12
P03 - Tách dữ liệu từ tên tệp	13
C06 - Chuỗi: chuẩn hóa, định dạng và không sửa tại chỗ	14
P04 - Điều kiện và giá trị ở ranh giới	15
P05 - List và việc hai tên dùng chung dữ liệu	16
C07 - Index và slice: chọn đúng phần dữ liệu	17
C08 - Thêm, xóa và đảo list	18
P06 - Dictionary và tra cứu theo khóa	19
C09 - Dictionary: duyệt, xóa và tìm cực trị	20
P07 - Chọn tuple và set đúng mục đích	21
C10 - Tuple, unpacking và lựa chọn collection	22
C17 - Tên file thành hồ sơ người bơi	23
P08 - Vòng for và biến tích lũy	24
C14 - Comprehension, enumerate, zip và Counter	25
P09 - While, sentinel và điều kiện dừng	26
C11 - range, break và continue	27
P10 - Hàm trả về giá trị	28
C13 - Khám phá API, traceback và module chuẩn	29
C15 - Ngẫu nhiên có thể kiểm thử	30

P11 - Module và trung bình có trọng số	31
C19 - Module, alias và package lồng nhau	32
P12 - Đọc file với with và UTF-8	33
P13 - Ngoại lệ và dữ liệu nhập không hợp lệ	34
C12 - Nhánh elif và input trong chương trình CLI	35
C16 - Ngày giờ và tìm mẫu bằng regex	36
P14 - Lưu cấu trúc dữ liệu bằng JSON	37
C21 - File modes, buffering và lỗi I/O	38
C22 - Duyệt thư mục và thao tác file trong vùng bài tập	39
P15 - Class và trạng thái riêng từng đối tượng	40
P16 - Kế thừa và ghi đè hành vi	41
C20 - So sánh đối tượng, biểu diễn và đóng gói	42
P17 - Tkinter: cửa sổ, widget và grid	43
P18 - Callback: nối thao tác và trạng thái	44
P19 - Biểu đồ phải khớp dữ liệu	45
P20 - Test trường hợp thường và trường hợp biên	46
P23 - Chuyển giao: báo cáo thời gian bơi	47
C18 - Báo cáo bơi từ file đến kết quả	48
P24 - Capstone: Sổ học tập USTH	49
T01 - Cửa sổ, focus và trạng thái nút	50
T02 - Lựa chọn dữ liệu: combo, radio, check và list	51
T03 - Bố cục: Frame, padding và co giãn	52
T04 - Menu và nhiều Notebook	53
T05 - Hộp thoại và lựa chọn file	54
T06 - Tooltip, relief và Canvas	55
T07 - Event loop, after và thanh tiến độ	56
T08 - StringVar, scope và component tái sử dụng	57

T09 - Biểu đồ trong Tk và thay đổi thang đo	58
T10 - Người học tự viết unit test cho GUI	59
T11 - Debug watch và mức log	60
T12 - Quốc tế hóa và định dạng theo ngôn ngữ	61
T13 - Slideshow GIF/JPEG và vòng đời ảnh	62
T14 - Thiết kế lặp và factory cho thành phần GUI	63
X01 - PW1: quản lý lớp bằng collection và CLI	64
A01 - Đa kế thừa, MRO và lớp trừu tượng	65
A02 - NumPy và xếp hạng GPA có trọng số	66
A03 - Curses: menu bàn phím cho quản lý điểm	67
A04 - Nén bytes và đóng gói nhiều file	68
A05 - Pickle, JSON và dữ liệu có kiểu	69
A14 - Đọc bảng số bằng NumPy	70
A15 - DataFrame từ CSV, Excel và database	71
A16 - SAS, Stata, HDF5 và MATLAB	72
A17 - Chọn loại biểu đồ và quản lý Figure	73
A18 - Dựng MySQL và thiết kế schema bằng Workbench	74
A19 - MySQL INSERT, UPDATE, DELETE và transaction	75
A20 - Nối CRUD MySQL vào giao diện	76
A21 - wxPython: cửa sổ và các nhóm control	77
A23 - Bitmap lát nền trong wxPython	78
A24 - OpenGL trong wx: cube và phép biến đổi 3D	79
A25 - Pyglet, shader và cube nhiều màu	80
A26 - Hoạt ảnh OpenGL: chuyển động theo thời gian	81
P21 - Thread và queue: nhận kết quả việc nền	82
P22 - Tiến trình con và mã kết thúc	83

A06 - Tiến trình, trạng thái và PCB	84
A07 - Mô phỏng FCFS, SJF, priority và round-robin	85
A08 - Tạo tiến trình trên Windows và fork/exec trên UNIX	86
A09 - Luồng chuẩn, pipe và vòng đời process	87
A10 - Bộ nhớ dùng chung, GIL và chọn concurrency	88
A11 - Race condition và Lock	89
A12 - Nhiều worker, dùng hợp tác và queue qua module	90
A13 - TCP và HTTP qua dịch vụ lab cục bộ	91
A22 - Hai toolkit: chặn đoán mainloop và trao đổi dữ liệu	92
X02 - PW2-PW4: domain objects, GPA và chia package	93
X03 - PW5-PW6: ba file, archive và pickle có nén	94
X04 - PW7: shell có chuyển hướng và pipeline	95
X05 - PW8: lưu pickle có nén trong background	96
X06 - PW9: ứng dụng quản lý điểm hoàn chỉnh bằng Tkinter	97

C01 | Chọn công cụ và dựng môi trường học

Lộ trình chính | 40 phút dự kiến | Học trước: Không

Nguồn (số trang PDF): SL 4-19; SL 19-21; HF 5-14; HF 89-90; GUI 187-225. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Bạn cần một nơi chạy bài học có thể dựng lại trên máy khác. Python chạy chương trình; editor giúp sửa mã; môi trường ảo tách thư viện của từng dự án.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
python --version
python -m venv .venv
.venv\Scripts\python -m pip --version
```

Các lệnh trên minh họa Windows. Linux/macOS dùng .venv/bin/python. Luôn gọi pip qua đúng interpreter của môi trường.

S2 | Kiểm tra hiểu

1. Hai dự án cần hai phiên bản thư viện khác nhau. Cách nào phù hợp?

A. Mỗi dự án có môi trường ảo | Đúng. Đúng: mỗi môi trường giữ bộ package riêng.

B. Cài đề liên tục vào một Python chung | Chưa đúng. Dự án kia có thể bị đổi dependency.

C. Đổi tên file .py | Chưa đúng. Tên file không tách thư viện.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chọn tình huống xử lý CSV, GUI và hard real-time: gán Python cho hai tình huống đầu; với hard real-time giải thích cần ràng buộc thời gian mà Python thông thường không bảo đảm.

2. Tạo .venv; chọn interpreter trong VS Code, cài Python extension. UI hiển thị đường dẫn interpreter đã chọn.

3. Cài matplotlib bằng python -m pip install matplotlib; đọc tên wheel/phiên bản Python/nền tảng trong log, lưu phiên bản thực cài. Không dùng tên wheel của sách 2017.

Thử thay đổi: Tạo dự án thứ hai; so sánh sys.executable ở hai terminal và giải thích vì sao package không tự chia sẻ.

S4 | Thử thách độc lập

Tạo lab_env mới, script hello.py in Hello PE. Nộp version Python, sys.executable, pip --version, log chạy script, requirements khóa phiên bản thực cài và ảnh interpreter VS Code. Giải thích một việc Python phù hợp và một việc cần cân nhắc hiệu năng/triển khai.

Môi trường: MANUAL_ENV. Challenge answer: answers/C01/answer.md; test_criteria.json khóa C01.

Gợi ý mở theo nhu cầu

1. Các lệnh trên minh họa Windows. Linux/macOS dùng .venv/bin/python. Luôn gọi pip qua đúng interpreter của môi trường.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: C01-K01 (môi trường); C01-K02 (tái tạo); C01-K03 (lựa chọn). Xem input/expected hoặc protocol trong test_criteria.json.

C02 | REPL, script và notebook: trạng thái đến từ đâu?

Lộ trình chính | 30 phút dự kiến | Học trước: C01

Nguồn (số trang PDF): SL 22-25; SL 28; HF 28-48,100-101. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Notebook giữ biến trong kernel giữa các cell. Một notebook chạy đúng phải tái tạo được kết quả khi khởi động kernel mới và chạy từ đầu.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
# Cell 1
count = 2
# Cell 2
count += 1
print(count)
```

Chạy cell 1 rồi 2 cho 3. Chạy lại riêng cell 2 cho 4. Script chạy lại từ đầu vẫn cho 3.

S2 | Kiểm tra hiểu

1. Sau Restart Kernel, chạy ngay cell 2 thì sao?

A. In 4 | Chưa đúng. Giá trị cũ đã mất.

B. NameError | Đúng. Đúng: count chưa tồn tại trong kernel mới.

C. Tự chạy cell 1 | Chưa đúng. Kernel không tự sắp thứ tự cell.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chạy phép tính 2+3 trong REPL rồi trong script không có print; vùng output cho thấy REPL hiện 5, script không tự hiện.

2. Tạo notebook trong VS Code, chọn kernel .venv; chạy cell 1, cell 2 hai lần và lưu dự đoán 3 rồi 4.

3. Restart + Run All; counter hiển thị 3; lưu notebook với kết quả sạch.

Thử thay đổi: Đổi count ban đầu thành 10, dự đoán kết quả khi chạy từ đầu và khi chỉ chạy cell 2.

S4 | Thử thách độc lập

Nộp counter.ipynb có cell khởi tạo base=7, cell tính total=base+5, cell in total; thêm Markdown giải thích state. Restart + Run All phải in 12. Tạo counter.py tương đương và chụp lỗi khi chạy cell tính trước khởi tạo.

Môi trường: MANUAL_NOTEBOOK. Challenge answer: answers/C02/answer.md; test_criteria.json khóa C02.

Gợi ý mở theo nhu cầu

1. Chạy cell 1 rồi 2 cho 3. Chạy lại riêng cell 2 cho 4. Script chạy lại từ đầu vẫn cho 3.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C02-K01 (clean run); C02-K02 (cell order). Xem input/expected hoặc protocol trong test_criteria.json.

C03 | Git: Lưu một thay đổi và gửi vào fork

Lộ trình chính | 40 phút dự kiến | Học trước: C01

Nguồn (số trang PDF): SL 96. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Commit lưu một mốc trong lịch sử; push gửi commit tới remote. Fork tạo bản repository dưới tài khoản của bạn để làm bài riêng.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
git clone <URL-fork-cua-ban> pe-lab
cd pe-lab
git status
git add README.md
git commit -m "First student commit"
git push origin HEAD
```

Sửa README trước git add. URL là fork của người học; không đẩy lên repository gốc của giảng viên.

S2 | Kiểm tra hiểu

1. Commit thành công nhưng chưa push. Bạn khác thấy commit trên GitHub chưa?

A. Chưa | Đúng. Đúng: commit mới còn ở local.

B. Đã thấy ngay | Chưa đúng. Commit local chưa gửi tới remote.

C. Phải xóa rồi clone lại | Chưa đúng. Chỉ cần push khi remote đúng.

S3 | Làm có hỗ trợ rồi thay đổi

1. Fork repository học tập do giảng viên chỉ định; xác nhận owner là tài khoản của mình.

2. Clone fork, sửa README với tên người học, xem git diff; tô sáng dòng mới.

3. Add, commit, kiểm tra log; push rồi so commit hash ở local và trên fork.

Thử thay đổi: Thêm một dòng mục tiêu học; tạo commit thứ hai và giải thích vì sao push không thay thế commit.

S4 | Thử thách độc lập

Theo PW0, fork repository môn học được giảng viên xác nhận (slide nêu SonTG/pp2022), clone, sửa README, commit First student commit, push vào fork. Nộp URL fork, hai hash commit, git remote -v và diff. Không cần cung cấp token/mật khẩu.

Môi trường: MANUAL_GIT. Challenge answer: answers/C03/answer.md; test_criteria.json khóa C03.

Gợi ý mở theo nhu cầu

1. Sửa README trước git add. URL là fork của người học; không đẩy lên repository gốc của giảng viên.

2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C03-K01 (remote); C03-K02 (evidence). Xem input/expected hoặc protocol trong test_criteria.json.

P01 | Tên biến và kết quả chương trình

Lộ trình chính | 25 phút dự kiến | Học trước: Không

Nguồn (số trang PDF): SL 28-35; HF 60-68. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Bạn đang làm màn hình chào cho Sổ học tập. Tên biến giúp chương trình dùng lại dữ liệu đã có; print đưa thông báo ra vùng kết quả. Đổi người đăng nhập thì thông báo phải đổi theo.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
name = "An"
message = "Chao " + name
print(message)
```

Dòng 1 gán name với chuỗi An. Dòng 2 ghép phần cố định và tên. Dòng 3 mới đưa kết quả ra màn hình; tạo biến chưa tự in gì.

S2 | Kiểm tra hiểu

1. Chương trình in gì?

- A. Chao An | Đúng. Đúng: name được đọc thành giá trị An.
 - B. Chao name | Chưa đúng. name ở ngoài dấu nháy nên là tên biến.
 - C. Không có gì | Chưa đúng. print ở dòng cuối tạo đầu ra.
2. Đổi name thành Binh thì cần sửa dòng nào?
- A. Chỉ print | Chưa đúng. print không quyết định giá trị của name.
 - B. Chỉ dòng 1 | Đúng. Đúng: hai dòng sau dùng lại giá trị.
 - C. Cả ba dòng | Chưa đúng. Không cần chép lại dữ liệu vào mọi dòng.

S3 | Làm có hỗ trợ rồi thay đổi

- Bấm khối name = "An", đặt vào dòng 1; mã trống -> có phép gán; chạy dòng này: bảng tên có name: "An", stdout vẫn rỗng.
- Đặt khối ghép chuỗi ở dòng 2; chạy: message chưa có -> "Chao An"; nối name với phần An trong kết quả.
- Đặt print(message) ở dòng 3; chạy: stdout rỗng -> "Chao An"; highlight dòng in, không đổi name.

Thử thay đổi: Đổi tên mẫu thành Chi, dự đoán rồi chạy lại từ đầu: stdout phải là Chao Chi. Hỏi vì sao không sửa print.

S4 | Thử thách độc lập

Biến name được cấp sẵn là chuỗi không rỗng. Viết script in đúng một dòng "Xin chào, <name>!". Không hard-code tên; không in thêm lời nhắc. Mỗi test cấp một name khác.

Môi trường: PY_CORE. Challenge answer: answers/P01/answer.md; test_criteria.json khóa P01.

Gợi ý mở theo nhu cầu

- Thông báo có phần cố định và phần thay đổi.
- Tên biến không đặt trong dấu nháy.
- Ghép "Xin chào, " + name + "!", rồi đưa vào print.

Phản hồi ngộ nhận: Nếu in ra chữ name: bạn đã đưa tên biến vào chuỗi; hãy so sánh "name" với name.

Nghiem thu bài học

Ví dụ công khai P01-a: {"name": "An"} -> "Xin chào, An!\n"

P02 | Chuỗi nhập vào và phép tính số

Lộ trình chính | 25 phút dự kiến | Học trước: P01

Nguồn (số trang PDF): SL 35-46; HF 74-75. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Quầy câu lạc bộ nhận số vé từ ô nhập. Ô nhập trả chuỗi dù nhìn giống số. Muốn tính tiền, bạn cần chuyển nó thành số trước khi nhân.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
quantity_text = "3"
quantity = int(quantity_text)
total = quantity * 20
```

"3" là str; int tạo số 3. Nhân số cho 20 tạo 60; nhân chuỗi cho số sẽ lặp ký tự, không tính tiền.

S2 | Kiểm tra hiểu

- "3" * 2 cho giá trị nào?
 - "5" | Chưa đúng. Phép nhân không cộng nội dung chuỗi.
 - 6 | Chưa đúng. Đây là kết quả nhân hai số, nhưng đầu vào là chuỗi.
 - "33" | Đúng. Đúng: nhân chuỗi lặp lại chuỗi.
- int("3") + 2 bằng bao nhiêu?
 - 5 | Đúng. Đúng: sau chuyển kiểu là phép cộng số.
 - "32" | Chưa đúng. Nối chuỗi không xảy ra sau int.
 - Lỗi | Chưa đúng. Chuỗi "3" là đầu vào hợp lệ của int.

S3 | Làm có hỗ trợ rồi thay đổi

- Đặt int(quantity_text) vào phép gán quantity; chạy: quantity_text giữ "3", quantity có 3 kèm nhãn int.
 - Chọn phép nhân * 20 thay khối + 20; chạy: total có 60; khối sai giải thích cộng khác nhân, chưa sửa dữ liệu.
 - Bấm xem kiểu: hiện str cạnh "3" và int cạnh 3/60; không dùng màu đơn độc để phân biệt.
- Thử thay đổi: Thay "3" bằng "0"; dự đoán total là 0. Hỏi chuỗi "0" và số 0 khác nhau ở đâu.

S4 | Thử thách độc lập

Đầu vào quantity_text là chuỗi số nguyên không âm; price là int không âm. Gán tổng tiền số nguyên vào result. Đề chưa yêu cầu xử lý chuỗi không hợp lệ.

Môi trường: PY_CORE. Challenge answer: answers/P02/answer.md; test_criteria.json khóa P02.

Gợi ý mở theo nhu cầu

- Kiểm tra kiểu đầu vào từ ô nhập.
- Chuyển số lượng trước khi nhân đơn giá.
- result = int(quantity_text) * price

Phản hồi ngộ nhận: Nếu kết quả là chuỗi lặp: phép nhân đang tác động vào str; kiểm tra bước int.

Nghiem thu bài học

Ví dụ công khai P02-a: {"quantity_text": "4", "price": 15} -> 60

C04 | Tên biến, kiểu động và ghi chú có ích

Lộ trình chính | 30 phút dự kiến | Học trước: P01, P02

Nguồn (số trang PDF): SL 32-33; SL 38-45; SL 50-51,61; HF 25-26,63,88; HF 199-200; GUI 499-560. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Tên biến trở tới đối tượng, nên có thể được gán sang kiểu khác. Tên rõ và docstring giúp người tiếp theo hiểu hợp đồng của hàm.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
value = "8"
print(type(value).__name__)
value = int(value)
print(type(value).__name__)
MAX_SCORE = 10
```

Hai dòng in lần lượt str và int. MAX_SCORE viết hoa là quy ước; Python vẫn cho phép gán lại nó.

S2 | Kiểm tra hiểu

1. Tên nào hợp lệ và dễ hiểu cho điểm trung bình?
 - A. 2score | Chưa đúng. Tên không bắt đầu bằng chữ số.
 - B. class | Chưa đúng. class là từ khóa.
 - C. mean_score | Đúng. Đúng: snake_case diễn đạt ý nghĩa.

S3 | Làm có hỗ trợ rồi thay đổi

1. Phân loại student_id, 2id, class, mean_score bằng str.isidentifier và keyword.iskeyword; hiển thị lý do từng tên.
 2. Theo dõi value từ chuỗi sang số: mũi tên tên biến đổi đối tượng đích; không mô tả chuỗi bị biến đổi tại chỗ.
 3. Điền docstring của hàm cộng điểm; thêm comment giải thích quy tắc giới hạn, không lặp lại từng dòng mã.
- Thử thay đổi: Gán MAX_SCORE=20 rồi chạy lại: giải thích khác nhau giữa quy ước constant và bất biến.

S4 | Thử thách độc lập

Viết valid_name(name) nhận str, trả True nếu là identifier và không là keyword; thêm docstring. Viết type_trace() trả ("str","int") bằng cách tự gán lại value="8" rồi int(value). Nộp một comment giải thích lý do dùng tên snake_case.

Môi trường: PY_CORE. Challenge answer: answers/C04/answer.md; test_criteria.json khóa C04.

Gợi ý mở theo nhu cầu

1. Hai dòng in lần lượt str và int. MAX_SCORE viết hoa là quy ước; Python vẫn cho phép gán lại nó.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: C04-K01 (names); C04-K02 (types). Xem input/expected hoặc protocol trong test_criteria.json.

Giải thích MAX_SCORE vẫn có thể gán lại; comment phải nêu lý do, không chỉ đọc lại câu lệnh.

C05 | Toán tử, thứ tự tính và làm tròn xuống

Lộ trình chính | 30 phút dự kiến | Học trước: P02

Nguồn (số trang PDF): SL 34-36; SL 150. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một biểu thức đúng cú pháp vẫn có thể tính sai nếu đặt ngoặc sai. Làm tròn xuống khác làm tròn tới số gần nhất.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
import math
print(2 + 3 * 4)
print((2 + 3) * 4)
print(math.floor(8.79 * 10) / 10)
```

Kết quả là 14, 20 và 8.7. floor hướng về âm vô cùng; với -1.21, floor(x*10)/10 cho -1.3.

S2 | Kiểm tra hiểu

1. 17 // 5 và 17 % 5 lần lượt là gì?

A. 3 và 2 | Đúng. Đúng: thương nguyên và phần dư.

B. 3.4 và 0 | Chưa đúng. / mới cho thương thực.

C. 2 và 3 | Chưa đúng. 17 = 5*3+2.

S3 | Làm có hỗ trợ rồi thay đổi

1. Ghép cây biểu thức 2+3*4; nhánh nhân chạy trước, ô 3*4 đổi thành 12.

2. Thay bằng //, %, ** và / trên dữ liệu 17,5; hiển thị cả giá trị và kiểu.

3. Điền math.floor(x*10)/10, thử 8.79 và -1.21; đối chiếu round(x,1).

Thử thay đổi: Đổi 8.79 thành 8.71: cả hai cùng floor thành 8.7 nhưng round có thể khác ở giá trị trước.

S4 | Thử thách độc lập

Viết arithmetic(a,b) cho hai int, b>0, trả dict add/sub/mul/div/quot/rem/power. Viết floor_tenth(x) cho float hữu hạn theo đúng math.floor(x*10)/10. Không gọi round thay floor.

Môi trường: PY_CORE. Challenge answer: answers/C05/answer.md; test_criteria.json khóa C05.

Gợi ý mở theo nhu cầu

1. Kết quả là 14, 20 và 8.7. floor hướng về âm vô cùng; với -1.21, floor(x*10)/10 cho -1.3.

2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: C05-K01 (operators); C05-K02 (floor). Xem input/expected hoặc protocol trong test_criteria.json.

P03 | Tách dữ liệu từ tên tệp

Lộ trình chính | 25 phút dự kiến | Học trước: P02

Nguồn (số trang PDF): SL 42-49; HF 137-178. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Tên tệp của câu lạc bộ chứa tên và môn tập, ví dụ An-swim.txt. Ta cần lấy hai phần để hiển thị. Hãy bỏ đúng phần đuôi trước, rồi tách tại dấu phân cách.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
filename = "An-swim.txt"
stem = filename.removesuffix(".txt")
parts = stem.split("-")
```

removesuffix trả chuỗi mới; filename không tự bị sửa. split trả list hai phần; phần đầu có chỉ số 0. Không gọi split tiếp trên cả list.

S2 | Kiểm tra hiểu

1. parts có giá trị nào?

A. ["An-swim.txt"] | Chưa đúng. Khi chỉ định -, chuỗi này được chia.

B. ["An", "swim"] | Đúng. Đúng: hậu tố đã bỏ và dấu - chia hai phần.

C. "An-swim" | Chưa đúng. Đó là stem, trước khi split.

2. Để bỏ đúng .txt, chọn gì?

A. split(".") luôn chỉ có hai phần | Chưa đúng. Tên có thể có thêm dấu chấm; không giả định chỉ một.

B. rstrip(".txt") | Chưa đúng. rstrip xử lý tập ký tự, có thể bỏ nhiều hơn hậu tố.

C. removesuffix(".txt") | Đúng. Đúng: ý định là bỏ hậu tố nguyên vẹn.

S3 | Làm có hồ trợ rồi thay đổi

1. Đặt removesuffix vào sau filename; chạy: stem chưa có -> "An-swim"; đánh dấu phần .txt được loại ở bản kết quả, giữ nguyên đầu vào.

2. Đặt split("-") sau stem; chạy: parts -> ["An", "swim"]; hai ô có chỉ số 0 và 1.

3. Chọn ô parts[0]; vùng khám phá hiện "An" và kiểu str; chọn cả parts hiện list.

Thử thay đổi: Thử "text-run.txt"; đối chiếu rstrip với removesuffix và tự giải thích ý nghĩa hậu tố, không bắt học thuộc hai đáp án.

S4 | Thử thách độc lập

filename luôn có dạng <name>-<activity>.txt; name/activity không chứa dấu -. Tạo result là list [name, activity]. Tên có thể chứa dấu chấm. Không sửa dữ liệu nguồn.

Môi trường: PY_CORE. Challenge answer: answers/P03/answer.md; test_criteria.json khóa P03.

Gợi ý mở theo nhu cầu

1. Làm hai phép biến đổi theo thứ tự.

2. Bỏ hậu tố chính xác trước khi chia theo -.

3. filename.removesuffix(".txt").split("-")

Phản hồi ngộ nhận: Nếu mất các chữ t/x cuối activity, bạn có thể đã dùng rstrip như lệnh xóa hậu tố.

Nghiem thu bài học

Ví dụ công khai P03-a: {"filename": "Mai-run.txt"} -> ["Mai", "run"]

C06 | Chuỗi: chuẩn hóa, định dạng và không sửa tại chỗ

Lộ trình chính | 30 phút dự kiến | Học trước: P03

Nguồn (số trang PDF): SL 49; SL 330; HF 130-136; HF 153-173. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Tên nhập từ form có thể thừa khoảng trắng và khác kiểu chữ. Các phương thức chuỗi tạo kết quả mới; cần giữ kết quả đó.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
raw = " aN "
clean = raw.strip().lower().title()
print(clean)
print(f"{23:04d}")
print("23".zfill(4))
```

clean là An, raw vẫn còn khoảng trắng. Hai cách cuối cho 0023; zfill làm việc với chuỗi.

S2 | Kiểm tra hiểu

1. raw.upper() không gán lại. raw có đổi không?
- A. Có | Chưa đúng. str là immutable.
- B. Không | Đúng. Đúng: cần dùng chuỗi được trả về.
- C. Chỉ ký tự đầu đổi | Chưa đúng. upper không phải title, và vẫn trả chuỗi mới.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chạy strip rồi lower rồi title ở ba dòng riêng; mỗi bước hiện chuỗi trước/sau trong ô mới.
 2. Gộp thành chain và đổi chiều; không thay đổi thứ tự các phương thức khi chưa chứng minh tương đương.
 3. So f-string với "{: .1f}".format(name,score); chỉ tô phần placeholder được thay.
- Thử thay đổi: Thay newline LF bằng CRLF trong danh sách tên, dùng splitlines để vẫn nhận các dòng.

S4 | Thử thách độc lập

Viết labels(text) nhận nhiều dòng tên, bỏ dòng trống sau strip; chuẩn hóa title và trả list dạng "001: An", "002: Binh".
Viết score_label(name,score) dùng format hoặc f-string trả "An: 8.5" với một chữ số thập phân. Không sửa text nguồn.

Môi trường: PY_CORE. Challenge answer: answers/C06/answer.md; test_criteria.json khóa C06.

Gợi ý mở theo nhu cầu

1. clean là An, raw vẫn còn khoảng trắng. Hai cách cuối cho 0023; zfill làm việc với chuỗi.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: C06-K01 (lines); C06-K02 (format). Xem input/expected hoặc protocol trong test_criteria.json.

Học viên tách một chain thành ba biến trung gian và giải thích cùng kết quả; so lower, upper và title trên aN.

P04 | Điều kiện và giá trị ở ranh giới

Lộ trình chính | 25 phút dự kiến | Học trước: P02

Nguồn (số trang PDF): SL 52-57. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Bảng điểm cần hiển thị một nhãn theo điều kiện. Trong bài giả lập, điểm từ 5 trở lên là Dat. Máy chỉ chọn một nhánh dựa trên kết quả True/False của phép so sánh.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
score = 5
if score >= 5:
    label = "Dat"
else:
    label = "Chưa dat"
```

Vì 5 >= 5 là True, chỉ nhánh if gán label. Thử để xác định dòng thuộc nhánh nào.

S2 | Kiểm tra hiểu

1. score=5 cho nhãn gì?

- A. Dat | Đúng. Đúng: >= có bao gồm dấu bằng.
- B. Chưa dat | Chưa đúng. Bạn đã bỏ qua trường hợp bằng ngưỡng.
- C. Cả hai | Chưa đúng. if/else chọn một nhánh ở lần chạy này.

2. Muốn chỉ số lớn hơn 5, dùng gì?

- A. >= 5 | Chưa đúng. >= còn nhận cả số 5.
- B. > 5 | Đúng. Đúng: > loại giá trị bằng 5.
- C. = 5 | Chưa đúng. = dùng cho phép gán, không phải so sánh này.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt score >= 5 vào ô điều kiện; chạy: condition -> True; đường nhánh Dat có nhãn Được chọn.

2. Đặt label = "Dat" trong nhánh if với 4 dấu cách; bước chạy đổi label chưa có -> "Dat".

3. Đặt nhánh else; khi điều kiện True, nhánh này ghi Không chạy, không tạo một lần gán thứ hai.

Thử thay đổi: Đổi score thành 4.9 rồi 5.1; dự đoán nhánh mỗi lần. Hỏi vì sao test đúng 5 vẫn cần có.

S4 | Thử thách độc lập

Ngữ cảnh mới: fee là số không âm. Nếu fee >= 100 thì result = "Free", ngược lại "Paid". Kiểu result là str. Đây là quy ước bài tập giao hàng.

Môi trường: PY_CORE. Challenge answer: answers/P04/answer.md; test_criteria.json khóa P04.

Gợi ý mở theo nhu cầu

- 1. Xác định ranh giới và nó thuộc nhóm nào.
- 2. Điều kiện cần nhận cả 100.
- 3. Dùng fee >= 100; mỗi nhánh gán result một chuỗi.

Phản hồi ngộ nhận: Nếu 100 bị Paid: điều kiện của bạn có bỏ mất dấu bằng không?

Nghệ thuật bài học

Ví dụ công khai P04-a: {"fee": 120} -> "Free"

P05 | List và việc hai tên dùng chung dữ liệu

Lộ trình chính | 25 phút dự kiến | Học trước: P02

Nguồn (số trang PDF): SL 67-78; HF 76-79, 235-244. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Bạn muốn thử thêm một điểm mà chưa thay danh sách chính. Gán một list cho tên mới không tự sao chép nó. Cần một danh sách mới nếu thay đổi không được ảnh hưởng bản gốc.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
scores = [6, 8]
draft = scores.copy()
draft.append(9)
```

copy tạo list ngoài mới. append thay draft và trả None, không trả list vừa thay. Trong bài chỉ có các số nên bản sao nông là đủ.

S2 | Kiểm tra hiểu

1. scores sau ba dòng là gì?
 - A. [6,8,9] | Chưa đúng. Đó là draft; scores được giữ nguyên.
 - B. None | Chưa đúng. append trả None nhưng không gán nó vào scores.
 - C. [6,8] | Đúng. Đúng: append tác động lên bản sao.
2. Nếu draft = scores thì append sẽ ảnh hưởng gì?
 - A. Cả hai tên thấy thay đổi | Đúng. Đúng: hai tên cùng tham chiếu đối tượng.
 - B. Không tên nào | Chưa đúng. append thay đổi list đó.
 - C. Chỉ draft | Chưa đúng. Hai tên đang dùng chung một list.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chọn khối copy thay phép gán trực tiếp; chạy: hai nút list riêng xuất hiện, cùng dữ liệu [6,8].
 2. Đặt append(9) lên draft; chạy: draft tăng 2 -> 3 ô; scores giữ 2 ô.
 3. Thử nhánh minh họa draft = scores trong vùng thử; một nút list có hai mũi nối; app ghi rõ đây là lần chạy mới.
- Thử thay đổi: Đổi điểm thêm thành 0; dự đoán cả scores và draft, không chỉ list sau thêm.

S4 | Thử thách độc lập

scores là list số; extra là số. Tạo result là bản sao scores có extra ở cuối, giữ nguyên scores kể cả khi rỗng. Không có list lồng nhau.

Môi trường: PY_CORE. Challenge answer: answers/P05/answer.md; test_criteria.json khóa P05.

Gợi ý mở theo nhu cầu

1. Bạn cần bảo vệ danh sách đầu vào.
2. Tạo list mới rồi thêm phần tử.
3. result = scores.copy(); sau đó result.append(extra).

Phản hồi ngộ nhận: Nếu result là None: bạn có gán kết quả của append không? Nếu scores bị đổi: kiểm tra bản sao.

Nghiem thu bài học

Ví dụ công khai P05-a: {"scores": [6, 8], "extra": 9} -> [[6, 8, 9], [6, 8]]

C07 | Index và slice: chọn đúng phần dữ liệu

Lộ trình chính | 30 phút dự kiến | Học trước: P03, P05

Nguồn (số trang PDF): SL 47-48; SL 71-76; HF 166-173. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Index lấy một phần tử; slice lấy một đoạn với điểm dừng không bao gồm. Chỉ số âm đếm từ cuối.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
items = [10,20,30,40,50]
print(items[-1], items[1:4:2])
items[1:3] = [99]
print(items)
```

In 50 và [20,40]; sau gán đoạn, list là [10,99,40,50]. Gán slice có thể đổi chiều dài list.

S2 | Kiểm tra hiểu

1. "Python"[1:4] là gì?

A. yth | Đúng. Đúng: các index 1,2,3.

B. ytho | Chưa đúng. Index 4 bị loại bởi stop.

C. Pyt | Chưa đúng. Start là 1, không phải 0.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tô index 0..4 dưới từng ô rồi thêm -5..-1 ở hàng dưới.

2. Chọn start=1, stop=4, step=2; chỉ ô 20 và 40 sáng.

3. Thay đoạn [1:3] bằng [99]; hai ô bị thay bởi một ô, cập nhật chỉ số.

Thử thay đổi: Thử slice vượt cuối và index vượt cuối; giải thích vì sao một cái trả đoạn ngắn còn cái kia IndexError.

S4 | Thử thách độc lập

Viết select_parts(text,items) trả (text[:3], text[::-1], items[-3:], items[1::2]). Viết replace_middle(items) trả bản sao với đoạn [1:3] thay bằng [99], không sửa đầu vào.

Môi trường: PY_CORE. Challenge answer: answers/C07/answer.md; test_criteria.json khóa C07.

Gợi ý mở theo nhu cầu

1. In 50 và [20,40]; sau gán đoạn, list là [10,99,40,50]. Gán slice có thể đổi chiều dài list.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: C07-K01 (slice); C07-K02 (mutation). Xem input/expected hoặc protocol trong test_criteria.json.

C08 | Thêm, xóa và đảo list

Lộ trình chính | 30 phút dự kiến | Học trước: P05, C07

Nguồn (số trang PDF): SL 68-70; SL 77-78. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

append thêm một đối tượng; extend thêm từng phần tử. Xóa theo giá trị và xóa theo vị trí là hai yêu cầu khác nhau.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
a=[1]; a.append([2,3])
b=[1]; b.extend([2,3])
print(a,b)
print(b.pop(1))
print(b)
```

a thành [1,[2,3]], b thành [1,2,3]. pop(1) trả 2 và đồng thời xóa phần tử đó.

S2 | Kiểm tra hiểu

1. remove(2) trên [2,5,2] để lại gì?

A. [5] | Chưa đúng. remove chỉ xóa lần xuất hiện đầu.

B. [5,2] | Đúng. Đúng.

C. [2,5] | Chưa đúng. Đây là kết quả pop cuối, không phải remove(2).

S3 | Làm có hỗ trợ rồi thay đổi

1. Kéo list [2,3] vào append: hiện một ô list lồng; đổi sang extend: hiện hai ô số.

2. Đánh dấu lần xuất hiện đầu của giá trị 2 rồi remove; các ô còn lại dồn trái.

3. Thực hiện pop, del và reverse; ghi riêng giá trị trả về và list sau thao tác.

Thử thay đổi: Thử remove với giá trị không có: dự đoán ValueError; trước thao tác cần kiểm tra membership nếu hợp đồng muốn bỏ qua.

S4 | Thử thách độc lập

Viết edit(values,extras,target): sao chép values; extend extras; nếu target có trong list thì remove lần đầu; pop phần tử cuối nếu còn (rỗng trả None); reverse phần còn lại; trả (list_mới,phần_tử_pop).

Môi trường: PY_CORE. Challenge answer: answers/C08/answer.md; test_criteria.json khóa C08.

Gợi ý mở theo nhu cầu

1. a thành [1,[2,3]], b thành [1,2,3]. pop(1) trả 2 và đồng thời xóa phần tử đó.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghệ thuật bài học

Các ca bắt buộc: C08-K01 (normal); C08-K02 (empty). Xem input/expected hoặc protocol trong test_criteria.json.

Trên [10,20,30], học viên thực hiện del items[1] ra [10,30] và phân biệt với remove(1) gây ValueError.

P06 | Dictionary và tra cứu theo khóa

Lộ trình chính | 25 phút dự kiến | Học trước: P05

Nguồn (số trang PDF): SL 83-87; HF 81-83. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Hai bạn có thể trùng tên. Sổ học tập dùng mã sinh viên để tra điểm. Dictionary nối mỗi khóa với một giá trị; thiếu khóa cần có cách phản hồi rõ.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
marks = {"S01": 8, "S02": 5}
score = marks.get("S03", None)
```

S03 không có nên get trả None. Điểm 0 vẫn là dữ liệu hợp lệ, không được nhầm với thiếu điểm. Khóa trùng trong dict không giữ hai bản ghi riêng.

S2 | Kiểm tra hiểu

1. score bằng gì?

A. 0 | Chưa đúng. Chưa có điểm không đồng nghĩa điểm 0.

B. None | Đúng. Đúng: trả mặc định vì khóa không tồn tại.

C. 5 | Chưa đúng. Không tự lấy giá trị cuối cùng.

2. marks có S01:0; get("S01",None) trả gì?

A. Lỗi | Chưa đúng. get đọc được khóa này.

B. None | Chưa đúng. Khóa tồn tại nên không dùng mặc định.

C. 0 | Đúng. Đúng: 0 là giá trị hợp lệ.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chọn khóa S01; hàng tương ứng nổi viền, value preview -> 8.

2. Đặt .get(student_id, None) vào mã; chạy với S03: lookup ghi Không tìm thấy -> score=None.

3. Thử khóa có giá trị 0; highlight tìm thấy, score -> 0; nhấn khác trạng thái thiếu.

Thử thay đổi: Thêm S03:0 vào dữ liệu rồi chạy lại; tự giải thích vì sao kết quả đổi từ None sang 0.

S4 | Thử thách độc lập

marks là dict mã -> điểm số, student_id là str. Gán result = điểm nếu có mã, nếu không thì None. Không thay marks. Không tra theo tên hay vị trí.

Môi trường: PY_CORE. Challenge answer: answers/P06/answer.md; test_criteria.json khóa P06.

Gợi ý mở theo nhu cầu

1. Bài toán là tra bằng khóa.

2. Thiếu khóa phải khác điểm 0.

3. marks.get(student_id, None)

Phản hồi ngộ nhận: Nếu điểm 0 bị coi là thiếu: đừng dùng điều kiện truthiness để suy ra khóa có tồn tại.

Nghiem thu bài học

Ví dụ công khai P06-a: {"marks": {"S01": 8}, "student_id": "S01"} -> 8

C09 | Dictionary: duyệt, xóa và tìm cực trị

Lộ trình chính | 30 phút dự kiến | Học trước: P06

Nguồn (số trang PDF): SL 85-87; SL 330; HF 86-87. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Duyệt dictionary mặc định đi qua khóa. Muốn xét cả mã và điểm, dùng items; muốn tìm mã điểm cao nhất, so theo giá trị.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
marks={"A":8,"B":6}
print(list(marks.keys()))
print(list(marks.values()))
print(list(marks.items()))
print("A" in marks, 8 in marks)
```

Ba view biểu diễn khóa, giá trị và cặp; membership mặc định kiểm tra khóa nên kết quả cuối True, False.

S2 | Kiểm tra hiểu

1. max(marks) có chắc là mã có điểm cao nhất không?
- A. Có | Chưa đúng. max chưa nhận key để so theo điểm.
- B. Không, mặc định so khóa | Đúng. Đúng: dùng key=marks.get nếu muốn so giá trị.
- C. Luôn trả điểm | Chưa đúng. Kết quả vẫn là một khóa.

S3 | Làm có hỗ trợ rồi thay đổi

1. Nói mỗi key với value; chuyển tab keys/values/items và giữ highlight cùng cặp.
 2. Xóa mã bằng pop(id,None), hiện phần bị lấy ra; xóa lại mã đó thì None.
 3. Tìm min/max theo điểm; thử hòa điểm và công bố quy tắc phụ.
- Thử thay đổi: So "ab" in "abc", 8 in [8], 8 in {8}, 8 in {"A":8}; giải thích membership của dict.

S4 | Thử thách độc lập

Viết summarize(marks,remove_id): không sửa marks; pop mã nếu có; trả {remaining:dict, removed:value_or_None, best:id_or_None,worst:id_or_None}. Nếu hòa điểm chọn mã nhỏ nhất theo thứ tự chuỗi.

Môi trường: PY_CORE. Challenge answer: answers/C09/answer.md; test_criteria.json khóa C09.

Gợi ý mở theo nhu cầu

1. Ba view biểu diễn khóa, giá trị và cặp; membership mặc định kiểm tra khóa nên kết quả cuối True, False.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C09-K01 (ties); C09-K02 (zero_empty). Xem input/expected hoặc protocol trong test_criteria.json.

Yêu cầu người học tự in keys, values, items trong guided; kiểm tra đầy đủ cặp dù cách trình bày thứ tự khác.

P07 | Chọn tuple và set đúng mục đích

Lộ trình chính | 25 phút dự kiến | Học trước: P05, P06

Nguồn (số trang PDF): SL 65, 81-82; HF 78-85, 174-182. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Danh sách đăng ký có nhiều lượt nhưng một người có thể đăng ký lại. Báo cáo cần cả số lượt và số người khác nhau. Một tuple hai giá trị giúp chuyển hai kết quả cùng lúc.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
ids = ["S01", "S01", "S02"]
unique_ids = set(ids)
report = (len(ids), len(unique_ids))
```

`len(ids)=3`, `len(unique_ids)=2`. Set dùng để bỏ trùng, không được dùng thứ tự hiển thị set làm ý nghĩa. Tuple giữ hai vị trí của báo cáo.

S2 | Kiểm tra hiểu

1. report là gì?

A. (3,2) | Đúng. Đúng: số lượt trước, số mã khác nhau sau.

B. (2,3) | Chưa đúng. Bạn đã đổi thứ tự hai trường.

C. (3,3) | Chưa đúng. Set loại mã lặp.

2. Có nên lấy "mã đầu tiên" của set để xếp hạng?

A. Có, luôn ổn định | Chưa đúng. Không dựa vào thứ tự set cho hợp đồng.

B. Không, set không là thứ tự xếp hạng | Đúng. Đúng: xếp hạng cần thứ tự được quy định.

C. Chỉ khi có 2 phần tử | Chưa đúng. Số phần tử không tạo ý nghĩa thứ tự.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chọn 3 thẻ đăng ký; vùng list giữ cả hai S01.

2. Đặt `set(ids)`; khi chạy, vùng set còn hai nhãn duy nhất, ghi Không có thứ tự xếp hạng.

3. Đặt hai len vào tuple; chạy: report có ô 0=3, ô 1=2; đổi chỗ ô báo ý nghĩa khác.

Thử thay đổi: Đổi ids thành [] rồi dự đoán report; kiểm tra (0,0).

S4 | Thử thách độc lập

ids là list chuỗi. Tạo result là tuple (tổng số lượt, số mã khác nhau). Chấp nhận đầu vào rỗng. Không thay ids.

Môi trường: PY_CORE. Challenge answer: `answers/P07/answer.md`; `test_criteria.json` khóa P07.

Gợi ý mở theo nhu cầu

1. Hai số đo có ý nghĩa khác nhau.

2. Set giúp đếm số mã khác nhau.

3. Tạo tuple từ `len(ids)` và `len(set(ids))`.

Phản hồi ngộ nhận: Nếu hai số luôn bằng nhau: bạn đang đếm trên cùng một collection, bỏ mất yêu cầu phân biệt lượt và người.

Nghiem thu bài học

Ví dụ công khai P07-a: `{"ids": ["A", "A", "B"]}` -> [3, 2]

C10 | Tuple, unpacking và lựa chọn collection

Lộ trình chính | 30 phút dự kiến | Học trước: P07

Nguồn (số trang PDF): SL 64-67,81; SL 81-82; HF 78-80; HF 174-182. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Tuple giữ nhóm giá trị theo vị trí và không cho thay phần tử trực tiếp. Unpacking gán từng vị trí với tên có ý nghĩa.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
record=("An", 19, "50m")
name, age, distance=record
print(name, age, distance)
```

Ba biến nhận ba phần tử. Nếu số biến không khớp số phần tử sẽ ValueError; gán record[1]=20 sẽ TypeError.

S2 | Kiểm tra hiểu

1. Tuple chứa một list thì list bên trong có thể append không?
- A. Có | Đúng. Đúng: tuple cố định tham chiếu phần tử, không làm mọi đối tượng bên trong bất biến.
- B. Không vì tuple bất biến | Chưa đúng. Bất biến không tự lan vào list lồng.
- C. Chỉ được nếu tuple dài 1 | Chưa đúng. Độ dài tuple không quyết định.

S3 | Làm có hỗ trợ rồi thay đổi

1. Gán ba thẻ tên vào ba vị trí; chạy unpack và hiện các biến.
 2. Bỏ một biến để quan sát ValueError, rồi sửa đủ số biến.
 3. So tuple hồ sơ, list lượt bơi, dict tra theo mã và set mã duy nhất; chọn theo yêu cầu.
- Thử thay đổi: Thay record bằng ("Binh", 20, "100m"); dự đoán biến nào đổi. Thử sửa tuổi bằng tạo tuple mới.

S4 | Thử thách độc lập

Viết birthday(record) nhận tuple (name, age, distance), age int >= 0, dùng unpack trả tuple tuổi tăng 1; không sửa đầu vào. Giải thích vì sao dùng list cho danh sách lượt bơi có thể thêm mới.

Môi trường: PY_CORE. Challenge answer: answers/C10/answer.md; test_criteria.json khóa C10.

Gợi ý mở theo nhu cầu

1. Ba biến nhận ba phần tử. Nếu số biến không khớp số phần tử sẽ ValueError; gán record[1]=20 sẽ TypeError.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C10-K01 (unpack); C10-K02 (zero). Xem input/expected hoặc protocol trong test_criteria.json.

Học viên dự đoán rồi xác nhận TypeError cho gán phần tử tuple, ValueError cho unpack sai số lượng; phân loại đúng cả bốn collection.

C17 | Tên file thành hồ sơ người bơi

Lộ trình chính | 30 phút dự kiến | Học trước: C07, C10, P03

Nguồn (số trang PDF): HF 103-118; HF 183-186. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Trước khi tính thời gian, bạn cần biết dữ liệu thuộc ai và môn nào. Chia công việc thành đọc tên file, đọc nội dung, tính toán và trình bày giúp tìm lỗi đúng chỗ.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
filename="An-19-50m-Free.txt"
name,age,distance,stroke=filename.removesuffix(".txt").split("-")
print(name,int(age),distance,stroke)
```

Bốn trường có nghĩa khác nhau; age được chuyển int, distance giữ đơn vị m trong chuỗi.

S2 | Kiểm tra hiểu

1. Tên An-19-50m-Free.txt có lỗi tuổi thì sửa ở bước nào?
- A. Tính trung bình lượt bơi | Chưa đúng. Bước tính lượt không đọc tuổi.
- B. Phân tích metadata | Đúng. Đúng: xác thực từng trường ngay sau tách tên file.
- C. Vẽ biểu đồ | Chưa đúng. Biểu đồ không sửa metadata.

S3 | Làm có hỗ trợ rồi thay đổi

1. Sắp bốn nhiệm vụ metadata -> read -> mean -> format; nối đầu ra bước trước vào đầu vào bước sau.
 2. Tách .txt bằng removesuffix rồi split; bốn thẻ tên nhận bốn giá trị.
 3. Đổi age từ str sang int; thử tên thiếu trường và hiển thị lỗi có ngữ cảnh.
- Thử thay đổi: Thay An bằng tên có dấu chấm; giải thích vì sao không split toàn tên theo dấu chấm.

S4 | Thử thách độc lập

Viết parse_filename(filename) cho basename dạng name-age-distance-stroke.txt, các trường không chứa dấu -, tuổi int>=0, distance là số nguyên dương theo sau m, name/stroke không rỗng. Sai raise ValueError. Trả dict name,age,distance,stroke. Nộp sơ đồ phân rã pipeline bốn bước.

Môi trường: PY_CORE. Challenge answer: answers/C17/answer.md; test_criteria.json khóa C17.

Gợi ý mở theo nhu cầu

1. Bốn trường có nghĩa khác nhau; age được chuyển int, distance giữ đơn vị m trong chuỗi.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: C17-K01 (metadata); C17-K02 (invalid). Xem input/expected hoặc protocol trong test_criteria.json.

Sơ đồ có đầu vào/đầu ra từng bước; không lẫn tuổi với dữ liệu lượt bơi.

P08 | Vòng for và biến tích lũy

Lộ trình chính | 25 phút dự kiến | Học trước: P04, P05

Nguồn (số trang PDF): SL 88-94; HF 224-244. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một lớp có nhiều điểm. Bạn cần biết có bao nhiêu điểm từ 5 trở lên theo quy ước bài tập. Bộ đếm bắt đầu từ 0 và chỉ tăng khi phần tử hiện tại đạt điều kiện.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
scores = [4, 5, 8]
count = 0
for score in scores:
    if score >= 5:
        count += 1
```

Ba lượt score lần lượt là 4,5,8. count qua từng lượt là 0,1,2. Không đặt count=0 trong vòng lặp vì sẽ xóa kết quả tích lũy.

S2 | Kiểm tra hiểu

1. count cuối bằng bao nhiêu?

- A. 1 | Chưa đúng. Bạn có thể đã quên tính đúng bằng 5.
- B. 3 | Chưa đúng. 3 là số phần tử, không phải số điểm đạt.
- C. 2 | Đúng. Đúng: 5 và 8 đạt.

2. Đặt count=0 bên trong for gây gì?

- A. Xóa kết quả trước mỗi lượt | Đúng. Đúng: bộ đếm không còn cộng dồn.
- B. Không ảnh hưởng | Chưa đúng. Vị trí khởi tạo ảnh hưởng kết quả.
- C. Luôn nhanh hơn | Chưa đúng. Tốc độ không sửa được lỗi logic.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt count=0 trước for; chạy: count chưa có -> 0, bảng trace chưa có lượt.

2. Đặt if score>=5 trong for; chạy lượt score=4: False, count 0->0; thêm dòng trace [4,False,0].

3. Đặt count += 1 vào if; chạy tiếp: trace [5,True,1], [8,True,2]; dòng đang chạy nối với phần tử tương ứng.

Thử thay đổi: Đổi danh sách thành [5,5,0]; dự đoán count=2 và giải thích vì sao điểm lặp vẫn được đếm hai lần.

S4 | Thử thách độc lập

Ngữ cảnh mới: stocks là list số nguyên không âm. Dùng for để đếm kho có ít nhất 10 món; gán vào result. Danh sách rỗng cho 0. Mục tiêu công khai là dùng vòng for.

Môi trường: PY_CORE. Challenge answer: answers/P08/answer.md; test_criteria.json khóa P08.

Gợi ý mở theo nhu cầu

- 1. Tạo bộ đếm trước khi duyệt.
- 2. Mỗi kho đạt điều kiện chỉ tăng một lần.
- 3. Trong for stock in stocks, nếu stock >= 10 thì result += 1.

Phản hồi ngộ nhận: Nếu kết quả chỉ còn 0 hoặc 1: kiểm tra bạn có khởi tạo lại bộ đếm trong for không.

Nghệ thuật bài học

Ví dụ công khai P08-a: {"stocks": [9, 10, 14]} -> 2

C14 | Comprehension, enumerate, zip và Counter

Lộ trình chính | 30 phút dự kiến | Học trước: P08, C09

Nguồn (số trang PDF): SL 330. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Cùng một tập dữ liệu có thể được lọc, đánh số hoặc ghép theo vị trí. Hãy chọn phép biến đổi theo đầu ra cần có.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
from collections import Counter
print([x*x for x in [-2,0,3] if x>0])
print(list(enumerate(["A", "B"],1)))
print(list(zip(["A", "B"], [8,9])))
print(Counter(["A", "B", "A"]).most_common(1))
```

Kết quả là [9], [(1,A),(2,B)], [(A,8),(B,9)] và [(A,2)] với các tên là chuỗi.

S2 | Kiểm tra hiểu

1. zip hai list độ dài 3 và 2 mặc định tạo bao nhiêu cặp?

A. 2 | Đúng. Đúng: dừng ở iterable ngắn hơn.

B. 3 | Chưa đúng. Không tự thêm phần tử thiếu.

C. Báo lỗi | Chưa đúng. Muốn bắt mismatch có thể dùng strict, nhưng mặc định không lỗi.

S3 | Làm có hỗ trợ rồi thay đổi

1. Biến vòng for + if + append thành comprehension, giữ cùng thứ tự.

2. Ghép mã và điểm bằng zip; thêm thứ hạng bằng enumerate bắt đầu 1.

3. Đếm phiếu bằng Counter; so số lượt với số mã duy nhất.

Thử thay đổi: Thêm một mã nhưng không thêm điểm; dự đoán cặp nào bị mất và bổ sung kiểm tra độ dài trước zip.

S4 | Thử thách độc lập

Viết report(ids,scores): nếu độ dài khác nhau raise ValueError; trả list (thứ_tự,mã,điểm) chỉ với điểm>=5, thứ tự đánh lại từ 1. Viết top_votes(ids,k) trả Counter(ids).most_common(k), k>=0; hòa tần suất giữ thứ tự gặp đầu.

Môi trường: PY_CORE. Challenge answer: answers/C14/answer.md; test_criteria.json khóa C14.

Gợi ý mở theo nhu cầu

1. Kết quả là [9], [(1,A),(2,B)], [(A,8),(B,9)] và [(A,2)] với các tên là chuỗi.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịem thu bài học

Các ca bắt buộc: C14-K01 (filter); C14-K02 (count); C14-K03 (mismatch). Xem input/expected hoặc protocol trong test_criteria.json.

P09 | While, sentinel và điều kiện dừng

Lộ trình chính | 25 phút dự kiến | Học trước: P08

Nguồn (số trang PDF): SL 90-93; HF 228-230. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một lô nhập điểm dừng -1 để báo đã kết thúc. Điểm sau dấu này không thuộc lô. Vòng while cần điều kiện tiếp tục và một cách tiến tới lúc dừng.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
values = [6, 8, -1, 9]
i = 0
total = 0
while i < len(values) and values[i] != -1:
    total += values[i]
    i += 1
```

Kiểm tra chỉ số trước để không đọc ngoài list. Lượt đầu cộng 6, lượt sau cộng 8; khi thấy -1 dừng nên 9 không được cộng. and đánh giá từ trái sang phải, về sau không cần chạy khi về trước False.

S2 | Kiểm tra hiểu

1. total cuối là gì?

A. 23 | Chưa đúng. Bạn đã cộng cả dữ liệu sau dấu kết thúc.

B. 14 | Đúng. Đúng: chỉ cộng 6 và 8.

C. 13 | Chưa đúng. -1 là dấu kết thúc, không là điểm để cộng.

2. Nếu quên i += 1?

A. Tự tăng i | Chưa đúng. while không tự tăng biến chỉ số.

B. Luôn dừng ngay | Chưa đúng. Điều kiện không tự đổi sang False.

C. Có thể lặp mãi ở phần tử đầu | Đúng. Đúng: i không tiến nên vẫn đọc cùng phần tử.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt điều kiện i < len(values) ở về trái and; chạy kiểm tra i=0 hợp lệ, rồi đọc values[0].

2. Đặt cộng tổng trước tăng i; trace lượt 1 [i=0,value=6,total=6], lượt 2 [i=1,value=8,total=14].

3. Chạy kiểm tra i=2: value=-1, condition=False; con trở ra sau while, total giữ 14.

Thử thay đổi: Đổi thành [-1,6] và []; cả hai total=0. Giải thích vì sao cần cả kiểm tra hết list.

S4 | Thử thách độc lập

values là list số nguyên không âm, có thể có -1 làm sentinel. Dùng while tính tổng trước -1 đầu tiên, hoặc toàn bộ nếu không có; result là int. List rỗng cho 0.

Môi trường: PY_CORE. Challenge answer: answers/P09/answer.md; test_criteria.json khóa P09.

Gợi ý mở theo nhu cầu

1. Xác định hai lý do phải dừng.

2. Kiểm tra i trong phạm vi trước khi đọc values[i].

3. Điều kiện i < len(values) and values[i] != -1; tăng i mỗi lượt.

Phản hồi ngộ nhận: Nếu IndexError ở list rỗng: bạn có đọc phần tử trước khi kiểm tra chỉ số không?

Nghiệm thu bài học

Ví dụ công khai P09-a: {"values": [3, 4, -1, 9]} -> 7

C11 | range, break và continue

Lộ trình chính | 30 phút dự kiến | Học trước: P08, P09

Nguồn (số trang PDF): SL 79-80,93; SL 91-92. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

range sinh các vị trí theo bước nhảy và không chứa stop. continue bỏ phần còn lại của lượt hiện tại; break kết thúc cả vòng.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
for n in range(1,8):
    if n==6: break
    if n%2==0: continue
    print(n)
```

Chương trình in 1,3,5. Đến 6 gặp break nên không xét 7.

S2 | Kiểm tra hiểu

1. list(range(5,0,-2)) là gì?

A. [5,3,1] | Đúng. Đúng: bước âm đi về stop nhưng không chứa stop.

B. [5,3,1,0] | Chưa đúng. Bước -2 không chạm 0, stop cũng bị loại.

C. [] | Chưa đúng. Start lớn hơn stop phù hợp bước âm.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt start/stop/step trên trục số; tô các giá trị sinh ra.

2. Chạy từng lượt ví dụ; continue làm con trở sang lượt tiếp, break đưa con trở ra ngoài vòng.

3. Đổi break thành continue ở n=6; dự đoán sẽ có thêm 7.

Thử thay đổi: Đổi step thành 0; giải thích ValueError và vì sao không thể tiến tới điểm dừng.

S4 | Thử thách độc lập

Viết collect(start,stop,step,blocked): dùng range; gặp giá trị blocked thì break; bỏ số chia hết cho 3 bằng continue; trả các giá trị còn lại. step khác 0; blocked có thể không xuất hiện.

Môi trường: PY_CORE. Challenge answer: answers/C11/answer.md; test_criteria.json khóa C11.

Gợi ý mở theo nhu cầu

1. Chương trình in 1,3,5. Đến 6 gặp break nên không xét 7.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C11-K01 (forward); C11-K02 (backward). Xem input/expected hoặc protocol trong test_criteria.json.

P10 | Hàm trả về giá trị

Lộ trình chính | 25 phút dự kiến | Học trước: P08, P09

Nguồn (số trang PDF): SL 58-62; HF 25-43. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Bảng điểm và biểu đồ cùng cần tính trung bình. Đưa phép tính vào một hàm để cả hai dùng lại. Hàm nhận dữ liệu qua tham số và đưa kết quả lại cho nơi gọi bằng return.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
def mean_score(scores):
    if not scores:
        return None
    return sum(scores) / len(scores)
```

```
answer = mean_score([6, 8])
```

sum và len là công cụ đã có; bài tập trung vào tham số/return. List rỗng được xử lý trước phép chia. print chỉ hiển thị, không thay thế giá trị trả về.

S2 | Kiểm tra hiểu

1. answer là gì?

A. 7.0 | Đúng. Đúng: hàm trả số trung bình.

B. None | Chưa đúng. Hàm chỉ trả None với list rỗng.

C. Chuỗi "7.0" | Chưa đúng. Không có bước chuyển sang chuỗi.

2. Thay return bằng print ở cuối thì nơi gọi nhận gì?

A. Số vừa in | Chưa đúng. In ra không chuyển số thành return.

B. None | Đúng. Đúng: hàm kết thúc không return giá trị sẽ trả None.

C. List ban đầu | Chưa đúng. Tham số không tự được trả về.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt tham số scores trong def; gọi với [6,8]: tạo khung lời gọi có scores, không đổi list nguồn.

2. Đặt guard list rỗng trước phép chia; lần này False, chạy sum=14 và len=2.

3. Đặt return; giá trị 7.0 đi về answer, khung hàm kết thúc; stdout vẫn rỗng.

Thử thay đổi: Gọi hàm lần lượt với [0,10] và []; kết quả 5.0, None. Hỏi vì sao gọi lần hai không dùng dữ liệu lần một.

S4 | Thử thách độc lập

Viết mean_score(scores) nhận list số trong [0,10], trả trung bình số học hoặc None nếu rỗng. Không sửa scores, không in. Sai số float cho phép 1e-9.

Môi trường: PY_CORE. Challenge answer: answers/P10/answer.md; test_criteria.json khóa P10.

Gợi ý mở theo nhu cầu

1. Tách rõ dữ liệu vào và kết quả trả về.

2. Xử lý rỗng trước khi chia.

3. if not scores: return None; còn lại return sum(scores)/len(scores).

Phản hồi ngộ nhận: Nếu màn hình có số nhưng test nhận None: hãy kiểm tra return thay vì print.

Nghiem thu bài học

Ví dụ công khai P10-a: {} -> 7.0

C13 | Khám phá API, traceback và module chuẩn

Lộ trình chính | 30 phút dự kiến | Học trước: P03, P10

Nguồn (số trang PDF): HF 51-60; HF 61-73; HF 151-155. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Khi không nhớ tên phương thức, hãy kiểm tra kiểu, danh sách thuộc tính và trợ giúp. Traceback cho biết đường đi tới lỗi; dòng cuối mô tả loại lỗi.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
text="An-Boi"
print(type(text))
print("split" in dir(text))
help(str.split)
# text.append("x") -> AttributeError
```

append thuộc list, không thuộc str. Đọc traceback từ dòng lỗi của mã mình và thông báo cuối, rồi chọn phương thức phù hợp.

S2 | Kiểm tra hiểu

1. Muốn tính trung bình bằng thư viện chuẩn, chọn module nào?

- A. statistics | Đúng. Đúng: statistics có mean.
- B. tkinter | Chưa đúng. tkinter tạo giao diện.
- C. random | Chưa đúng. random lấy mẫu ngẫu nhiên.

S3 | Làm có hỗ trợ rồi thay đổi

1. Gõ type, dir, help theo ví dụ; UI mở kết quả từng lệnh, không hiện cùng lúc ba khối dài.
2. Chạy "An".append("x") trong ô lỗi; chọn loại AttributeError và dòng gọi sai.
3. Tra help(statistics.mean), import đúng module rồi tính [4,6,8].

Thử thay đổi: Chuyển đầu vào sang list; giải thích vì sao append lúc này tồn tại nhưng không giúp tính trung bình.

S4 | Thử thách độc lập

Viết mean_or_none(values) dùng statistics.mean với list số hữu hạn, rỗng trả None. Nộp thêm ba câu lệnh type/dir/help đã dùng để tìm str.split và chú thích traceback của "abc".append("d").

Môi trường: PY_CORE. Challenge answer: answers/C13/answer.md; test_criteria.json khóa C13.

Gợi ý mở theo nhu cầu

1. append thuộc list, không thuộc str. Đọc traceback từ dòng lỗi của mã mình và thông báo cuối, rồi chọn phương thức phù hợp.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: C13-K01 (mean). Xem input/expected hoặc protocol trong test_criteria.json.

Bản giải thích phải xác định AttributeError, đối tượng str và lời gọi append; có lệnh khám phá thực, không chỉ ảnh code sẵn.

C15 | Ngẫu nhiên có thể kiểm thử

Lộ trình chính | 30 phút dự kiến | Học trước: P07, P10, C13

Nguồn (số trang PDF): SL 330; HF 39-43. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Ngẫu nhiên hữu ích khi chọn câu hỏi ôn tập. Một bộ sinh riêng có seed giúp tái tạo lỗi mà không thay đổi trạng thái ngẫu nhiên của toàn ứng dụng.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
import random
rng=random.Random(17)
print(rng.choice(["A", "B", "C"]))
print(rng.randint(0,10))
print(0<=rng.random()<1)
```

choice chọn phần tử, randint bao gồm hai đầu, random tạo float từ 0 đến dưới 1. Seed không biến ngẫu nhiên thành dữ liệu bí mật an toàn.

S2 | Kiểm tra hiểu

1. randint(0,10) có thể trả 10 không?
- A. Có | Đúng. Đúng: cả hai đầu đều được bao gồm.
- B. Không | Chưa đúng. Đừng nhầm với stop của range.
- C. Chỉ khi seed=10 | Chưa đúng. Seed không quy định biên như vậy.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chọn bộ bài ba mã; mỗi lần bấm Rút chỉ hiện thẻ được chọn.
2. Khởi tạo lại cùng seed và cùng thứ tự lời gọi; so hai chuỗi kết quả.
3. Thêm một lời gọi random trước choice; quan sát chuỗi sau thay đổi.

Thử thay đổi: Đổi bộ bài rỗng: thiết kế hợp đồng rõ thay vì để choice lỗi không giải thích.

S4 | Thử thách độc lập

Viết sample(deck,seed): deck list không rỗng hoặc rỗng; nếu rỗng trả None. Nếu có, dùng random.Random(seed), theo thứ tự choice(deck), randint(0,10), random(); trả tuple ba kết quả. Không thay deck hoặc global random state.

Môi trường: PY_CORE. Challenge answer: answers/C15/answer.md; test_criteria.json khóa C15.

Gợi ý mở theo nhu cầu

1. choice chọn phần tử, randint bao gồm hai đầu, random tạo float từ 0 đến dưới 1. Seed không biến ngẫu nhiên thành dữ liệu bí mật an toàn.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: C15-K01 (repeat); C15-K02 (bounds). Xem input/expected hoặc protocol trong test_criteria.json.

P11 | Module và trung bình có trọng số

Lộ trình chính | 25 phút dự kiến | Học trước: P10

Nguồn (số trang PDF): SL 133-151; GUI 499-521. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một môn nhiều tín chỉ có trọng số khác môn ít tín chỉ. Đội đã viết công thức vào `grading.py`. Nhiệm vụ của bạn là dùng module đó từ màn hình báo cáo, không chép công thức vào mọi file.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
from grading import weighted_mean
gpa = weighted_mean([8, 6], [3, 1])
```

`grading.py` do hệ thống cấp. `weighted_mean` tính tổng điểm nhân tín chỉ chia tổng tín chỉ: $(8*3+6*1)/4=7.5$. Import dùng tên module, không viết đường dẫn hay đuôi `.py`.

S2 | Kiểm tra hiểu

1. Kết quả mẫu là gì?

A. 14 | Chưa đúng. 14 là tổng điểm, chưa chia.

B. 7 | Chưa đúng. 7 là trung bình không trọng số.

C. 7.5 | Đúng. Đúng: môn 8 có trọng số 3.

2. Cách import nào đúng ở đây?

A. `from grading import weighted_mean` | Đúng. Đúng: gọi hàm đã được module cung cấp.

B. `import "grading.py"` | Chưa đúng. import không nhận chuỗi tên file như vậy.

C. `from grading.py import weighted_mean` | Chưa đúng. Tên module không có đuôi `.py` trong cú pháp này.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chọn tab `grading.py` chỉ đọc; hiện hợp đồng hàm và công thức, không cần học NumPy để gọi.

2. Đặt khối import ở `main.py`; sau load tên `weighted_mean` xuất hiện trong bảng công cụ.

3. Đặt lời gọi với hai list; chạy: gửi `marks/credits`, nhận 7.5; không thêm `stdout`.

Thử thay đổi: Thay `credits=[1,3]`, dự đoán `gpa=6.5`; giải thích điểm nào nặng hơn.

S4 | Thử thách độc lập

Cấp sẵn `grading.py` trong workspace. Viết `report(marks, credits)` gọi `grading.weighted_mean` hoặc hàm import từ đó rồi trả kết quả. Hai list dài bằng nhau, điểm hữu hạn `[0,10]`, tín chỉ nguyên dương; rỗng trả `None`. Không làm tròn và không sửa đầu vào.

Môi trường: PY_CORE. Challenge answer: `answers/P11/answer.md`; `test_criteria.json` khóa P11.

File/học liệu cấp kèm: `grading.py`.

Gợi ý mở theo nhu cầu

1. Tìm hợp đồng hàm trong module được cấp.

2. Import tên hàm, không import đường dẫn file.

3. `from grading import weighted_mean`; trong `report` return `weighted_mean(marks, credits)`.

Phản hồi ngộ nhận: Nếu trả 7 thay vì 7.5: bạn đang bỏ qua tín chỉ hoặc đã viết lại sai công thức.

Nghiệm thu bài học

Ví dụ công khai P11-a: {} -> 7.5

C19 | Module, alias và package lồng nhau

Lộ trình chính | 40 phút dự kiến | Học trước: P11, C13

Nguồn (số trang PDF): SL 136-139,151; SL 142; SL 143-148; GUI 499-560. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Chia chương trình theo trách nhiệm giúp tái sử dụng mà không copy mã. `__init__.py` của package khác với phương thức `__init__` của một class.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
# domains/grades.py
def passed(score): return score>=5
# domains/__init__.py
from .grades import passed
# main.py
import domains as dm
print(dm.passed(6))
```

Import domains nạp API đã export; alias dm chỉ là tên khác trong module gọi. Relative import chấm chỉ package hiện tại.

S2 | Kiểm tra hiểu

- domains/`__init__.py` tự tạo Student object mỗi khi gọi hàm không?
- A. Có | Chưa đúng. Đừng nhầm file package với constructor class.
- B. Không | Đúng. Đúng: nội dung file chạy khi module được nạp, không phải mỗi lần gọi hàm.
- C. Chỉ khi có alias | Chưa đúng. Alias không làm đổi cơ chế đó.

S3 | Làm có hỗ trợ rồi thay đổi

- Tạo cây domains/grades.py và `__init__.py`; UI nối import với file được nạp.
- Đổi import domains thành import domains as dm và sửa nơi gọi; thử gọi tên cũ để thấy NameError.
- Thêm domains/reports/summary.py và import tương đối lên grades; chạy từ thư mục gốc.

Thử thay đổi: Thêm print trong main dưới main guard; import main phải không tự chạy giao diện.

S4 | Thử thách độc lập

Tạo package domains với grades.py: `passed(score)`, reports/summary.py: `count_passed(scores)`, `__init__.py` export `count_passed`. main.py import domains as dm và in kết quả [4,5,8] chỉ khi chạy trực tiếp. Nộp đủ file, không sửa sys.path để che cấu trúc sai.

Môi trường: MANUAL_MULTIFILE. Challenge answer: answers/C19/answer.md; test_criteria.json khóa C19.

Gợi ý mở theo nhu cầu

- Import domains nạp API đã export; alias dm chỉ là tên khác trong module gọi. Relative import chấm chỉ package hiện tại.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C19-K01 (package); C19-K02 (nested). Xem input/expected hoặc protocol trong test_criteria.json.

P12 | Đọc file với with và UTF-8

Lộ trình chính | 25 phút dự kiến | Học trước: P03, P08, P10

Nguồn (số trang PDF): SL 152-167; HF 196-215. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Danh sách thành viên cần được lưu sau khi đóng chương trình. File là dữ liệu tồn tại ngoài lần chạy. with giúp quản lý khoảng thời gian mở file và đóng nó khi rời khỏi.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
with open("names.txt", encoding="utf-8") as f:
    names = []
    for line in f:
        name = line.strip()
        if name:
            names.append(name)
```

Fixture names.txt có " An ", một dòng trống và "Binh". strip bỏ khoảng trắng ngoài cùng; không xóa khoảng trắng giữa tên. File chỉ tồn tại trong khu bài học.

S2 | Kiểm tra hiểu

- Với ba dòng đã nêu, names là gì?
 - ["AnBinh"] | Chưa đúng. Mỗi dòng có tên là một phần tử riêng.
 - ["An", "Binh"] | Đúng. Đúng: dòng trống bị bỏ, hai tên giữ riêng.
 - [" An ", "", "Binh"] | Chưa đúng. Bạn chưa áp dụng strip và kiểm tra rỗng.
- File được đóng khi nào trong with?
 - Chỉ khi tắt máy | Chưa đúng. with không cần đợi đến lúc tắt máy.
 - Chỉ khi gọi print | Chưa đúng. print không quản lý vòng đời file.
 - Khi rời khỏi with | Đúng. Đúng: context manager quản lý đóng file.

S3 | Làm có hỗ trợ rồi thay đổi

- Bấm tệp names.txt; preview 3 dòng giữ số dòng; đặt with open(...encoding...) vào đầu mã.
 - Chạy từng dòng: line=" An \n" -> name="An" -> append; dòng trống -> "" nên bỏ qua.
 - Rời with: trạng thái tệp Mở -> Đóng; names cuối ["An", "Binh"].
- Thử thay đổi: Fixture mới chỉ có "Mai Anh"; dự đoán giữ dấu cách bên trong.

S4 | Thử thách độc lập

Viết load_names(path). File luôn tồn tại, UTF-8. Trả list các dòng sau strip, bỏ dòng rỗng và giữ thứ tự/trùng lặp. Không ghi file. Xử lý thiếu file ở bài sau.

Môi trường: PY_CORE. Challenge answer: answers/P12/answer.md; test_criteria.json khóa P12.

Gợi ý mở theo nhu cầu

- Mỗi dòng cần chuẩn hóa khoảng trắng.
- Dùng with và chỉ append khi chuỗi sau strip không rỗng.
- with open(path, encoding="utf-8") as f; duyệt line, name=line.strip().

Phản hồi ngộ nhận: Nếu mất khoảng trắng giữa Mai Anh: strip chỉ cần bỏ ở hai đầu; không thay toàn bộ khoảng trắng.

Nghiem thu bài học

Ví dụ công khai P12-a: {} -> ["An", "Binh"]

P13 | Ngoại lệ và dữ liệu nhập không hợp lệ

Lộ trình chính | 25 phút dự kiến | Học trước: P04, P10

Nguồn (số trang PDF): SL 168-170; HF 151-155. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Người dùng có thể gõ tám thay vì 8. Chương trình không nên đóng đột ngột. Ta thử chuyển số, xử lý ValueError và kiểm tra khoảng điểm riêng.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
def parse_score(text):
    try:
        score = float(text)
    except ValueError:
        return None
    if 0 <= score <= 10:
        return score
    return None
```

Sai định dạng khác số ngoài khoảng. Không bắt mọi lỗi rồi im lặng. Điều kiện khoảng cũng loại NaN/Infinity trong hợp đồng bài này.

S2 | Kiểm tra hiểu

1. `parse_score("abc")` trả gì?

A. None | Đúng. Đúng: ValueError được xử lý.

B. Chương trình luôn dừng | Chưa đúng. except xử lý lỗi chuyển số này.

C. 0 | Chưa đúng. Sai nhập không có nghĩa điểm 0.

2. `parse_score("11")` đi qua nhánh nào?

A. except ValueError | Chưa đúng. 11 chuyển sang float được.

B. Chuyển số thành công rồi bị loại ngoài khoảng | Đúng. Đúng: lỗi quy tắc khác lỗi chuyển kiểu.

C. Trả 11 | Chưa đúng. Đề chỉ nhận từ 0 đến 10.

S3 | Làm có hỗ trợ rồi thay đổi

1. Nhập "abc" vào ô mô phỏng; chạy float -> ValueError; con trỏ nhảy except, result None.

2. Đổi "11"; float -> 11.0, điều kiện khoảng False; không highlight except.

3. Đổi "0"; float -> 0.0, điều kiện True; return 0.0 có nhãn hợp lệ.

Thử thay đổi: So sánh " 7.5 " và ""; dự đoán 7.5 và None, giải thích hai đường đi.

S4 | Thử thách độc lập

Viết `parse_score(text)`, `text` luôn là str. Trả float hữu hạn trong [0,10], hoặc None với chuỗi không chuyển được/ngoài khoảng/nan/inf. Khoảng trắng ngoài chuỗi được phép. Không in thông báo trong hàm.

Môi trường: PY_CORE. Challenge answer: answers/P13/answer.md; test_criteria.json khóa P13.

Gợi ý mở theo nhu cầu

1. Tách bước đổi số và kiểm tra khoảng.

2. Chỉ bắt ValueError khi float không đọc được chuỗi.

3. try float(text); except ValueError: return None; rồi kiểm tra 0 <= score <= 10.

Phản hồi ngộ nhận: Nếu mọi lỗi đều bị che: thu hẹp except về lỗi dự kiến để lỗi lập trình vẫn được nhận ra.

Nghiem thu bài học

Ví dụ công khai P13-a: {} -> 7.5

C12 | Nhánh elif và input trong chương trình CLI

Lộ trình chính | 30 phút dự kiến | Học trước: P04, P13

Nguồn (số trang PDF): SL 56-57; SL 62. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một chuỗi if/elif chọn nhánh đầu tiên đúng. input trả chuỗi, nên cần đổi kiểu và xử lý lỗi trước khi phân loại.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
score=8
if score>=8:
    label="Gioi"
elif score>=5:
    label="Dat"
else:
    label="Chua dat"
print(label)
```

8 thỏa hai ngưỡng nhưng chỉ nhánh đầu chạy. Các ngưỡng này là quy ước bài tập, không phải quy chế USTH.

S2 | Kiểm tra hiểu

1. Nếu kiểm tra ≥ 5 trước ≥ 8 thì điểm 9 ra gì?
- A. Gioi | Chưa đúng. Nhánh đầu đã nhận điểm 9.
- B. Dat | Đúng. Đúng: thứ tự điều kiện làm thay đổi kết quả.
- C. Cả hai | Chưa đúng. elif không chạy tiếp sau nhánh đúng.

S3 | Làm có hỗ trợ rồi thay đổi

1. Nhập "8" qua input; hiện kiểu str rồi float sau chuyển đổi.
 2. Thử 4.9, 5, 7.9, 8; tô đúng một nhánh của chuỗi if/elif.
 3. Thêm nhánh ngoài cùng kiểm tra dữ liệu hợp lệ; nhánh trong mới phân loại.
- Thử thay đổi: Nhập "abc", "nan" và "11"; phân biệt lỗi chuyển kiểu với giá trị ngoài hợp đồng.

S4 | Thử thách độc lập

Viết classify(text) trả Gioi nếu $8 \leq \text{score} \leq 10$, Dat nếu $5 \leq \text{score} < 8$, Chưa dat nếu $0 \leq \text{score} < 5$, Invalid nếu không phải số hữu hạn trong $0 \leq \text{score} \leq 10$. Viết main() gọi input("Diem: "), rồi print nhãn; chỉ chạy main khi file là __main__.

Môi trường: PY_CORE. Challenge answer: answers/C12/answer.md; test_criteria.json khóa C12.

Gợi ý mở theo nhu cầu

1. 8 thỏa hai ngưỡng nhưng chỉ nhánh đầu chạy. Các ngưỡng này là quy ước bài tập, không phải quy chế USTH.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghệ thuật bài học

Các ca bắt buộc: C12-K01 (boundaries); C12-K02 (invalid). Xem input/expected hoặc protocol trong test_criteria.json.

Chạy file qua CLI: nhập 8, stdout gồm prompt Diem: và Gioi; import module không hỏi input.

C16 | Ngày giờ và tìm mẫu bằng regex

Lộ trình chính | 30 phút dự kiến | Học trước: P03, P13

Nguồn (số trang PDF): SL 330. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Chuỗi ngày tháng cần được phân tích theo định dạng đã biết. Regex tìm cấu trúc văn bản; không thay thế việc kiểm tra ngày có tồn tại.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
from datetime import datetime, timedelta
import re
day=datetime.strptime("2024-02-28", "%Y-%m-%d")
print((day+timedelta(days=2)).strftime("%Y-%m-%d"))
print(re.sub(r"\s+", " ", "An   Binh"))
```

Hai ngày sau 28/02/2024 là 01/03/2024 vì năm nhuận. sub thay mọi dãy khoảng trắng bằng một dấu cách.

S2 | Kiểm tra hiểu

1. Regex khớp 2024-02-31 chứng minh ngày đó hợp lệ chưa?
 - A. Rồi | Chưa đúng. Regex cấu trúc không xác nhận lịch.
 - B. Chưa | Đúng. Đúng:.strptime sẽ báo ngày không tồn tại.
 - C. Chỉ cần đúng 10 ký tự | Chưa đúng. Độ dài không đủ.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tách năm/tháng/ngày theo format; ngày sai chuyển ô lỗi ValueError.
2. Cộng timedelta qua ranh giới tháng; trực lịch nhảy đúng số ngày.
3. Dùng re.search tìm mã SV gồm S và ba chữ số; dùng re.sub chuẩn hóa khoảng trắng.

Thử thay đổi: Thử ngày 2023-02-29 rồi 2024-02-29; so kết quả. Nên datetime.now phụ thuộc thời điểm chạy nên không dùng làm expected cố định.

S4 | Thử thách độc lập

Viết due_date(text,days) nhận ngày YYYY-MM-DD hợp lệ, trả ngày sau days (int, có thể âm) cùng định dạng. Viết extract_id(text) trả mã đầu tiên khớp r"\bS\d{3}\b" hoặc None; clean_spaces(text) gộp khoảng trắng rồi strip.

Môi trường: PY_CORE. Challenge answer: answers/C16/answer.md; test_criteria.json khóa C16.

Gợi ý mở theo nhu cầu

1. Hai ngày sau 28/02/2024 là 01/03/2024 vì năm nhuận. sub thay mọi dãy khoảng trắng bằng một dấu cách.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C16-K01 (calendar); C16-K02 (regex). Xem input/expected hoặc protocol trong test_criteria.json.

P14 | Lưu cấu trúc dữ liệu bằng JSON

Lộ trình chính | 25 phút dự kiến | Học trước: P06, P12, P13

Nguồn (số trang PDF): SL 179-182; GUI ch.7 là đối chiếu lưu DB, không nguồn JSON. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Danh sách điểm có nhiều trường nên lưu từng dòng thủ công dễ nhầm. JSON biểu diễn list, dict và các giá trị đơn giản dưới dạng văn bản. Đọc lại cần khôi phục dữ liệu chứ không chỉ lấy chuỗi.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
import json
state = {"students": [{"id": "S01", "name": "An"}]}
with open("state.json", "w", encoding="utf-8") as f:
    json.dump(state, f, ensure_ascii=False)
```

dump ghi dữ liệu có cấu trúc. Khi đọc dùng load. Bài chỉ dùng kiểu JSON hỗ trợ, không lưu trực tiếp object Student hay khóa tuple.

S2 | Kiểm tra hiểu

1. json.load(file) phục vụ việc gì?

- A. Chỉ trả nguyên văn mọi ký tự | Chưa đúng. Đó là cách đọc text thuần, chưa parse.
- B. Chạy mã Python trong file | Chưa đúng. JSON không phải lệnh chạy mã Python.
- C. Đọc lại cấu trúc dữ liệu | Đúng. Đúng: parse văn bản JSON thành kiểu phù hợp.

2. Có tự lưu mọi đối tượng Python vào JSON không?

- A. Không, phải chuyển sang dữ liệu được hỗ trợ | Đúng. Đúng: bài dùng list/dict với kiểu đơn giản.
- B. Chỉ cần đổi đuôi file | Chưa đúng. Đổi tên không đổi định dạng.
- C. Có | Chưa đúng. Đối tượng tự định nghĩa cần chuyển biểu diễn.

S3 | Làm có hỗ trợ rồi thay đổi

- 1. Đặt dump trong with mode w; chạy: file ảo chưa có -> JSON chứa students; trạng thái tệp trở lại Đóng.
- 2. Bấm Reset runtime, giữ file theo kịch bản demo được ghi rõ; biến state biến mất, file vẫn có.
- 3. Đặt load trong with mode r; chạy: restored có dict tương đương, hiển thị so sánh cấu trúc thay vì chuỗi JSON.

Thử thay đổi: Thêm tên "Hà"; dự đoán đọc lại vẫn đúng Unicode, không chấm thứ tự khóa trong văn bản.

S4 | Thử thách độc lập

Viết save_state(path, state) và load_state(path). state chỉ gồm list/dict khóa str và giá trị JSON hữu hạn. File thiếu: load_state trả {"students": []}; file JSON hỏng phải phát sinh JSONDecodeError, không âm thầm đổi thành rỗng. save ghi đề có chủ đích trong khu bài tập.

Môi trường: PY_CORE. Challenge answer: answers/P14/answer.md; test_criteria.json khóa P14.

Gợi ý mở theo nhu cầu

- 1. Phân biệt file thiếu và file đã có nhưng hỏng.
- 2. Dùng json.dump/load trong with; chỉ xử lý FileNotFoundError theo hợp đồng.
- 3. load_state bắt FileNotFoundError rồi trả {"students": []}; các lỗi JSON giữ nguyên.

Phản hồi ngộ nhận: Nếu file hỏng bị trả danh sách rỗng: dữ liệu lỗi đang bị che; chỉ bắt trường hợp thiếu file.

Nghiem thu bài học

Ví dụ công khai P14-a: {} -> {"students": [{"id": "S01", "name": "Hà"}]}

C21 | File modes, buffering và lỗi I/O

Lộ trình chính | 30 phút dự kiến | Học trước: P12, P13, P14

Nguồn (số trang PDF): SL 158-160; SL 161-163; SL 164-165; SL 169-170; SL 171-172. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Chế độ mở quyết định đọc, ghi đè hay nối. Buffer giữ dữ liệu tạm; flush đẩy buffer của Python nhưng không đồng nghĩa dữ liệu chắc chắn đã tới thiết bị vật lý.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
with open("log.txt", "w", encoding="utf8") as f:
    f.write("A\n")
with open("log.txt", "a", encoding="utf8") as f:
    f.write("B\n")
    f.flush()
with open("log.txt", encoding="utf8") as f:
    print(f.readline(), f.read())
```

w tạo/ghi đè, a nối cuối, r đọc. b dùng bytes; + cho phép đọc và ghi nhưng vẫn phải quản lý vị trí con trỏ.

S2 | Kiểm tra hiểu

- Mở log đang có bằng w để thêm một dòng sẽ thế nào?
 - Giữ toàn bộ và nối | Chưa đúng. a mới là nối.
 - Xóa nội dung cũ lúc mở | Đúng. Đúng: lựa chọn mode là một phần hợp đồng dữ liệu.
 - Chỉ đọc | Chưa đúng. r mới chỉ đọc.

S3 | Làm có hỗ trợ rồi thay đổi

- Mở fixture bằng r, đọc 1 ký tự rồi readline rồi readlines; con trỏ di chuyển, hiển thị phần còn lại.
- Dùng r+b trên bytes ABC; seek(1), write(b"Z"), flush, seek(0) đọc AZC.
- Dùng TemporaryDirectory làm vùng thử; chạy xong thư mục tự dọn. Mock PermissionError để kiểm tra mà không sửa quyền hệ thống.

Thử thay đổi: Đổi a thành w trên bản sao log và giải thích mất dòng đầu. Thử read trên handle đã đóng: ValueError.

S4 | Thử thách đọc lập

Viết append_line(path, text) nối text+newline UTF-8; read_status(path) trả ("ok", nội_dung), ("missing", None) khi FileNotFoundError, ("denied", None) khi PermissionError; lỗi khác truyền ra. Dùng with. Nộp thêm trace lab binary và TemporaryDirectory.

Môi trường: PY_CORE. Challenge answer: answers/C21/answer.md; test_criteria.json khóa C21.

Gợi ý mở theo nhu cầu

- w tạo/ghi đè, a nối cuối, r đọc. b dùng bytes; + cho phép đọc và ghi nhưng vẫn phải quản lý vị trí con trỏ.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: C21-K01 (append); C21-K02 (permission). Xem input/expected hoặc protocol trong test_criteria.json.

Trace bắt buộc gồm r/w/a/b/+, read(size), readline, readlines và seek; binary ABC thành AZC; xác nhận folder tạm không tồn tại sau context.

C22 | Duyệt thư mục và thao tác file trong vùng bài tập

Lộ trình chính | 30 phút dự kiến | Học trước: P12, C21

Nguồn (số trang PDF): SL 183-188; SL 187-188; SL 328. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Thư mục có thể lồng nhiều tầng. Duyệt đệ quy tìm file bên trong; đường dẫn tương đối được hiểu từ working directory hiện tại.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
from pathlib import Path
root=Path("lab")
root.mkdir(exist_ok=True)
(root/"a.txt").write_text("A",encoding="utf8")
(root/"a.txt").rename(root/"b.txt")
print([p.name for p in root.iterdir()])
```

mkdir tạo folder, rename đổi tên. iterdir chỉ duyệt trực tiếp; os.walk đi nhiều tầng. Bài chỉ thao tác fixture của mình.

S2 | Kiểm tra hiểu

1. os.chdir thay đổi gì?

A. Di chuyển file | Chưa đúng. File không bị di chuyển.

B. Working directory của tiến trình | Đúng. Đúng: đường dẫn tương đối sau đó được giải từ nơi mới.

C. Tên module | Chưa đúng. Module không tự đổi tên.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo cây lab/a.txt và lab/sub/b.txt; iterdir hiện sub như một mục, walk hiện b.txt ở tầng dưới.

2. Lưu os.getcwd, đổi sang lab, quan sát Path("a.txt").resolve rồi khôi phục cwd bằng finally.

3. Đổi tên a.txt thành renamed.txt, unlink đúng file vừa tạo; rmdir thử folder không rỗng rồi giải thích lỗi.

Thử thay đổi: Thêm file .csv ở tầng sâu; bộ lọc .txt phải bỏ qua nó nhưng vẫn duyệt folder.

S4 | Thử thách độc lập

Viết text_files(root) trả list đường dẫn tương đối dạng POSIX của mọi file .txt dưới root, sắp tăng dần; không theo symlink directory. Dùng os.walk. Nộp trace tạo, đổi tên, xóa một file thử và khôi phục cwd.

Môi trường: PY_CORE. Challenge answer: answers/C22/answer.md; test_criteria.json khóa C22.

Gợi ý mở theo nhu cầu

1. mkdir tạo folder, rename đổi tên. iterdir chỉ duyệt trực tiếp; os.walk đi nhiều tầng. Bài chỉ thao tác fixture của mình.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C22-K01 (recursive); C22-K02 (empty). Xem input/expected hoặc protocol trong test_criteria.json.

Trace cwd trở lại đường dẫn ban đầu kể cả khi thao tác lỗi; chỉ xóa fixture đã tạo, không dùng đường dẫn home hoặc thư mục nguồn.

P15 | Class và trạng thái riêng từng đối tượng

Lộ trình chính | 25 phút dự kiến | Học trước: P10, P14

Nguồn (số trang PDF): SL 107-117; GUI 156-187. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Mỗi sinh viên có mã và danh sách điểm riêng. Class mô tả cách tạo và thao tác với một sinh viên. Hai đối tượng cùng class không được vô tình dùng chung list điểm.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
class Student:
    def __init__(self, student_id):
        self.student_id = student_id
        self.scores = []
    def add_score(self, score):
        self.scores.append(score)
```

self là đối tượng đang nhận lời gọi. Tạo scores bên trong __init__ giúp mỗi instance có list riêng. Bài chưa dạy quyền truy cập như cơ chế bảo mật.

S2 | Kiểm tra hiểu

- Tạo a,b rồi a.add_score(8): điểm của b là gì?
 - Không thể có hai instance | Chưa đúng. Một lớp có thể tạo nhiều đối tượng.
 - [] | Đúng. Đúng: mỗi lần __init__ tạo list riêng.
 - [8] | Chưa đúng. Chỉ đúng nếu bạn dùng chung list ngoài ý muốn.
- self trong add_score là gì?
 - Điểm đang thêm | Chưa đúng. Điểm là tham số score riêng.
 - Tên lớp cố định | Chưa đúng. Tên class và instance khác nhau.
 - Đối tượng nhận lời gọi | Đúng. Đúng: a.add_score dùng a làm self.

S3 | Làm có hỗ trợ rồi thay đổi

- Đặt self.scores=[] trong __init__; tạo a=S01 rồi b=S02: hai thẻ đối tượng, hai list rỗng.
 - Chạy a.add_score(8): chỉ list của a đổi []->[8], b vẫn[].
 - Bấm b rồi xem thuộc tính: student_id=S02, scores=[]; mã không tự chạy lại khi đổi thẻ xem.
- Thử thay đổi: Thêm điểm0 cho b; dự đoán a=[8], b=[0]. Hỏi lỗi nếu scores=[] đặt ở thân class.

S4 | Thử thách độc lập

Viết Student(student_id) có thuộc tính student_id, scores; add_score(score) thêm điểm hợp lệ [0,10] được cấp sẵn; average() trả trung bình hoặc None. Mỗi instance có list riêng. Không cần validation trong bài này.

Môi trường: PY_CORE. Challenge answer: answers/P15/answer.md; test_criteria.json khóa P15.

Gợi ý mở theo nhu cầu

- Dữ liệu nào thuộc riêng mỗi sinh viên?
- Tạo list mới trong __init__, không làm thuộc tính class dùng chung.
- self.scores=[] trong __init__; average xử lý list rỗng trước chia.

Phản hồi ngộ nhận: Nếu thêm điểm cho a làm b đổi theo: kiểm tra list có được tạo lại ở mỗi lần __init__ không.

Nghiệm thu bài học

Ví dụ công khai P15-a: {} -> 7.0

P16 | Kế thừa và ghi đè hành vi

Lộ trình chính | 25 phút dự kiến | Học trước: P15

Nguồn (số trang PDF): SL 118-131; GUI 499-541. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Sổ học tập có báo cáo thông thường và báo cáo học bổng. Cả hai đều cho biết đủ điều kiện hay chưa, nhưng tiêu chí khác. Kế thừa cho phép giữ giao diện và thay hành vi cần thiết.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
class Course:
    def passed(self, score):
        return score >= 5

class HonorsCourse(Course):
    def passed(self, score):
        return score >= 8
```

Cùng gọi passed(7), Course trả True còn HonorsCourse trả False. Đây là override theo loại đối tượng; không cần if phân biệt tên lớp ở nơi sử dụng.

S2 | Kiểm tra hiểu

- HonorsCourse().passed(7) là gì?
 - False | Đúng. Đúng: phương thức lớp con được dùng.
 - 7 | Chưa đúng. Phương thức trả bool, không trả điểm.
 - True | Chưa đúng. Lớp con đã đổi ngưỡng thành 8.
- Để thay hành vi, lớp con cần gì?
 - Sao chép toàn bộ app | Chưa đúng. Sao chép app không phải mục tiêu kế thừa.
 - Phương thức cùng tên phù hợp giao diện | Đúng. Đúng: override giữ nơi gọi ổn định.
 - Đổi mọi nơi gọi | Chưa đúng. Không cần nếu giao diện vẫn phù hợp.

S3 | Làm có hỗ trợ rồi thay đổi

- Đặt lớp con với (Course); sơ đồ ghi HonorsCourse kế thừa Course.
 - Đặt phương thức passed trong lớp con; chạy với 7: con trở vào phương thức con, result False.
 - Chọn đối tượng Course cùng đầu vào 7; trace vào phương thức cha, result True; hai lần chạy có run_id khác.
- Thử thay đổi: Đổi điểm thành 8; cả hai True. Giải thích vì sao tên phương thức giống mà không phải lúc nào kết quả giống.

S4 | Thử thách độc lập

Viết Course.passed(score) với ngưỡng 5 và HonorsCourse(Course).passed(score) với ngưỡng 8; score là số hữu hạn [0,10], trả bool. Tên gọi theo tình huống giả lập, không là quy chế USTH.

Môi trường: PY_CORE. Challenge answer: answers/P16/answer.md; test_criteria.json khóa P16.

Gợi ý mở theo nhu cầu

- Giữ giao diện passed cho hai loại đối tượng.
- Đặt phương thức cùng tên trong lớp con.
- class HonorsCourse(Course): def passed(self, score): return score >= 8

Phản hồi ngộ nhận: Nếu cả hai luôn dùng ngưỡng 5: kiểm tra tên phương thức override và loại instance đang gọi.

Nghịệm thu bài học

Ví dụ công khai P16-a: {} -> [true, false]

C20 | So sánh đối tượng, biểu diễn và đóng gói

Lộ trình chính | 30 phút dự kiến | Học trước: P15, P16

Nguồn (số trang PDF): SL 114; SL 115-117; SL 121-122; SL 128-131. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Object có thể định nghĩa cách hiển thị và so sánh. Dấu underscore thể hiện ý định nội bộ; name mangling không phải hàng rào bảo mật.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
class Grade:
    def __init__(self,value): self._value=value
    def __str__(self): return f"Grade({self._value})"
    def __lt__(self,other): return self._value<other._value
print(str(Grade(8)))
```

str gọi __str__. sorted có thể dùng __lt__. Accessor kiểm soát quy tắc cập nhật thay vì để trạng thái tùy ý.

S2 | Kiểm tra hiểu

1. isinstance(x,Grade) và issubclass(Child,Grade) nhận cùng loại đối số không?
- A. Có | Chưa đúng. Đối số đầu có vai trò khác nhau.
- B. Không: object so với class | Đúng. Đúng: một cái xét instance, một cái xét quan hệ lớp.
- C. Cả hai chỉ nhận chuỗi | Chưa đúng. Chúng không nhận tên class dưới dạng chuỗi.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo hai Grade 8 và 5; sorted đổi thứ tự theo giá trị, không theo địa chỉ object.
 2. Thêm property value; thử cập nhật 11, giữ giá trị trước và hiện ValueError.
 3. So _value với __value và quan sát tên mangle trong vars; giải thích vẫn có thể truy cập nếu biết tên.
- Thử thay đổi: Thêm subclass HonorsGrade và dự đoán isinstance(instance,Grade), issubclass(HonorsGrade,Grade).

S4 | Thử thách độc lập

Viết Grade(value) cho số hữu hạn trong 0..10 (không bool), property value có setter kiểm tra, __str__ dạng Grade(8) nếu value được cấp int 8, __lt__ so theo value với Grade, khác kiểu trả NotImplemented. Không cập nhật khi dữ liệu sai.

Môi trường: PY_CORE. Challenge answer: answers/C20/answer.md; test_criteria.json khóa C20.

Gợi ý mở theo nhu cầu

1. str gọi __str__. sorted có thể dùng __lt__. Accessor kiểm soát quy tắc cập nhật thay vì để trạng thái tùy ý.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: C20-K01 (sort); C20-K02 (validation). Xem input/expected hoặc protocol trong test_criteria.json.

Học viên phải tự viết và giải thích hai phép isinstance/issubclass với subclass; nêu underscore không bảo vệ bí mật.

P17 | Tkinter: cửa sổ, widget và grid

Lộ trình chính | 25 phút dự kiến | Học trước: P11, P15

Nguồn (số trang PDF): SL 300-325; GUI 55-125. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Người dùng cần nhập tên qua form. Widget là thành phần như nhãn và ô nhập; grid đặt chúng vào hàng và cột. Event loop chờ và xử lý sự kiện để cửa sổ hoạt động.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
import tkinter as tk
from tkinter import ttk
root = tk.Tk()
ttk.Label(root, text="Ten").grid(row=0, column=0)
ttk.Entry(root).grid(row=0, column=1)
root.mainloop()
```

Hai widget cùng row0 nên đứng cạnh nhau. mainloop giữ vòng xử lý sự kiện. Import/root/mainloop là scaffold được cấp; người học sửa vị trí widget.

S2 | Kiểm tra hiểu

- Ô nhập ở đâu so với nhãn?
 - Bên dưới | Chưa đúng. Muốn bên dưới cần đổi row.
 - Không xác định vì column | Chưa đúng. grid dùng row/column để đặt widget.
 - Cùng hàng, bên phải | Đúng. Đúng: row bằng nhau, column1 sau column0.
- Muốn nhãn mới ở hàng tiếp theo dùng gì?
 - row=1 | Đúng. Đúng: hàng đếm từ 0.
 - row=0 | Chưa đúng. row0 là hàng đang có.
 - column=-1 | Chưa đúng. Đây không phải cách chọn hàng tiếp theo.

S3 | Làm có hỗ trợ rồi thay đổi

- Chọn Label Ten đặt ô(row0,col0); mã thêm .grid(...); ở simulator chỉ layout đổi sau Chạy.
 - Chọn Entry đặt ô(row0,col1); chạy: form có hai widget cùng hàng; highlight tọa độ bằng nhãn chữ.
 - Bấm mở rộng cửa sổ trong desktop lab; không chấm pixel tuyệt đối, chỉ quan hệ vị trí và nội dung.
- Thử thay đổi: Đổi Entry sang row1,column0; dự đoán nó nằm dưới nhãn rồi chạy.

S4 | Thử thách độc lập

Trong khung Tkinter được cấp, tạo form title="So học tập" với Label "Ma SV" ở(0,0), Entry ở(0,1), Label "Diem" ở(1,0), Entry ở(1,1). Cấp khóa định danh sid_entry/score_entry để QA lấy widget. Chưa có nút lưu. Chạy profile DESKTOP_TK hoặc simulator được ghi nhãn.

Môi trường: DESKTOP_TK. Challenge answer: answers/P17/answer.md; test_criteria.json khóa P17.

Gợi ý mở theo nhu cầu

- Vẽ hai hàng và hai cột trước.
- Mã sinh viên ở hàng0, điểm ở hàng1; nhãn cột0, ô nhập cột1.
- Gọi .grid(row=..., column=...) riêng trên từng widget để giữ biến widget.

Phản hồi ngộ nhận: Nếu biến entry là None: bạn có gán kết quả .grid() thay vì đối tượng Entry không?

Nghiệm thu bài học

TK17-a: tạo root sạch, build_form(root), kiểm tra title và đúng hai Entry; sid_entry.grid_info() row0/column1, score_entry row1/column1.

TK17-b: Label Ma SV ở(0,0), Diem ở(1,0), các widget cùng vùng form; không chồng nhau khi phóng to chữ.

TK17-c: dùng Tab nhập S01 rồi8; hai ô giữ dữ liệu riêng; đóng cửa sổ không còn tiến trình treo.

P18 | Callback: nôi thao tác và trạng thái

Lộ trình chính | 25 phút dự kiến | Học trước: P13, P17

Nguồn (số trang PDF): SL 312-325; GUI 156-183. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Form chưa lưu gì chỉ vì người dùng gõ vào ô. Nút Lưu phát sinh sự kiện; callback đọc dữ liệu, kiểm tra rồi cập nhật trạng thái. Tham chiếu hàm khác với gọi hàm ngay khi dựng nút.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
def on_save():
    status.set("Đã lưu")
```

```
button = ttk.Button(root, text="Lưu", command=on_save)
```

status là StringVar đã cấp. command nhận hàm, không nhận kết quả của on_save(). Callback phải kiểm tra dữ liệu trước khi cập nhật số.

S2 | Kiểm tra hiểu

1. command=on_save() có vấn đề gì?
 - A. Luôn tạo hai nút | Chưa đúng. Lỗi này không liên quan số nút.
 - B. Gọi ngay khi tạo nút | Đúng. Đúng: dấu () thực hiện lời gọi ngay.
 - C. Chỉ chạy sau click | Chưa đúng. Để truyền callback dùng tên hàm chưa gọi.
2. Nhập điểm abc rồi Lưu nên thế nào?
 - A. Xóa cả form | Chưa đúng. Xóa dữ liệu làm người học mất ngữ cảnh.
 - B. Lưu0 | Chưa đúng. Không đổi lỗi nhập thành điểm0.
 - C. Báo lỗi, giữ dữ liệu để sửa | Đúng. Đúng: phản hồi giúp sửa và chưa lưu.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt tên on_save vào command; tạo form: status="Chưa lưu", saved=[]; callback chưa chạy.
 2. Nhập "7.5" rồi click Lưu; một event -> callback -> saved=[7.5], status="Đã lưu".
 3. Nhập "abc" rồi click: saved không đổi, status="Diem khong hop le"; focus quay ô điểm, giữ chuỗi vừa nhập.
- Thử thay đổi: Nhập0 và click; đây là giá trị hợp lệ, phải thêm0.0, không báo thiếu.

S4 | Thử thách độc lập

Viết build_form(root) trả dict entry/button/status/saved. Form có Entry, nút Lưu, Label trạng thái. Ban đầu Chưa lưu và saved=[]. Click với chuỗi chuyển được thành float hữu hạn [0,10] thì append đúng một điểm, status Đã lưu. Sai thì không thêm, status Diem khong hop le, giữ text và focus. Có thể dùng parse_score của P13.

Môi trường: DESKTOP_TK. Challenge answer: answers/P18/answer.md; test_criteria.json khóa P18.

Gợi ý mở theo nhu cầu

1. Việc lưu chỉ diễn ra trong sự kiện click.
2. Truyền tên callback và kiểm tra điểm trước khi append.
3. command=on_save; trong hàm, sai thì return trước saved.append.

Phản hồi ngộ nhận: Nếu dữ liệu được lưu khi mở form: command đã gọi hàm quá sớm. Nếu0 bị loại: kiểm tra bạn có dùng if score không.

Nghiem thu bài học

TK18-a: dựng form sạch; saved=[], status=Chưa lưu, chưa ghi dữ liệu trước click.

TK18-b: nhập7.5, invoke button một lần: saved=[7.5], status=Đã lưu; nhập0, invoke: saved=[7.5,0.0].

TK18-c: lần lượt abc,11,nan,inf: không append, status lỗi, giữ text/focus; event không bị nối callback hai lần.

P19 | Biểu đồ phải khớp dữ liệu

Lộ trình chính | 25 phút dự kiến | Học trước: P06, P10, P11

Nguồn (số trang PDF): SL 329; GUI 187-225. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Biểu đồ giúp so sánh điểm giữa các môn. Mỗi cột phải tương ứng một môn và đúng chiều cao. Nhân trực cho người xem biết số đang đo điều gì; hình đẹp không bù được dữ liệu sai.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
from matplotlib.figure import Figure
fig = Figure()
ax = fig.subplots()
ax.bar(["PY", "DB"], [8, 6])
ax.set_ylabel("Diem")
ax.set_ylim(0, 10)
```

Hai cột theo thứ tự PY rồi DB, chiều cao 8 và 6. Figure là object để test dữ liệu, không cần mở cửa sổ trong runner headless. Backend hiển thị do dev cấp.

S2 | Kiểm tra hiểu

1. Cột DB cao bao nhiêu?

- A. 6 | Đúng. Đúng: dữ liệu của DB là 6.
- B. 10 | Chưa đúng. 10 là giới hạn trục, không phải điểm DB.
- C. 8 | Chưa đúng. 8 thuộc PY.

2. Chấm hình biểu đồ bằng cách nào tốt hơn ở đây?

- A. Chỉ số ảnh pixel | Chưa đúng. Font/backend có thể làm khác pixel dù cùng dữ liệu.
- B. Kiểm tra nhãn, dữ liệu cột, giới hạn trục | Đúng. Đúng: chấm điều có ý nghĩa trong hợp đồng.
- C. Chỉ thấy có hình là đạt | Chưa đúng. Có hình chưa bảo đảm thể hiện đúng dữ liệu.

S3 | Làm có hỗ trợ rồi thay đổi

- 1. Chọn hai cặp môn/điểm; đặt labels và values vào ax.bar; chạy: hai cột 8/6.
 - 2. Thêm ylabel Diem; chạy: nhãn trục xuất hiện, dữ liệu cột không đổi.
 - 3. Thêm ylim(0,10); chạy: thang cố định 0-10, không co trục để khuếch đại chênh lệch.
- Thử thay đổi: Đổi DB=0; cột có chiều cao 0 vẫn có nhãn DB, không bỏ môn.

S4 | Thử thách độc lập

Viết build_chart(labels, values) trả matplotlib.figure.Figure. Hai list cùng độ dài, nhãn str duy nhất, điểm hữu hạn 0-10; có thể rỗng. Một axes, bar đúng thứ tự, ylabel Diem, ylim 0-10. Không gọi show, không sửa đầu vào.

Môi trường: OPTIONAL_LIBS:matplotlib. Challenge answer: answers/P19/answer.md; test_criteria.json khóa P19.

Gợi ý mở theo nhu cầu

- 1. Mỗi nhãn phải đi cùng đúng một giá trị.
- 2. Tạo Figure rồi axes; kiểm tra dữ liệu trước phần trang trí.
- 3. ax.bar(labels, values); ax.set_ylabel("Diem"); ax.set_ylim(0,10).

Phản hồi ngộ nhận: Nếu thứ tự cột thay đổi: bạn đã tự sort nhãn mà chưa di chuyển giá trị cùng cặp?

Nghệ thu bài học

Ví dụ công khai P19-a: {} -> {"heights": [8, 6], "labels": ["PY", "DB"], "ylabel": "Diem", "ylim": [0.0, 10.0]}

P20 | Test trường hợp thường và trường hợp biên

Lộ trình chính | 25 phút dự kiến | Học trước: P04, P10

Nguồn (số trang PDF): GUI 339-409; PE 5-8. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một hàm chạy được chưa chắc xử lý đúng mọi dữ liệu. Nếu quy tắc là điểm ≥ 5 , test chỉ dùng 8 sẽ không phát hiện hàm viết nhầm >5 . Trường hợp biên giúp kiểm tra đúng điều kiện.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
def passed(score):
    return score >= 5

assert passed(4) is False
assert passed(5) is True
assert passed(8) is True
```

Ba test bao gồm dưới ngưỡng, đúng ngưỡng và trên ngưỡng. Assert thất bại giúp chỉ ra khác biệt mong đợi, không chứng minh toàn bộ chương trình tuyệt đối đúng.

S2 | Kiểm tra hiểu

1. Test nào bắt lỗi viết >5 ?

- A. 8 | Chưa đúng. 8 True ở cả hai.
- B. 4 | Chưa đúng. 4 vẫn False với cả hai cách.
- C. 5 | Đúng. Đúng: hai cách khác nhau tại 5.

2. Một hàm qua ba test có chắc không còn lỗi?

- A. Không; test chỉ kiểm tra trường hợp đã bao phủ | Đúng. Đúng: test là bằng chứng có phạm vi.
- B. Không cần test nữa | Chưa đúng. Cần chọn thêm test khi có rủi ro chưa bao phủ.
- C. Có | Chưa đúng. Vẫn có thể có đầu vào khác hoặc vi phạm hợp đồng.

S3 | Làm có hỗ trợ rồi thay đổi

- 1. Chạy hàm sai >5 với test 8: Passed; UI ghi Chưa kiểm tra ranh giới, không gán Hoàn toàn đúng.
- 2. Thêm test 5 kỳ vọng True; chạy: actual False -> Failed, highlight điều kiện.
- 3. Sửa ≥ 5 rồi chạy cả ba: 3/3 qua; giữ lịch sử test phát hiện lỗi.

Thử thay đổi: Đổi ngưỡng thành 10; đề nghị ba giá trị mới 9, 10, 11 và giải thích vai trò từng giá trị.

S4 | Thử thách độc lập

Viết `make_cases()` trả list các tuple (score, expected_bool) cho quy tắc ≥ 5 với score trong $[0, 10]$. Phải có ít nhất một số dưới 5, đúng 5, trên 5; mọi expected phải đúng. Chấm khả năng phân biệt hàm đúng và ba mutant: >5 , luôn True, luôn False. Không chỉ đếm số case.

Môi trường: PY_CORE. Challenge answer: answers/P20/answer.md; test_criteria.json khóa P20.

Gợi ý mở theo nhu cầu

- 1. Tìm điểm mà $\geq$ và $>$ khác nhau.
- 2. Thêm một điểm dưới và một điểm trên để bắt hàm hằng.
- 3. Bộ mẫu: (4, False), (5, True), (8, True).

Phản hồi ngộ nhận: Nếu tất cả case đều ≥ 5 , bộ test chưa phát hiện được hàm luôn trả True.

Nghệ thuật bài học

Ví dụ công khai P20-a: {} -> true

P23 | Chuyển giao: báo cáo thời gian bơi

Lộ trình chính | 45 phút dự kiến | Học trước: P03, P08, P10, P12, P13

Nguồn (số trang PDF): HF 191-258; SL 42-49, 88-94, 152-167. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Câu lạc bộ bơi ghi thời gian dưới dạng m:ss.hh. Chuỗi này tiện đọc nhưng không thể lấy trung bình trực tiếp. Ta đổi tất cả sang đơn vị một phần trăm giây, tính toán rồi đổi lại để hiển thị.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
time_text = "1:02.50"
minutes, rest = time_text.split(":")
seconds, hundredths = rest.split(".")
value = int(minutes)*6000 + int(seconds)*100 + int(hundredths)
```

1 phút=6000 phần trăm giây, 2 giây=200, cộng50 cho6250. Tính bằng số nguyên giúp tránh ghép sai đơn vị. Dữ liệu bài là mới, không dùng lại dữ liệu vận động viên trong sách.

S2 | Kiểm tra hiểu

- 1:02.50 đổi thành bao nhiêu?
 - 10250 | Chưa đúng. Phút không bằng100 giây.
 - 62.50 phút | Chưa đúng. Đơn vị mục tiêu là phần trăm giây.
 - 6250 | Đúng. Đúng: 6000+200+50.
- 6000 phần trăm giây phải định dạng gì?
 - 1:00.00 | Đúng. Đúng: giây và phần trăm có hai chữ số.
 - 1:0.0 | Chưa đúng. Thiếu số0 theo định dạng đề.
 - 0:60.00 | Chưa đúng. Cần chuyển đủ60 giây thành1 phút.

S3 | Làm có hỗ trợ rồi thay đổi

- Checkpoint1: đặt split theo : rồi .; chạy: "1:02.50" -> ["1","02","50"], nhấn đơn vị từng phần.
 - Checkpoint2: đặt nhân6000/100 và cộng; chạy từng phép:6000+200+50 ->6250.
 - Checkpoint3: với [6000,6250], tổng12250/count2 ->6125; định dạng integer divmod ->"1:01.25".
- Thử thay đổi: Thử ["0:59.99","1:00.01"]; dự đoán trung bình đúng1:00.00 trước chạy.

S4 | Thử thách độc lập

Viết swim_report(line). line là chuỗi các m:ss.hh phân cách dấu phẩy, có khoảng trắng ngoài mỗi mục; m nguyên>=0, ss00-59, hh00-99. Dữ liệu sai định dạng ngoài phạm vi; chuỗi rỗng/trắng trả {"count":0,"mean":None}. Trung bình làm tròn half-up tới phần trăm giây; trả dict count và mean m:ss.hh. Bài45 phút chia hai phiên; không buộc làm một lần trên mobile.

Môi trường: PY_CORE. Challenge answer: answers/P23/answer.md; test_criteria.json khóa P23.

Gợi ý mở theo nhu cầu

- Thống nhất một đơn vị số trước khi tính trung bình.
- Dùng phần trăm giây; tách lại bằng chia nguyên và phần dư.
- Làm tròn half-up với $(2*total+n)/(2*n)$; dùng divmod và :02d khi định dạng.

Phản hồi ngộ nhận: Nếu 0:59.99 và1:00.01 không cho1:00.00: kiểm tra chuyển đơn vị, phần dư và carry qua phút.

Nghiệm thu bài học

Ví dụ công khai P23-a: {} -> {"count": 2, "mean": "1:01.25"}

C18 | Báo cáo bơi từ file đến kết quả

Lộ trình chính | 45 phút dự kiến | Học trước: C17, P12, P23, C13

Nguồn (số trang PDF): HF 200-207; HF 245-246; HF 256-260. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Một báo cáo hoàn chỉnh phải nối đúng người bơi với thời gian trong file. Tách hàm metadata khỏi hàm thống kê để kiểm tra từng phần và cả luồng.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
from statistics import mean
times=[6200,6400]
average=mean(times)
print(average)
# 6300 phần trăm giây -> 1:03.00
```

Đổi từng m:ss.hh về số nguyên phần trăm giây trước khi tính mean để mọi giá trị cùng đơn vị.

S2 | Kiểm tra hiểu

- File thiếu nên trả báo cáo rỗng như file có dòng trắng không?
- A. Có | Chưa đúng. Trả rỗng sẽ che lỗi đường dẫn.
B. Không, đây là hai tình huống khác nhau | Đúng. Đúng: thiếu file giữ FileNotFoundError; dòng trắng có hợp đồng riêng.
- C. Luôn tạo file mới | Chưa đúng. Bài đọc không được tự ghi file.

S3 | Làm có hỗ trợ rồi thay đổi

- Tạo file An-19-50m-Free.txt, dòng đầu 1:02.00,1:04.00; dòng sau là ghi chú. Đọc readlines rồi chọn dòng 0.
- Ghép parse_filename với bước đọc; hiện metadata và chỉ dòng dữ liệu được dùng.
- Điền mean rồi half-up và divmod; hiện 6300 -> 1 phút,3 giây,0 phần trăm.

Thử thay đổi: Thay dòng đầu bằng 0:59.99,1:00.00: trung bình half-up là 1:00.00; kiểm tra carry qua phút.

S4 | Thử thách độc lập

Viết file_report(path) dùng parse_filename ở C17 (file cấp sẵn metadata.py), chỉ đọc dòng đầu UTF-8 qua readlines; các thời gian hợp lệ m:ss.hh. File rỗng/trắng: count=0,mean=None. Trả metadata cộng count,mean. Dùng statistics.mean; làm tròn half-up đến phần trăm giây. Không sửa file; thiếu file giữ FileNotFoundError.

Môi trường: PY_CORE. Challenge answer: answers/C18/answer.md; test_criteria.json khóa C18.

File/học liệu cấp kèm: metadata.py.

Gợi ý mở theo nhu cầu

- Đổi từng m:ss.hh về số nguyên phần trăm giây trước khi tính mean để mọi giá trị cùng đơn vị.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: C18-K01 (pipeline); C18-K02 (carry_empty). Xem input/expected hoặc protocol trong test_criteria.json.

P24 | Capstone: Sổ học tập USTH

Lộ trình chính | 75 phút dự kiến | Học trước: P14, P15, P18, P19, P20

Nguồn (số trang PDF): SL 97-98, 132, 150-151, 190-192, 299, 327; GUI ch.4/8/11. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 3 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Bạn nhận ticket hoàn thiện sổ học tập: thêm sinh viên, tạo môn, nhập điểm và xem báo cáo. Dữ liệu phải nhất quán dù người dùng nhập sai, bấm lại hoặc mở ứng dụng lần sau. Hãy xây từng checkpoint và kiểm tra trước khi nối màn hình.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
state = {"students": {"S01": "An"},
        "courses": {"PY": "Python"}, "marks": {}}
# Sau hành động hợp lệ ghi điểm 8:
state["marks"]["S01"] = {"PY": 8}
```

marks lồng dict tránh key tuple khi lưu JSON. Chỉ ghi điểm nếu cả mã sinh viên và mã môn đã có. Core xử lý action độc lập GUI giúp test dễ hơn.

S2 | Kiểm tra hiểu

1. Nhập điểm cho S99 chưa có nên làm gì?

- A. Tự tạo sinh viên vô danh | Chưa đúng. Đề yêu cầu sinh viên tồn tại trước.
- B. Từ chối, giữ nguyên state | Đúng. Đúng: không tạo bản ghi mờ mờ.
- C. Ghi điểm rồi báo lỗi | Chưa đúng. Báo lỗi sau khi ghi vẫn làm dữ liệu sai.

2. Mã môn PY tồn tại, nhập điểm mới cho cùng sinh viên/PY thì sao?

- A. Tạo một môn thứ hai | Chưa đúng. Không tạo bản ghi trùng lặp.
- B. Cấm mọi thay đổi | Chưa đúng. Bài có chức năng sửa điểm.
- C. Cập nhật điểm cũ | Đúng. Đúng: cùng cập định danh thì cập nhật theo hợp đồng.

S3 | Làm có hỗ trợ rồi thay đổi

- Checkpoint A: chạy action add_student S01/An -> students từ {} thành {"S01": "An"}; feedback success; thử trùng mã -> state giữ nguyên.
- Checkpoint B: add_course PY/Python rồi set_mark S01/PY/8 -> một cặp điểm; gọi sai mã -> ValueError, không ghi một phần.
- Checkpoint C: UI đọc state từ service, gọi save_state; mở lại file và dựng report/biểu đồ; test lỗi JSON báo rõ, không thay dữ liệu bằng rỗng.

Thử thay đổi: Đổi tên hiển thị thành An Nguyen, mã giữ S01; điểm vẫn theo mã. Thử sửa 8 thành 0 phải giữ 0 hợp lệ.

S4 | Thử thách độc lập

Lỗi bắt buộc: apply_action(state, action) trả state mới, không sửa đầu vào. state gồm students:dict id->name; courses:dict id->name; marks:dict sid->dict cid->score. action type add_student/add_course có id,name; set_mark có student_id,course_id,score. ID/tên non-empty str, không chỉ trắng; ID trùng bị ValueError; set_mark đòi hai mã có sẵn và score số hữu hạn 0-10 (không bool). Action sai/thiếu trường báo ValueError. Capstone gồm 3 phiên: core, persistence, GUI; lỗi mẫu bên dưới là hợp đồng service, phân tích hợp nghiệm thu riêng.

Môi trường: PY_CORE + DESKTOP_TK. Challenge answer: answers/P24/answer.md; test_criteria.json khóa P24.

Gợi ý mở theo nhu cầu

- Tách cập nhật state khỏi đọc ô nhập và vẽ giao diện.
- Xác thực toàn bộ action trước khi tạo thay đổi có hiệu lực.
- Dùng bản sao state; kiểm tra mã tồn tại rồi mới.setdefault(sid, {})[cid]=score.

Phản hồi ngộ nhận: Nếu lỗi vẫn để lại điểm đã ghi: kiểm tra thử tự validate và commit trạng thái. Nếu mất điểm 0: tránh dùng truthiness cho điểm.

Nghiệm thu bài học

Ví dụ công khai P24-a: {} -> {"students": {"S01": "An"}, "courses": {}, "marks": {}}

Kiểm tra tích hợp: khởi động với file thiếu/hỏng; nhập và sửa dữ liệu; refresh biểu đồ; lưu và mở lại; giữ ứng dụng đáp ứng khi I/O chạy nền. Chạy đầy đủ protocol CAP24 trong test_criteria.json, gồm các ca biên và hành vi lỗi.

T01 | Cửa sổ, focus và trạng thái nút

Lộ trình chính | 30 phút dự kiến | Học trước: P17, P18

Nguồn (số trang PDF): SL 302-307; SL 309; GUI 55-88; GUI 125-156. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Ứng dụng cần cho người dùng biết cửa sổ nào đang làm việc và thao tác nào được phép. Focus nhận bàn phím; disabled ngăn thao tác trên widget.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
root.title("Ho so")
root.geometry("420x240")
root.resizable(False,False)
entry.focus_set()
button.configure(text="Da luu",state="disabled")
```

Thuộc tính cửa sổ và thuộc tính widget có vai trò khác nhau. Khóa resize chỉ phù hợp form cố định đã kiểm tra độ vừa chữ.

S2 | Kiểm tra hiểu

1. Đổi text của nút thành Da luu có tự vô hiệu hóa nút không?
- A. Có | Chưa đúng. Text và state độc lập.
- B. Không, phải đổi state | Đúng. Đúng: cần state disabled.
- C. Nút tự biến mất | Chưa đúng. Đổi text không xóa widget.

S3 | Làm có hỗ trợ rồi thay đổi

1. So một thao tác nhập mã qua CLI và form GUI; chọn GUI khi cần nhiều trường/sự kiện, CLI khi tự động hóa lệnh.
2. Đặt title, size, icon; thử kéo mép cửa sổ trước và sau resizable(False,False).
3. Nhập An, bấm Luu: text nút đổi Da luu, disabled; mở cửa sổ trợ giúp bằng Toplevel.

Thử thay đổi: Bật resize lại, nhập tên dài và kiểm tra việc cắt chữ; giải thích vì sao không nên luôn khóa kích thước.

S4 | Thử thách độc lập

Viết build(root): cửa sổ title Ho so, size 420x240, không resize; Entry có focus khi hiện. Nút Luu khi click đổi thành Da luu và disabled. Nút Tro giúp mở Toplevel title Huong dan (không tạo Tk thứ hai). Gắn icon ảnh 16x16 được tạo bằng PhotoImage và giữ tham chiếu. Trả dict entry/save/help.

Môi trường: DESKTOP_TK. Challenge answer: answers/T01/answer.md; test_criteria.json khóa T01.

Gợi ý mở theo nhu cầu

1. Thuộc tính cửa sổ và thuộc tính widget có vai trò khác nhau. Khóa resize chỉ phù hợp form cố định đã kiểm tra độ vừa chữ.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: T01-K01 (properties). Xem input/expected hoặc protocol trong test_criteria.json.

Hiện cửa sổ: gõ ngay đi vào Entry; icon thấy trên title/taskbar theo hỗ trợ OS; click trợ giúp mở đúng một Toplevel mỗi lần; đóng child không tắt root.

T02 | Lựa chọn dữ liệu: combo, radio, check và list

Lộ trình chính | 45 phút dự kiến | Học trước: P17, P18

Nguồn (số trang PDF): SL 314; SL 315; SL 316; GUI 55-88; GUI 125-156. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Các loại widget diễn đạt loại lựa chọn khác nhau. Radio dùng chung một biến cho lựa chọn đơn; check giữ một trạng thái độc lập.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
level=tk.StringVar(value="basic")
for value in ["basic", "advanced"]:
    ttk.Radiobutton(parent, text=value, value=value, variable=level).pack()
course=ttk.Combobox(parent, values=["PY", "DB"], state="readonly")
```

Hai radio cùng level nên chỉ một giá trị được chọn. Combobox readonly chỉ cho chọn trong danh sách.

S2 | Kiểm tra hiểu

- Mỗi radio dùng một StringVar khác nhau thì điều gì sai?
 - Không còn nhóm loại trừ nhau | Đúng. Đúng: mỗi nút có state riêng nên có thể cùng chọn.
 - Không đổi gì | Chưa đúng. Biến chung là cơ chế của nhóm.
 - Tất cả tự disabled | Chưa đúng. State disabled là thuộc tính khác.

S3 | Làm có hỗ trợ rồi thay đổi

- Tạo combo PY/DB và hai radio basic/advanced; đổi lựa chọn, ô state hiển thị đúng giá trị.
- Thêm check Nhạc học ban đầu False và check Xác nhận ban đầu True nhưng disabled; giải thích khác nhau giữa giá trị và khả năng tương tác.

3. Thêm Listbox nhiều mã, ScrolledText ghi chú và Spinbox số buổi 1..7; tạo các label bằng vòng lặp.

Thử thay đổi: Xóa selection Listbox và thử gõ 99 vào Spinbox: giới hạn mũn tên không thay thế validation đầu vào.

S4 | Thử thách độc lập

Viết build(root) trả các widget/variable: course readonly mặc định PY; level basic; reminder False; confirmed True disabled; students Listbox A/B/C chọn một; notes ScrolledText; sessions Spinbox 1..7 mặc định 1. collect() trả dict dữ liệu; sessions gõ ngoài 1..7 phải ValueError; selection rỗng trả student=None.

Môi trường: DESKTOP_TK. Challenge answer: answers/T02/answer.md; test_criteria.json khóa T02.

Gợi ý mở theo nhu cầu

- Hai radio cùng level nên chỉ một giá trị được chọn. Combobox readonly chỉ cho chọn trong danh sách.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghị thu bài học

Các ca bắt buộc: T02-K01 (defaults). Xem input/expected hoặc protocol trong test_criteria.json.

Thử đủ mỗi widget bằng bàn phím; radio loại trừ nhau, disabled check không đổi; Spinbox nhập 99 bị từ chối bởi collect; ScrolledText cuộn khi có 20 dòng.

T03 | Bố cục: Frame, padding và co giãn

Lộ trình chính | 30 phút dự kiến | Học trước: P17

Nguồn (số trang PDF): SL 310-311; SL 318-319; SL 320-321; GUI 88-125. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Container chia form thành vùng có trách nhiệm riêng. grid xếp theo hàng/cột; pack xếp theo cạnh; place đặt tọa độ hoặc tỷ lệ.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
frame=tkk.LabelFrame(root,text="Ho so",padding=12)
frame.pack(fill="both",expand=True)
frame.columnconfigure(1,weight=1)
entry.grid(row=0,column=1,sticky="ew",padx=6,pady=6)
```

pack quản lý frame trong root, grid quản lý widget trong frame nên không xung đột. Không dùng pack và grid quản lý con trong cùng parent.

S2 | Kiểm tra hiểu

1. Entry sticky="ew" nhưng cột weight=0 có chắc giãn hết khi cửa sổ lớn lên không?
- A. Có | Chưa đúng. sticky chỉ lấp ô đã được cấp.
- B. Không, cột cần được phân chia phần dư | Đúng. Đúng: weight quyết định phân chia phần dư cho cột.
- C. Entry tự đổi parent | Chưa đúng. Parent không đổi.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo root -> outer Frame -> LabelFrame; tô đường biên từng container khi chọn trong cây.
 2. Tăng padding 0 thành 12; khoảng trống quanh widget tăng, không đổi dữ liệu.
 3. So ba panel: pack theo cạnh, grid theo bảng, place relx/rely; kéo resize và quan sát panel nào giữ được nội dung.
- Thử thay đổi: Tắt weight rồi bật lại để dự đoán Entry có nhận phần chiều rộng mới không.

S4 | Thử thách độc lập

Viết build(root) tạo outer pack expand/fill both, LabelFrame Ho so dùng grid trong outer, Label Ten và Entry trong group bằng grid. Cột Entry weight=1 sticky ew, padding 12. Thêm badge Label trong container riêng bằng place(relx=1,x=-8,y=8,anchor=ne). Trả outer/group/entry/badge.

Môi trường: DESKTOP_TK. Challenge answer: answers/T03/answer.md; test_criteria.json khóa T03.

Gợi ý mở theo nhu cầu

1. pack quản lý frame trong root, grid quản lý widget trong frame nên không xung đột. Không dùng pack và grid quản lý con trong cùng parent.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: T03-K01 (managers). Xem input/expected hoặc protocol trong test_criteria.json.

Ở 480x240 và 800x400, Entry giãn theo chiều ngang, label vẫn đọc được; badge cách góc phải 8px; không chồng widget.

T04 | Menu và nhiều Notebook

Lộ trình chính | 30 phút dự kiến | Học trước: T03, P18

Nguồn (số trang PDF): GUI 88-125; GUI 499-560. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Tab chia các công việc liên quan thành trang; menu chứa hành động dùng ở nhiều trang. Một ứng dụng có thể có notebook chính và notebook con.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
tabs=ttk.Notebook(root)
tabs.add(student_frame,text="Sinh vien")
tabs.add(report_frame,text="Bao cao")
menu.add_command(label="Moi",command=reset_form)
```

Callback chỉ được gọi khi chọn menu. Notebook giữ frame mỗi trang, không phải tự tạo Tk mới.

S2 | Kiểm tra hiểu

1. `command=reset_form()` có đúng khi gắn menu không?
- A. Có | Chưa đúng. Cần truyền hàm.
- B. Không, hàm bị gọi ngay | Đúng. Đúng: dùng `command=reset_form`.
- C. Đúng nếu tab đầu | Chưa đúng. Tab không quyết định thời điểm gọi.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo tab Sinh vien/Bao cao, chuyển tab và giữ nội dung Entry.
 2. Trong Bao cao thêm tab Bang/Bieu do; hiển thị cây parent để phân biệt hai notebook.
 3. Thêm menu Tep/Moi và Tro giúp/Gioi thieu; chọn Moi xóa đúng Entry và giữ tab hiện tại.
- Thử thay đổi: Thêm menu chọn trực tiếp tab Bao cao; dự đoán notebook con có cần tạo lại không.

S4 | Thử thách độc lập

Viết `build(root)`: notebook chính Sinh vien/Bao cao; Entry tên ở Sinh vien; notebook con Bang/Bieu do ở Bao cao. Menu Tep -> Moi xóa Entry; menu Xem -> Bao cao chọn tab báo cáo. Trả `tabs/nested/entry/menu_file/menu_view/report`.

Môi trường: DESKTOP_TK. Challenge answer: `answers/T04/answer.md`; `test_criteria.json` khóa T04.

Gợi ý mở theo nhu cầu

1. Callback chỉ được gọi khi chọn menu. Notebook giữ frame mỗi trang, không phải tự tạo Tk mới.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: T04-K01 (`menu_tabs`). Xem `input/expected` hoặc `protocol` trong `test_criteria.json`.

T05 | Hộp thoại và lựa chọn file

Lộ trình chính | 30 phút dự kiến | Học trước: P18, C22

Nguồn (số trang PDF): GUI 125-156; GUI 225-283. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Hộp thoại thông tin, cảnh báo và lỗi cần đúng mục đích. Khi người dùng hủy chọn file, ứng dụng phải giữ dữ liệu hiện có.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
path=filedialog.askopenfilename(parent=root)
if path:
    messagebox.showinfo("Đã chọn", path, parent=root)
```

Hủy trả chuỗi rỗng. parent gắn hộp thoại với cửa sổ chủ; ứng dụng một hộp thoại độc lập có thể tạo root rồi withdraw.

S2 | Kiểm tra hiểu

1. Hủy file dialog nên được báo là lỗi đọc file không?
- A. Có | Chưa đúng. Hủy là một hành động bình thường.
- B. Không, người dùng chưa yêu cầu đọc file | Đúng. Đúng: không đọc và không ghi gì.
- C. Luôn đóng app | Chưa đúng. Không có yêu cầu thoát app.

S3 | Làm có hỗ trợ rồi thay đổi

1. Lần lượt gọi showinfo/showwarning/showerror với cùng parent; quan sát biểu tượng và nội dung tương ứng.
 2. Tạo root ẩn cho hộp thoại độc lập; đóng hộp thoại rồi destroy root, không để cửa sổ trắng.
 3. Chọn file nguồn và thư mục đích chia sẻ đã mount; copy bằng shutil.copy2, báo lỗi riêng khi đường dẫn mất quyền.
- Thử thay đổi: Ngắt truy cập thư mục chia sẻ trong lab hoặc mock PermissionError; không báo Đã sao chép khi copy thất bại.

S4 | Thử thách độc lập

Viết copy_file(source, destination_dir) trả đường dẫn file đích sau copy2. Viết choose_and_copy(root): chọn file rồi folder; hủy ở bất kỳ bước nào trả None; thành công showinfo, lỗi OSError showerror và trả None. Đích có thể là share mạng do lab cấp; không tự cấu hình mạng.

Môi trường: PY_CORE. Challenge answer: answers/T05/answer.md; test_criteria.json khóa T05.

Gợi ý mở theo nhu cầu

1. Hủy trả chuỗi rỗng. parent gắn hộp thoại với cửa sổ chủ; ứng dụng một hộp thoại độc lập có thể tạo root rồi withdraw.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: T05-K01 (copy); T05-K02 (cancel). Xem input/expected hoặc protocol trong test_criteria.json.

Lab native: thực hiện ba loại messagebox; independent dialog không để root trắng; copy sang share mạng và đối chiếu byte. Local copy PASS không chứng minh mạng đã được kiểm thử.

T06 | Tooltip, relief và Canvas

Lộ trình chính | 30 phút dự kiến | Học trước: T03, P18

Nguồn (số trang PDF): GUI 125-156. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Tooltip giải thích ngắn khi cần; Canvas biểu diễn đối tượng theo tọa độ. Các đối tượng vẽ cần ID để cập nhật, thay vì vẽ thêm vô hạn.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
canvas=tk.Canvas(root,width=240,height=120)
item=canvas.create_rectangle(10,10,50,50,fill="teal")
canvas.move(item,20,0)
print(canvas.coords(item))
```

Tọa độ rectangle chuyển từ [10,10,50,50] thành [30,10,70,50]. Widget relief chỉ thay về viền, không đổi dữ liệu.

S2 | Kiểm tra hiểu

- Mỗi lần di chuyển lại create_rectangle thay vì move thì sao?
- Chỉ có một hình | Chưa đúng. create tạo item mới.
- Số hình tăng | Đúng. Đúng: cần giữ ID và cập nhật item đó.
- Canvas tự xóa hình cũ | Chưa đúng. Không có tự xóa.

S3 | Làm có hỗ trợ rồi thay đổi

- So Label relief sunken/raised; đánh dấu đây là tín hiệu thị giác, vẫn cần nhãn chữ.
 - Hover/focus vào nút: mở Toplevel tooltip; leave/blur: đóng tooltip.
 - Bấm Phai: move rectangle +20px; panel tọa độ và hình cập nhật cùng lúc.
- Thử thay đổi: Bấm 10 lần; clamp hình không vượt canvas. Bật reduced motion thì vị trí nhảy trực tiếp.

S4 | Thử thách độc lập

Viết build(root): Canvas 240x120 chứa một rectangle [10,10,50,50]; nút Phai tăng x 20 nhưng cạnh phải tối đa 240. Label relief sunken. Tooltip nút hiện "Dịch sang phải" khi Enter hoặc FocusIn và đóng khi Leave/FocusOut. Trả canvas/item/move/show_tip/hide_tip.

Môi trường: DESKTOP_TK. Challenge answer: answers/T06/answer.md; test_criteria.json khóa T06.

Gợi ý mở theo nhu cầu

- Tọa độ rectangle chuyển từ [10,10,50,50] thành [30,10,70,50]. Widget relief chỉ thay về viền, không đổi dữ liệu.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: T06-K01 (position). Xem input/expected hoặc protocol trong test_criteria.json.

Tab đến nút cũng mở tooltip; blur đóng; Enter/Leave nhiều lần không tăng số tooltip còn sống.

T07 | Event loop, after và thanh tiến độ

Lộ trình chính | 30 phút dự kiến | Học trước: P18, T06

Nguồn (số trang PDF): SL 325; GUI 125-156. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Event loop chỉ xử lý sự kiện khi callback nhường quyền. after đặt một bước cho lần sau; sleep trong callback làm cả giao diện chờ.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
def tick():
    progress["value"] += 10
    if progress["value"] < 100:
        root.after(100, tick)
root.after(100, tick)
```

Mỗi tick làm ít việc rồi trả về. Đây là mô phỏng tiến độ; tác vụ I/O thật cần báo tiến độ thực từ worker.

S2 | Kiểm tra hiểu

1. root.after(100, tick()) có đúng không?
 - A. Có | Chưa đúng. Cần truyền hàm, không gọi ngay.
 - B. Không: tick chạy ngay | Đúng. Đúng: dùng tick.
 - C. Giống sleep | Chưa đúng. after với callback không khóa như sleep.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chạy mô phỏng 0..100 từng bước 10; tô thanh progress và trạng thái job.
 2. Trong lúc chạy thử click nút đếm khác: counter vẫn tăng.
 3. Thêm Cancel bằng after_cancel; trạng thái stopped giữ tiến độ, không hiện completed.
- Thử thay đổi: Bấm Start hai lần nhanh: xác định chỉ một lịch tick hoạt động; Reset đưa value về 0.

S4 | Thử thách độc lập

Viết ProgressJob(root): có bar ttk.Progressbar maximum=100, start(), tick(), cancel(); start hủy lịch cũ và reset 0; mỗi tick tăng 10, lên lịch 100ms nếu <100; cancel hủy after đang chờ. Không dùng sleep hoặc vòng while update.

Môi trường: DESKTOP_TK. Challenge answer: answers/T07/answer.md; test_criteria.json khóa T07.

Gợi ý mở theo nhu cầu

1. Mỗi tick làm ít việc rồi trả về. Đây là mô phỏng tiến độ; tác vụ I/O thật cần báo tiến độ thực từ worker.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: T07-K01 (lifecycle). Xem input/expected hoặc protocol trong test_criteria.json.

Kiểm tra native đủ 1 giây: bar tiến đều, nút phụ vẫn click được; Cancel dừng mọi cập nhật; đóng cửa sổ không callback vào widget đã hủy.

T08 | StringVar, scope và component tái sử dụng

Lộ trình chính | 40 phút dự kiến | Học trước: P15, P18, C19, T03

Nguồn (số trang PDF): GUI 156-187; GUI 499-560. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Biến gắn widget nối giao diện với trạng thái. Component giữ trạng thái riêng giúp hai form không ghi đè tên của nhau.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
name=tk.StringVar(root,value="An")
entry=ttk.Entry(root,textvariable=name)
label=ttk.Label(root,textvariable=name)
name.set("Bình")
print(name.get())
```

set cập nhật Entry và Label cùng binding; get đọc giá trị. Biến global dùng chung để làm hai instance ảnh hưởng nhau.

S2 | Kiểm tra hiểu

- Hai form dùng chung một StringVar, nhập một form thì sao?
 - Form kia đổi theo | Đúng. Đúng: cả hai cùng binding.
 - Hoàn toàn độc lập | Chưa đúng. Phải có biến riêng mới độc lập.
 - Luôn lỗi | Chưa đúng. Dùng chung hợp lệ, nhưng có thể sai thiết kế.

S3 | Làm có hỗ trợ rồi thay đổi

- Cho hai Entry dùng một global name, nhập An và quan sát cả hai đổi.
 - Refactor thành NameForm với self.name riêng; bằng trạng thái tách hai instance.
 - Giữ normalize_name là hàm thuần dùng chung; giải thích không cần class cho mọi phép tính nhỏ.
- Thử thay đổi: Tạo instance thứ ba mà không sửa component; reset một form không đổi hai form còn lại.

S4 | Thử thách độc lập

Viết normalize_name(text) strip và title. Viết NameForm(ttk.Frame) có self.name StringVar riêng, Entry và Label cùng variable, nút Chuan hoa gọi normalize_name. Đặt hai NameForm cạnh nhau bằng code client. Nộp giải thích vì sao hàm chuẩn hóa để ngoài class và vì sao bỏ global state.

Môi trường: DESKTOP_TK. Challenge answer: answers/T08/answer.md; test_criteria.json khóa T08.

Gợi ý mở theo nhu cầu

- set cập nhật Entry và Label cùng binding; get đọc giá trị. Biến global dùng chung để làm hai instance ảnh hưởng nhau.
- Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: T08-K01 (independence). Xem input/expected hoặc protocol trong test_criteria.json.

Nộp before/after loại bỏ global và callback trộn I/O; chỉ ra ba trách nhiệm: chuẩn hóa, giữ state form, bố cục client. Không tạo class rỗng chỉ để bọc một hàm.

T09 | Biểu đồ trong Tk và thay đổi thang đo

Lộ trình chính | 30 phút dự kiến | Học trước: P19, T03, P18

Nguồn (số trang PDF): GUI 187-225. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Biểu đồ trong app phải cập nhật theo dữ liệu và giữ nhãn để hiểu. FigureCanvasTkAgg đưa Figure vào widget Tk; đổi giới hạn trục cần redraw.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
figure=Figure(figsize=(4,3))
axes=figure.add_subplot()
axes.plot([1,2],[4,8],label="Diem")
axes.legend()
canvas=FigureCanvasTkAgg(figure, master=root)
canvas.draw()
```

Figure chứa Axes; Tk canvas là adapter hiển thị. legend cần label trên đường dữ liệu.

S2 | Kiểm tra hiểu

1. Đổi ylim nhưng không redraw thì màn hình chắc đã cập nhật chưa?
 - A. Chắc rồi | Chưa đúng. Model thay đổi chưa bảo đảm ảnh được vẽ lại.
 - B. Chưa, cần draw/draw_idle | Đúng. Đúng.
 - C. Phải mở Tk mới | Chưa đúng. Chỉ cần redraw canvas hiện có.

S3 | Làm có hỗ trợ rồi thay đổi

1. Nhúng Figure vào frame; tô ranh giới giữa widget Tk và Axes.
 2. Thêm đường điểm và legend; đổi từ giới hạn cố định 0..10 sang 0..max+1.
 3. Thay dữ liệu [4,8] thành [2,12]; gọi set_data, đổi limits, draw_idle; không thêm line thừa.
- Thử thay đổi: Dùng dữ liệu toàn 0 và rỗng; quy định ylim tối thiểu 0..1 để không có khoảng bằng 0.

S4 | Thử thách độc lập

Viết Chart(ttk.Frame) có FigureCanvasTkAgg, một line label Diem, legend, xlabel Luot, ylabel Diem. update_values(values) nhận list số không âm, x=1..n, ylim 0..max(1,max(values)+1), xlim 0..max(1,n+1), gọi draw_idle. Không tạo thêm axes/line sau cập nhật.

Môi trường: DESKTOP_TK_MPL. Challenge answer: answers/T09/answer.md; test_criteria.json khóa T09.

Gợi ý mở theo nhu cầu

1. Figure chứa Axes; Tk canvas là adapter hiển thị. legend cần label trên đường dữ liệu.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: T09-K01 (chart_model). Xem input/expected hoặc protocol trong test_criteria.json.

Native resize và thay dữ liệu: legend/nhãn không cắt, thanh trục và line đổi đúng; không tích lũy canvas mới.

T10 | Người học tự viết unit test cho GUI

Lộ trình chính | 45 phút dự kiến | Học trước: P20, P18, C13

Nguồn (số trang PDF): GUI 339-409. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Test GUI nên kiểm tra hành vi người dùng nhận được và tách logic thuần để chạy nhanh. Một bài test phải bắt được lỗi, không chỉ chạy mà không crash.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
import unittest
class TestThreshold(unittest.TestCase):
    def test_boundary(self):
        self.assertTrue(passed(5))
if __name__=="__main__": unittest.main()
```

unittest tìm method test_*. Main guard cho phép chạy test trực tiếp mà không chạy khi module được import.

S2 | Kiểm tra hiểu

- Test passed(8)==True có bắt được mutant score>5 không?
- A. Có | Chưa đúng. Cả >=5 và >5 đều nhận 8.
- B. Không, cần test đúng 5 | Đúng. Đúng: test biên phân biệt hai chương trình.
- C. Chỉ cần thêm print | Chưa đúng. print không tự tạo assertion.

S3 | Làm có hỗ trợ rồi thay đổi

- Viết test cho ngưỡng dưới/đúng/trên 5; chạy runner, tô dòng assertion thất bại với mutant >5.
- Tạo Tk trong setUp, withdraw và destroy trong tearDown; test button.invoke rồi đọc label, không phụ thuộc pixel.
- Chạy cùng suite bằng CLI và Eclipse PyDev test runner hoặc VS Code Testing; ghi rõ công cụ thay thế, không coi workflow IDE là đã thực hiện nếu chỉ chạy CLI.

Thử thay đổi: Thay callback append hai lần; bộ test phải đổ vì saved có độ dài 2 thay vì 1.

S4 | Thử thách độc lập

Nộp test_form.py dùng unittest kiểm tra form được cấp ở form_app.py: score>=5 đổi label Dat, dưới 5 Chưa dat. Test 4.9/5/8; test UI nhập 5, invoke nút, label Dat. Main guard cho phép python test_form.py. Chạy mutant >5 phải có ít nhất một FAIL. Nộp ảnh test runner IDE.

Môi trường: DESKTOP_TK. Challenge answer: answers/T10/answer.md; test_criteria.json khóa T10.

File/học liệu cấp kèm: form_app.py.

Gợi ý mở theo nhu cầu

- unittest tìm method test_*. Main guard cho phép chạy test trực tiếp mà không chạy khi module được import.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: T10-K01 (suite); T10-K02 (native). Xem input/expected hoặc protocol trong test_criteria.json.

Thực hiện mutation >5: suite chuyển FAIL ở boundary và UI; sửa lại suite PASS. IDE runner phát hiện đủ bốn test, mở assertion lỗi tới đúng dòng.

T11 | Debug watch và mức log

Lộ trình chính | 30 phút dự kiến | Học trước: P13, T08

Nguồn (số trang PDF): GUI 339-409. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Breakpoint dừng ở thời điểm cần quan sát; watch theo dõi biểu thức. Log giúp xem luồng chạy khi không dừng chương trình.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
import logging
logger=logging.getLogger("pe.form")
logger.debug("parse input")
logger.info("saved")
logger.warning("invalid score")
logger.error("save failed")
```

Mức log lọc theo mức độ; không dùng print rải rác thay cho chính sách log. Không ghi mật khẩu hoặc dữ liệu nhạy cảm vào log.

S2 | Kiểm tra hiểu

1. Logger ở mức WARNING có hiện INFO không?
- A. Có | Chưa đúng. INFO thấp hơn ngưỡng.
- B. Không | Đúng. Đúng: WARNING và mức cao hơn mới qua.
- C. Chỉ hiện INFO | Chưa đúng. Ngược cơ chế lọc.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt breakpoint trước so sánh score; watch raw,score,score>=5; nhập 5 và bước qua từng dòng.
 2. Tạo lỗi dừng >5; watch chỉ ra biểu thức False tại ranh giới; sửa và chạy lại.
 3. Đổi mức log DEBUG sang WARNING; vùng log còn warning/error, không mất thông báo lỗi quan trọng.
- Thử thay đổi: Giữ UI chạy nhưng tắt log DEBUG; giải thích kết quả nghiệp vụ không được thay đổi vì mức log.

S4 | Thử thách độc lập

Viết save_message(text,logger): text strip rỗng thì warning("empty") và trả False; có dữ liệu thì debug("validated"), info("saved") và trả True. Nộp ảnh breakpoint/watch của lỗi >5 tại score=5 và bản sửa >=5; không lấy trace mô phỏng của PE thay cho IDE watch.

Môi trường: PY_CORE. Challenge answer: answers/T11/answer.md; test_criteria.json khóa T11.

Gợi ý mở theo nhu cầu

1. Mức log lọc theo mức độ; không dùng print rải rác thay cho chính sách log. Không ghi mật khẩu hoặc dữ liệu nhạy cảm vào log.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: T11-K01 (logs). Xem input/expected hoặc protocol trong test_criteria.json.

Ảnh IDE có giá trị raw, score và biểu thức điều kiện ở dòng breakpoint; minh chứng WARNING lọc INFO nhưng giữ ERROR.

T12 | Quốc tế hóa và định dạng theo ngôn ngữ

Lộ trình chính | 30 phút dự kiến | Học trước: T08, C16

Nguồn (số trang PDF): GUI 339-409. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Dịch nhãn và định dạng ngày/số là hai việc khác nhau. Tách khóa thông điệp khỏi widget giúp đổi toàn bộ giao diện mà không mất dữ liệu đang nhập.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
TEXT={"vi":{"save":"Lưu"},"en":{"save":"Save"}}
button.configure(text=TEXT[language]["save"])
# Ngày: vi 31/12/2024; en 12/31/2024
```

Ngôn ngữ hiển thị không nên quyết định cách lưu dữ liệu. Lưu ngày ở ISO, số ở kiểu số; chỉ format ở rìa giao diện.

S2 | Kiểm tra hiểu

1. Đổi language có nên xóa nội dung Entry đang nhập không?
- A. Có | Chưa đúng. Đó là dữ liệu người dùng, không phải bản dịch.
- B. Không | Đúng. Đúng: chỉ cập nhật nhãn và định dạng trình bày.
- C. Chỉ khi đổi sang English | Chưa đúng. Không phụ thuộc ngôn ngữ đích.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tách title/save/name thành catalog vi/en; widget chỉ giữ khóa thông điệp.
 2. Bấm đổi ngôn ngữ, cả title và button đổi cùng lượt; Entry giữ An.
 3. Format cùng số 1234.5 và ngày 2024-12-31 theo vi/en; không thay giá trị lưu.
- Thử thay đổi: Xóa một khóa trong catalog đích; hiển thị fallback từ en và ghi cảnh báo thiếu bản dịch.

S4 | Thử thách độc lập

Viết tr(lang,key): fallback en nếu ngôn ngữ/khóa thiếu, nếu en cũng thiếu trả key. Viết format_date(iso,lang) theo vi dd/mm/YYYY, en mm/dd/YYYY. format_number(value,lang) hai chữ số: vi 1.234,50; en 1,234.50. Viết apply_language(root,button,lang) đổi title và nút bằng catalog, không chạm Entry.

Môi trường: PY_CORE. Challenge answer: answers/T12/answer.md; test_criteria.json khóa T12.

Gợi ý mở theo nhu cầu

1. Ngôn ngữ hiển thị không nên quyết định cách lưu dữ liệu. Lưu ngày ở ISO, số ở kiểu số; chỉ format ở rìa giao diện.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: T12-K01 (locale); T12-K02 (fallback). Xem input/expected hoặc protocol trong test_criteria.json.

Native: hai lần đổi ngôn ngữ cập nhật mọi nhãn theo catalog và giữ Entry; quy tắc format hai locale trong đề là thiết kế bài, không bao quát toàn bộ locale thế giới.

T13 | Slideshow GIF/JPEG và vòng đời ảnh

Lộ trình chính | 30 phút dự kiến | Học trước: T07

Nguồn (số trang PDF): GUI 455-498. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Tk cần giữ tham chiếu tới ảnh đang hiển thị. Pillow đọc nhiều định dạng, còn after điều khiển chuyển ảnh mà không khóa giao diện.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
from PIL import Image, ImageTk
photo=ImageTk.PhotoImage(Image.open("slide.jpg"))
label.configure(image=photo)
label.image=photo
```

Giữ label.image ngăn ảnh Python bị thu gom khi biến local hết scope. Mở file ảnh bằng context để đóng handle.

S2 | Kiểm tra hiểu

1. Nếu chỉ giữ tên file mà không giữ PhotoImage thì sao?
- A. Ảnh luôn còn | Chưa đúng. Tên file không giữ object PhotoImage.
- B. Ảnh có thể biến mất khi object bị thu gom | Đúng. Đúng.
- C. Tk tự đọc lại file | Chưa đúng. Không có tự đọc lại như vậy.

S3 | Làm có hỗ trợ rồi thay đổi

1. Lab cấp một GIF đỏ và một JPEG xanh; Next chuyển index 0->1->0.
 2. Thêm after 1000ms và giữ token lịch; Pause hủy token.
 3. Thử danh sách rỗng: hiện Không có ảnh, không chia modulo 0.
- Thử thay đổi: Bấm Start hai lần; chỉ còn một lịch timer, không tăng tốc ngoài ý muốn.

S4 | Thử thách độc lập

Viết SlideShow(root,paths) với label, index, show(), next(), start(), stop(); giữ photo, hỗ trợ GIF/JPEG bằng Pillow. Ảnh thumbnail tối đa 320x200. Danh sách rỗng hiện Không có ảnh. start hủy lịch trước và lên lịch mỗi giây; stop hủy lịch. Fixture do test tự tạo bằng Pillow.

Môi trường: DESKTOP_TK_PIL. Challenge answer: answers/T13/answer.md; test_criteria.json khóa T13.

Gợi ý mở theo nhu cầu

1. Giữ label.image ngăn ảnh Python bị thu gom khi biến local hết scope. Mở file ảnh bằng context để đóng handle.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghệ thuật bài học

Các ca bắt buộc: T13-K01 (images). Xem input/expected hoặc protocol trong test_criteria.json.

Chạy native qua ít nhất hai chu kỳ timer và Pause; ảnh không trắng; đóng cửa sổ sau stop không có TclError.

T14 | Thiết kế lập và factory cho thành phần GUI

Lộ trình chính | 45 phút dự kiến | Học trước: T08, T10

Nguồn (số trang PDF): GUI 339-409; GUI 499-560. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Khi yêu cầu đổi, một thiết kế dễ sửa cần giữ hợp đồng giữa các phần. Factory chọn loại component; không nên thêm pattern nếu một hàm đơn giản đã đủ.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
def make_validator(kind):
    if kind=="name": return lambda value: bool(value.strip())
    if kind=="score": return score_valid
    raise ValueError(kind)
```

Client chỉ biết validator nhận text và trả bool. Khi thêm loại mới, kiểm thử hợp đồng cũ vẫn phải chạy.

S2 | Kiểm tra hiểu

1. Có cần biến mọi hàm thành factory để code tốt hơn không?
 - A. Có | Chưa đúng. Pattern có chi phí và không là mục tiêu tự thân.
 - B. Không, chỉ dùng khi cần chọn nhiều biến thể | Đúng. Đúng: lý do sử dụng phải xuất phát từ yêu cầu.
 - C. Factory thay thế test | Chưa đúng. Vẫn cần kiểm tra hành vi.

S3 | Làm có hỗ trợ rồi thay đổi

1. Vòng 1: form chỉ có tên, tiêu chí tên không rỗng; test và thu phản hồi.
2. Vòng 2: thêm điểm; tách validation khỏi widget bằng factory; giao diện hiển thị lỗi gần đúng field.
3. Vòng 3: yêu cầu giữ input sau lỗi; viết acceptance case rồi cập nhật callback.

Thử thay đổi: Đổi yêu cầu từ điểm 0..10 thành tên khóa học không rỗng; xác định test nào thay, test nào phải giữ.

S4 | Thử thách độc lập

Viết make_validator(kind) trả callable cho name (strip không rỗng) và score (float hữu hạn 0..10), kind lạ raise ValueError. Nộp hai phiên bản wireframe form, một phản hồi cụ thể, thay đổi đáp ứng phản hồi và test hồi quy. Chứng minh form không xóa dữ liệu khi validation sai.

Môi trường: PY_CORE. Challenge answer: answers/T14/answer.md; test_criteria.json khóa T14.

Gợi ý mở theo nhu cầu

1. Client chỉ biết validator nhận text và trả bool. Khi thêm loại mới, kiểm thử hợp đồng cũ vẫn phải chạy.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: T14-K01 (factory). Xem input/expected hoặc protocol trong test_criteria.json.

Rubric: yêu cầu ban đầu rõ; phản hồi có nguồn/tình huống; before/after đáp ứng đúng phản hồi; regression case giữ input sai. Thiếu một trong bốn thì chưa hoàn thành phần thiết kế lập.

X01 | PW1: quản lý lớp bằng collection và CLI

Lộ trình chính | 90 phút dự kiến | Học trước: C12, C16, C09, C10, P10

Nguồn (số trang PDF): SL 29-31; SL 44-45; SL 58-61; SL 83-86; SL 88-89,94; SL 90. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 3 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Một hệ thống quản lý lớp cần nối hồ sơ, môn học và điểm bằng mã định danh. Bài này dùng collection và hàm để hiểu dữ liệu trước khi chuyển sang class.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
students={"S01":{"name":"An","dob":"2005-01-02"}}
courses={"PY":{"name":"Python"}}
marks={"PY":{"S01":8.0}}
```

Điểm nằm dưới cặp course_id/student_id; tên có thể trùng nhưng mã không trùng. DoB lưu ISO để kiểm tra bằng date.

S2 | Kiểm tra hiểu

- Hai sinh viên cùng tên có phải là cùng một người không?
 - Có | Chưa đúng. Tên không là định danh ổn định.
 - Không, dùng mã để phân biệt | Đúng. Đúng.
 - Chỉ cần lấy người đầu | Chưa đúng. Lấy người đầu có thể gán nhầm điểm.

S3 | Làm có hỗ trợ rồi thay đổi

- Nhập số sinh viên2, hai bộ id/name/dob; nhập số môn1 và id/name; hiển thị bảng tương ứng.
- Chọn PY và nhập điểm từng mã sinh viên; map cập nhật theo mã.
- Liệt kê sinh viên, môn và điểm theo môn; thử hai người trùng tên.

Thử thay đổi: Đổi số lượng thành0; chương trình phải có collection rỗng hợp lệ, không hỏi hồ sơ thừa.

S4 | Thử thách độc lập

Theo PW1 (SL97-98), tạo 1.student.mark.py dùng list/dict/tuple, không class. Nhập số lượng sinh viên/môn, hồ sơ id/name/dob và môn id/name; chọn môn nhập điểm cho mọi sinh viên. Cung cấp list_students/list_courses/course_marks. Mã trùng/DoB sai/điểm ngoài0..10 phải ValueError. Bản CLI có menu xem ba loại danh sách và exit. Commit/push vào fork của C03.

Môi trường: PY_CORE. Challenge answer: answers/X01/answer.md; test_criteria.json khóa X01.

Gợi ý mở theo nhu cầu

- Điểm nằm dưới cặp course_id/student_id; tên có thể trùng nhưng mã không trùng. DoB lưu ISO để kiểm tra bằng date.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: X01-K01 (PW1); X01-K02 (counts). Xem input/expected hoặc protocol trong test_criteria.json.

Đối chiếu từng chức năng PW1, chạy menu và bài input2sinh viên/1môn; push commit có file đúng tên lên fork. Lỗi nhập có thể nộp lại ở phiên luyện, không đánh dấu hoàn thành nếu app crash giữa luồng demo.

A01 | Đa kế thừa, MRO và lớp trừu tượng

Học phần nâng cao | 30 phút dự kiến | Học trước: P16, C20

Nguồn (số trang PDF): SL 123-124; SL 101-106. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Đa kế thừa cần quy tắc chọn phương thức rõ ràng. Lớp trừu tượng quy định thao tác lớp con phải cung cấp trước khi tạo instance.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
class A:
    def label(self): return "A"
class B:
    def label(self): return "B"
class C(A,B): pass
print(C().label())
print([c.__name__ for c in C.__mro__])
```

C().label() tìm ở C rồi A rồi B theo MRO của ví dụ. Đổi thứ tự base có thể đổi hành vi; không coi đa kế thừa là mặc định tốt hơn composition.

S2 | Kiểm tra hiểu

- Subclass chưa cài abstractmethod có tạo instance được không?
- Được | Chưa đúng. ABC theo dõi phương thức trừu tượng chưa cài.
- Không, TypeError | Đúng. Đúng.
- Chỉ cần đổi tên class | Chưa đúng. Tên class không đáp ứng hợp đồng.

S3 | Làm có hỗ trợ rồi thay đổi

- Duyệt từng class trong __mro__ và tô nơi tìm thấy label.
 - Tạo abstract Exporter.export; thử instantiate và đọc TypeError.
 - Tạo TextExporter cài export, thêm PrefixMixin; giải thích type conformance bằng cùng cách gọi export.
- Thử thay đổi: Đổi thứ tự hai mixin có cùng method; dự đoán hành vi và giải thích trước khi chạy.

S4 | Thử thách độc lập

Viết Exporter(ABC) có abstractmethod export(rows). TextExporter trả các chuỗi rows nối bằng dấu phẩy. UpperMixin.export gọi super().export(rows).upper(). LoudExporter(UpperMixin,TextExporter) trả chữ hoa. Nộp MRO và giải thích vì sao UpperMixin dùng super để phối hợp.

Môi trường: PY_CORE. Challenge answer: answers/A01/answer.md; test_criteria.json khóa A01.

Gợi ý mở theo nhu cầu

- C().label() tìm ở C rồi A rồi B theo MRO của ví dụ. Đổi thứ tự base có thể đổi hành vi; không coi đa kế thừa là mặc định tốt hơn composition.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: A01-K01 (dispatch); A01-K02 (abstract). Xem input/expected hoặc protocol trong test_criteria.json.

A02 | NumPy và xếp hạng GPA có trọng số

Học phần nâng cao | 30 phút dự kiến | Học trước: C05, P11, C09

Nguồn (số trang PDF): SL 150. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Điểm môn nhiều tín chỉ đóng góp nhiều hơn vào GPA. NumPy biểu diễn vector điểm và tín chỉ để tính tích vô hướng, rồi xếp sinh viên theo GPA giảm dần.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
import numpy as np
marks=np.array([8,6],dtype=float)
credits=np.array([3,1],dtype=float)
print(np.dot(marks,credits)/credits.sum())
```

Kết quả 7.5, khác trung bình không trọng số 7.0. Làm tròn điểm lúc nhập theo C05; không làm tròn từng tích.

S2 | Kiểm tra hiểu

1. Có nên chia tổng điểm có trọng số cho số môn không?
- A. Có | Chưa đúng. Sẽ sai khi tín chỉ khác nhau.
- B. Không, chia tổng tín chỉ | Đúng. Đúng.
- C. Chia số sinh viên | Chưa đúng. Số sinh viên không tham gia công thức GPA.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo hai array và quan sát shape,dtype; mismatch độ dài hiện lỗi hợp đồng.
 2. Hiện từng tích $8*3$ và $6*1$, tổng 30 chia 4.
 3. Thêm A GPA7.5,B GPA8,C GPA8; xếp B,C,A với mã tăng dần khi hòa.
- Thử thay đổi: Đổi tín chỉ [3,1] thành [1,3], dự đoán GPA=6.5.

S4 | Thử thách độc lập

Viết gpa(marks,credits) dùng NumPy: cùng độ dài, credits>0, điểm hữu hạn 0..10; rỗng trả None, sai raise ValueError. Viết ranking(records) nhận dict id -> (marks,credits), trả list (id,gpa) giảm dần GPA; None cuối, hòa chọn id tăng.

Môi trường: LIBS:numpy. Challenge answer: answers/A02/answer.md; test_criteria.json khóa A02.

Gợi ý mở theo nhu cầu

1. Kết quả 7.5, khác trung bình không trọng số 7.0. Làm tròn điểm lúc nhập theo C05; không làm tròn từng tích.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mời lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: A02-K01 (weighted); A02-K02 (invalid). Xem input/expected hoặc protocol trong test_criteria.json.

A03 | Curses: menu bàn phím cho quản lý điểm

Học phần nâng cao | 40 phút dự kiến | Học trước: C12, C19

Nguồn (số trang PDF): SL 150-151. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Curses vẽ giao diện trong terminal và đọc phím. Nó cần terminal tương tác; phần mô phỏng trên web chỉ giúp học luồng điều khiển.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
def main(screen):
    screen.addstr(0,0,"So hoc tap - q de thoat")
    screen.refresh()
    while screen.getkey()!="q": pass
curses.wrapper(main)
```

wrapper khởi tạo và khôi phục terminal kể cả khi lỗi. addstr cần tọa độ nằm trong kích thước terminal.

S2 | Kiểm tra hiểu

1. Vì sao dùng wrapper thay tự gọi main(None)?
- A. Để quản lý môi trường terminal | Đúng. Đúng: screen và chế độ terminal phải được chuẩn bị và dọn.
- B. Để chạy nhanh hơn mọi GUI | Chưa đúng. Không có bảo đảm tốc độ đó.
- C. Để tự tạo database | Chưa đúng. Curses chỉ lo UI terminal.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chạy trong terminal Linux/macOS hoặc Windows có windows-curses; đọc getmaxyx.
 2. Vẽ ba lựa chọn List students/List courses/Exit; phím lên xuống đổi hàng highlight.
 3. Enter chọn mục, q thoát; resize terminal nhỏ thì rút ngắn dòng, không crash.
- Thử thay đổi: Đổi dữ liệu danh sách và độ rộng terminal; UI phải cắt phần hiển thị nhưng không sửa bản ghi.

S4 | Thử thách độc lập

Viết menu curses với lựa chọn Students/Courses/Exit, mũi tên cuộn vòng, Enter chọn hiển thị An,Binh hoặc Python, q thoát. Dùng wrapper và clipping theo getmaxyx. Nộp ghi hình cả điều hướng, resize và khôi phục terminal sau thoát.

Môi trường: MANUAL_CURSES. Challenge answer: answers/A03/answer.md; test_criteria.json khóa A03.

Gợi ý mở theo nhu cầu

1. wrapper khởi tạo và khôi phục terminal kể cả khi lỗi. addstr cần tọa độ nằm trong kích thước terminal.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A03-K01 (keyboard); A03-K02 (cleanup). Xem input/expected hoặc protocol trong test_criteria.json.

A04 | Nén bytes và đóng gói nhiều file

Học phần nâng cao | 30 phút dự kiến | Học trước: C21, C22

Nguồn (số trang PDF): SL 173-178,191. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Nén làm thay đổi biểu diễn bytes; giải nén phải khôi phục đúng dữ liệu ban đầu. Archive còn giữ nhiều tên file trong một gói.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
import gzip,bz2,zlib
payload=b"PE "*100
print(gzip.decompress(gzip.compress(payload))==payload)
print(bz2.decompress(bz2.compress(payload))==payload)
```

Kích thước không phải luôn nhỏ hơn, nhất là dữ liệu ngắn. zip/tar giữ cấu trúc archive, gzip thường nén một luồng.

S2 | Kiểm tra hiểu

1. Nén chuỗi ngắn xong file lớn hơn có chứng minh thuật toán hỏng không?
- A. Có | Chưa đúng. Không được kết luận chỉ theo kích thước.
- B. Không, còn header và overhead | Đúng. Đúng: kiểm tra round-trip là tiêu chí chính.
- C. Chỉ đúng khi file rỗng | Chưa đúng. Cả chuỗi ngắn không rỗng cũng có thể lớn hơn.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đổi text thành UTF-8 bytes, nén bằng zlib/gzip/bz2 và đo len.
 2. Giải nén với đúng codec và so từng byte; thử sai codec để thấy lỗi.
 3. Đóng students.txt,courses.txt,marks.txt bằng ZipFile writestr; đọc theo tên không extract ra đường dẫn tùy ý.
- Thử thay đổi: Thử payload rỗng và bytes ngẫu nhiên; đừng đặt test bắt buộc luôn giảm kích thước.

S4 | Thử thách độc lập

Viết compress_bytes(data,method), decompress_bytes(blob,method) cho zlib/gzip/bz2, method lạ ValueError. Viết pack_files(files) nhận dict tên -> bytes tạo ZIP trong memory, unpack_files(blob) trả dict bytes; bài chỉ cho tên basename, không folder.

Môi trường: PY_CORE. Challenge answer: answers/A04/answer.md; test_criteria.json khóa A04.

Gợi ý mở theo nhu cầu

1. Kích thước không phải luôn nhỏ hơn, nhất là dữ liệu ngắn. zip/tar giữ cấu trúc archive, gzip thường nén một luồng.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A04-K01 (codecs); A04-K02 (archive). Xem input/expected hoặc protocol trong test_criteria.json.

Thêm lab tarfile: tạo tar.gz từ ba fixture bằng add(...,arcname=basename), đọc getnames/extractfile và so bytes; không extractall dữ liệu không tin cậy.

A05 | Pickle, JSON và dữ liệu có kiểu

Học phần nâng cao | 30 phút dự kiến | Học trước: A04, P14, C10

Nguồn (số trang PDF): SL 179-181,192; SL 182. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

JSON trao đổi cấu trúc đơn giản; pickle lưu nhiều kiểu Python hơn nhưng phụ thuộc Python và không được nạp từ nguồn không tin cậy.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
import pickle, json
value=("An",[8,9])
print(type(pickle.loads(pickle.dumps(value))).__name__)
print(type(json.loads(json.dumps(value))).__name__)
```

Pickle khôi phục tuple; JSON trở thành list. Không phải mọi object như file đang mở đều pickle được.

S2 | Kiểm tra hiểu

1. Có thể dùng pickle.load cho file người lạ tải lên không?
- A. Có vì chỉ là dữ liệu | Chưa đúng. Pickle không chỉ là parser dữ liệu thuần.
- B. Không, quá trình unpickle có thể chạy mã | Đúng. Đúng: chỉ lab file tự tạo và tin cậy.
- C. Có nếu đuôi .dat | Chưa đúng. Đuôi file không xác định mức tin cậy.

S3 | Làm có hỗ trợ rồi thay đổi

1. Round-trip tuple và dict chứa set bằng pickle; thử JSON với set để quan sát TypeError.
 2. Thử pickle file handle đang mở; ghi loại lỗi và giải thích tài nguyên bên ngoài không được giữ nguyên như dữ liệu.
 3. Gzip pickle bytes, giải nén rồi load; đối chiếu kiểu và giá trị.
- Thử thay đổi: Chuyển schema giữa hai phiên bản: giải thích cần version/migration và giữ module class khi pickle object.

S4 | Thử thách độc lập

Viết dump_state(state) trả gzip(pickle(state)), load_state(blob) trả object từ blob tự tạo trong lab. Fixture có tuple và set, phải giữ kiểu. Dữ liệu gzip hỏng phải phát sinh lỗi, không trả state rỗng. Nộp bảng so JSON/pickle về kiểu, trao đổi và mức tin cậy.

Môi trường: PY_CORE. Challenge answer: answers/A05/answer.md; test_criteria.json khóa A05.

Gợi ý mở theo nhu cầu

1. Pickle khôi phục tuple; JSON trở thành list. Không phải mọi object như file đang mở đều pickle được.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A05-K01 (types); A05-K02 (corrupt). Xem input/expected hoặc protocol trong test_criteria.json.

Bảng so phải nêu tuple->list của JSON, set không JSON mặc định, pickle chỉ dữ liệu tin cậy và class cần import đúng module.

A14 | Đọc bảng số bằng NumPy

Học phần nâng cao | 30 phút dự kiến | Học trước: A02, C21

Nguồn (số trang PDF): SL 328. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

File số đồng nhất có thể đọc thẳng vào array. File có header, kiểu hỗn hợp hoặc dữ liệu thiếu cần quy tắc đọc rõ ràng.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
data=np.loadtxt("scores.csv",delimiter=",",skiprows=1,usecols=[1,2],ndmin=2)
print(data.shape,data.dtype)
records=np.genfromtxt("mixed.csv",delimiter=",",names=True,dtype=None,encoding="utf8")
```

loadtxt phù hợp bảng số sạch; genfromtxt hỗ trợ tên cột và dữ liệu thiếu. recfromcsv trong phụ lục cũ không được dùng làm API mặc định ở NumPy mới.

S2 | Kiểm tra hiểu

1. usecols=[1,2] chọn cột thứ mấy?
- A. Thứ1 và2 | Chưa đúng. Index cột bắt đầu từ0.
- B. Thứ2 và3 | Đúng. Đúng.
- C. Hai dòng đầu | Chưa đúng. usecols là cột, không phải dòng.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo fixture id,math,python với A,8,6 và B,5,9; bỏ header và cột mã.
 2. Xem shape=(2,2), dtype=float; dùng help(np.loadtxt) đối chiếu skiprows/usecols.
 3. Thêm một giá trị thiếu; genfromtxt filling_values=-1 rồi phân biệt missing với điểm0.
- Thử thay đổi: File chỉ một dòng: ndmin=2 giữ shape(1,2), tránh code sau hiểu thành vector.

S4 | Thử thách độc lập

Viết load_scores(path) đọc CSV có header và ba cột id,math,python; trả ndarray float chỉ hai cột số, luôn2D. Viết load_missing(path) đọc hai cột số, có header, ô rỗng thành-1 bằng genfromtxt. Các file có ít nhất một dòng dữ liệu.

Môi trường: LIBS:numpy. Challenge answer: answers/A14/answer.md; test_criteria.json khóa A14.

Gợi ý mở theo nhu cầu

1. loadtxt phù hợp bảng số sạch; genfromtxt hỗ trợ tên cột và dữ liệu thiếu. recfromcsv trong phụ lục cũ không được dùng làm API mặc định ở NumPy mới.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mời lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A14-K01 (load). Xem input/expected hoặc protocol trong test_criteria.json.

Học viên đọc thêm fixture mixed bằng names=True,dtype=None, nêu tên field, shape và kiểu của từng field.

A15 | DataFrame từ CSV, Excel và database

Học phần nâng cao | 60 phút dự kiến | Học trước: A14, C19

Nguồn (số trang PDF): SL 328. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

DataFrame giữ nhãn cột và index cho dữ liệu bảng. Cùng schema có thể đến từ CSV, sheet Excel hoặc câu SELECT, nhưng cần kiểm tra kiểu và giá trị thiếu.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
frame=pd.read_csv("scores.csv",na_values=[""])
print(frame.head(),frame.tail(),frame.columns)
frame.info()
with pd.ExcelFile("scores.xlsx") as book:
    print(book.sheet_names)
```

head/tail xem mẫu, info xem kiểu và số ô không thiếu. Tên sheet khác với vị trí sheet; read_sql_query trả bảng từ DB.

S2 | Kiểm tra hiểu

1. DataFrame.info() có trả một DataFrame mới không?
- A. Có | Chưa đúng. Không dùng kết quả info để thay frame.
- B. Không, chủ yếu in thông tin và trả None | Đúng. Đúng.
- C. Trả list điểm | Chưa đúng. Đó không phải phép tổng hợp điểm.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đọc CSV, xem head/tail/index/columns/info và to_numpy; đánh dấu ô thiếu.
2. Tạo workbook có sheet Scores/Notes; chọn Scores bằng tên, chỉ lấy id,score.
3. Tạo SQLite lab bằng SQLAlchemy, inspect(engine).get_table_names; text SELECT và pandas.read_sql_query, đóng connection bằng with.

Thử thay đổi: Đổi thứ tự sheet: cách đọc theo tên Scores vẫn đúng; so với đọc sheet0.

S4 | Thử thách độc lập

Viết read_csv_scores(path) với cột id,score, dòng score thiếu giữ NaN; read_excel_scores(path) lấy sheet Scores và cột id,score; read_database(engine) SELECT id,score FROM scores ORDER BY id qua SQLAlchemy text và pandas. Nộp bản khám phá head/tail/info; không dùng engine.table_names API cũ.

Môi trường: LIBS:pandas,sqlalchemy,openpyxl. Challenge answer: answers/A15/answer.md; test_criteria.json khóa A15.

Gợi ý mở theo nhu cầu

1. head/tail xem mẫu, info xem kiểu và số ô không thiếu. Tên sheet khác với vị trí sheet; read_sql_query trả bảng từ DB.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: A15-K01 (csv_excel); A15-K02 (database). Xem input/expected hoặc protocol trong test_criteria.json.

Chia hai phiên 30 phút: CSV/Excel rồi DB. Người học phải nộp output khám phá kiểu/index, không chỉ DataFrame cuối.

A16 | SAS, Stata, HDF5 và MATLAB

Học phần nâng cao | 60 phút dự kiến | Học trước: A15

Nguồn (số trang PDF): SL 328. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Định dạng khoa học lưu bảng hoặc mảng kèm metadata. Đọc đúng file cần chọn reader theo format, kiểm tra shape và đóng tài nguyên đúng lúc.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
stata=pd.read_stata("scores.dta")
with h5py.File("scores.h5","r") as f:
    values=f["scores"][(0)]
mat=scipy.io.loadmat("scores.mat")["scores"]
```

HDF5 có group/dataset; MATLAB trả dictionary biến. Với SAS bài dùng cả reader sas7bdat và fixture XPORT; XPORT không chứng minh đã đọc sas7bdat.

S2 | Kiểm tra hiểu

1. Có thể đổi đuôi CSV thành .h5 để h5py đọc không?
- A. Có | Chưa đúng. Đổi tên không đổi cấu trúc file.
- B. Không, nội dung nhị phân phải đúng format | Đúng. Đúng.
- C. Chỉ cần encode UTF-8 | Chưa đúng. HDF5 có cấu trúc riêng, không là CSV text.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo fixture Stata,HDF5,MAT cùng giá trị [[8,6],[5,9]], đọc rồi so shape và values.
2. Tạo SAS XPORT bằng pyreadstat.write_xport version5, đọc pandas.read_sas(format=xport); xem tên cột.
3. Với fixture .sas7bdat do lab cấp, dùng pd.read_sas(format=sas7bdat); chỉ ra đây là format khác XPORT. Không dùng thuộc tính .value HDF5 cũ, dùng [()].

Thử thay đổi: Đổi tên dataset HDF5 thành marks; truy cập scores phải KeyError, không âm thầm chọn dataset đầu.

S4 | Thử thách độc lập

Viết read_science(path,kind): kind stata->DataFrame, sas7bdat/xport->DataFrame; hdf5 đọc dataset scores thành ndarray sau khi đóng file; mat đọc biến scores. Nộp bốn kết quả shape/giá trị và hai kết quả cho hai format SAS khi có fixture. Cột SAS bytes cần decode trước khi đổi chiều tên người.

Môi trường: LIBS:pandas,h5py,scipy,pyreadstat. Challenge answer: answers/A16/answer.md; test_criteria.json khóa A16.

Gợi ý mở theo nhu cầu

1. HDF5 có group/dataset; MATLAB trả dictionary biến. Với SAS bài dùng cả reader sas7bdat và fixture XPORT; XPORT không chứng minh đã đọc sas7bdat.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịem thu bài học

Các ca bắt buộc: A16-K01 (formats). Xem input/expected hoặc protocol trong test_criteria.json.

SAS7BDAT gate: dev cấp fixture thực cùng schema math/python và expected [[8,6],[5,9]] hoặc dùng fixture công khai đã kiểm chứng + expected tương ứng. Không đánh dấu format này PASS từ test XPORT.

A17 | Chọn loại biểu đồ và quản lý Figure

Học phần nâng cao | 90 phút dự kiến | Học trước: P19, A14

Nguồn (số trang PDF): SL 329. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 3 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Loại biểu đồ phụ thuộc câu hỏi: xu hướng, quan hệ, phân phối hay trường hai chiều. Tạo nhiều hình không có nghĩa mỗi hình đều phù hợp với dữ liệu.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
fig, axes = plt.subplots(1, 2)
axes[0].plot([1, 2, 3], [4, 7, 6], label="Diem")
axes[0].legend()
axes[1].hist([4, 7, 6], bins=[0, 5, 10])
fig.tight_layout(); fig.savefig("report.png"); plt.close(fig)
```

Histogram đếm theo khoảng; line nối các điểm có thứ tự. savefig trước close; Axes là vùng vẽ nằm trong Figure.

S2 | Kiểm tra hiểu

1. Muốn thấy phân phối điểm, chỉ nối các điểm theo tên sinh viên có phù hợp không?
- A. Luôn phù hợp | Chưa đúng. Tên sinh viên không tự có ý nghĩa liên tục.
- B. Không, histogram/box/violin có thể phù hợp hơn | Đúng. Đúng: chọn theo câu hỏi phân phối.
- C. Phải dùng 3D | Chưa đúng. 3D không giải quyết câu hỏi này.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo gallery1D: line/scatter/bar/barh/axhline/axvline/fill/fill_between; chú thích câu hỏi phù hợp cho mỗi loại.
2. Tạo histogram, boxplot, violinplot từ dữ liệu ít nhất ba giá trị khác nhau; so thông tin được giữ/mất.
3. Tạo gallery2D: imshow, pcolor, pcolormesh, contour, contourf, clabel, arrow, quiver, streamplot; colorbar có nhãn, aspect equal khi cần.

Thử thay đổi: Đổi limits/ticks/style, thêm annotation và alpha; nêu thay đổi nào chỉ trình bày, thay đổi nào có thể làm hiểu sai mức chênh lệch.

S4 | Thử thách độc lập

Nộp gallery.py tạo ba Figure: 2x4 loại1D; 1x3 phân phối; 2x4 ảnh/vector. Dùng cùng fixture được cấp trong đáp án mẫu, tự chọn màu khác nhưng giữ dữ liệu. Mỗi axes có title; có legend, ticks, limits, colorbar, annotation. Lưu PNG transparent và đóng mọi Figure. Giải thích ít nhất ba lựa chọn loại biểu đồ.

Môi trường: LIBS:matplotlib,numpy. Challenge answer: answers/A17/answer.md; test_criteria.json khóa A17.

Gợi ý mở theo nhu cầu

1. Histogram đếm theo khoảng; line nối các điểm có thứ tự. savefig trước close; Axes là vùng vẽ nằm trong Figure.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: A17-K01 (files). Xem input/expected hoặc protocol trong test_criteria.json.

Chia ba phiên 30 phút. Xem cả ba ảnh, không cắt title/legend; đổi chiều bar heights [2,5,3,6], histogram counts [1,1,2]; có bài thử cla/clf/close phân biệt xóa axes/figure và đóng window; interactive show chạy riêng backend native, không coi Agg là test show.

A18 | Dựng MySQL và thiết kế schema bằng Workbench

Học phần nâng cao | 60 phút dự kiến | Học trước: P14, C01

Nguồn (số trang PDF): GUI 283-339. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Ứng dụng cần tách cấu hình kết nối khỏi mã và có schema phản ánh định danh, quan hệ. Workbench giúp xem cấu trúc và chạy SQL trong database lab.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
config={"host": "127.0.0.1", "port": 3306, "user": "pe_lab", "password": os.environ["PE_DB_PASSWORD"],
        "database": "pe_lab"}
connection=mysql.connector.connect(**config)
cursor=connection.cursor(); cursor.execute("SELECT 1")
```

Server MySQL và package connector là hai thành phần khác nhau. Không hard-code mật khẩu trong lesson hoặc bản nộp.

S2 | Kiểm tra hiểu

1. pip install mysql-connector-python có tự tạo MySQL server đang chạy không?

A. Có | Chưa đúng. Đó là driver client.

B. Không | Đúng. Đúng: server cần được cài/chạy riêng.

C. Chỉ trên Windows | Chưa đúng. Vai trò package không đổi theo OS.

S3 | Làm có hỗ trợ rồi thay đổi

1. Lab cài MySQL Server và Workbench theo hệ điều hành; tạo database pe_lab và user chỉ có quyền trong database đó; kiểm tra service/port.

2. Kết nối bằng Workbench rồi bằng Python với cùng host/port/database; SELECT1 trả 1.

3. Thiết kế students(id,name,dob), courses(id,name,credits), marks(student_id,course_id,score); khóa kép marks và FK tới hai bảng.

Thử thay đổi: Đổi port sai và password sai trong bản cấu hình tạm; phân biệt lỗi kết nối với lỗi xác thực, không ghi password vào log.

S4 | Thử thách độc lập

Nộp schema.sql, ảnh sơ đồ quan hệ Workbench, cấu hình mẫu không có bí mật và log SELECT1 từ Python.

students.id/courses.id khóa chính; marks khóa kép và hai FK; credits>0, score0..10. Trong lab có quyền, tạo user pe_lab chỉ truy cập pe_lab. Server/version ghi lại chính xác.

Môi trường: MANUAL_MYSQL. Challenge answer: answers/A18/answer.md; test_criteria.json khóa A18.

Gợi ý mở theo nhu cầu

1. Server MySQL và package connector là hai thành phần khác nhau. Không hard-code mật khẩu trong lesson hoặc bản nộp.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: A18-K01 (schema); A18-K02 (connection). Xem input/expected hoặc protocol trong test_criteria.json.

A19 | MySQL INSERT, UPDATE, DELETE và transaction

Học phần nâng cao | 45 phút dự kiến | Học trước: A18

Nguồn (số trang PDF): GUI 283-339. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Thao tác dữ liệu cần chỉ rõ bản ghi nào thay đổi. Tham số SQL truyền dữ liệu riêng với câu lệnh; commit xác nhận thay đổi, rollback hủy transaction lỗi.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
cursor.execute("INSERT INTO students(id,name,dob) VALUES
(%s,%s,%s)", ("S01", "An", "2005-01-02"))
connection.commit()
```

Placeholder của mysql.connector là %s; không dùng nối chuỗi hay f-string để nhét input vào SQL.

S2 | Kiểm tra hiểu

1. DELETE FROM students thiếu WHERE làm gì?
 - A. Xóa một dòng đầu | Chưa đúng. Không có giới hạn một dòng.
 - B. Xóa mọi dòng phù hợp quyền/ràng buộc | Đúng. Đúng: WHERE xác định phạm vi.
 - C. Chỉ xóa schema | Chưa đúng. DROP TABLE mới bỏ bảng.

S3 | Làm có hỗ trợ rồi thay đổi

1. Insert S01, SELECT kiểm tra đủ ba trường; thử tên có dấu nháy để chứng minh tham số.
2. Update theo id S01, giữ S02; quan sát rowcount.
3. Delete theo id, commit rồi reconnect kiểm tra; thử FK lỗi và rollback.

Thử thay đổi: Update id không có: rowcount0; không tự tạo bản ghi mới. Delete đang được marks tham chiếu phải thông báo ràng buộc.

S4 | Thử thách độc lập

Viết Students(connection) với add(id,name,dob), rename(id,name), delete(id), all(); SQL tham số, write commit thành công/rollback nếu lỗi, luôn đóng cursor. all trả list tuple id,name,dob ORDER BY id. Không tự cascade xóa marks. Test trên MySQL pe_lab, không thay bằng SQLite rồi gọi là MySQL PASS.

Môi trường: MANUAL_MYSQL. Challenge answer: answers/A19/answer.md; test_criteria.json khóa A19.

Gợi ý mở theo nhu cầu

1. Placeholder của mysql.connector là %s; không dùng nối chuỗi hay f-string để nhét input vào SQL.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A19-K01 (CRUD); A19-K02 (transaction). Xem input/expected hoặc protocol trong test_criteria.json.

A20 | Nối CRUD MySQL vào giao diện

Học phần nâng cao | 60 phút dự kiến | Học trước: A19, T02, T04, P13

Nguồn (số trang PDF): GUI 283-339. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Form và database phải cùng phản ánh một lần thao tác. Chỉ hiển thị thành công sau commit; lỗi kết nối phải giữ dữ liệu nhập để người dùng sửa hoặc thử lại.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
def on_save():
    repository.add(id_entry.get(), name_entry.get(), dob_entry.get())
    refresh_list()
    status.configure(text="Đã lưu")
```

Callback gọi repository, không chép SQL rải trong từng widget. Trong ứng dụng thực với I/O lâu, áp dụng worker/queue và kết nối riêng theo worker.

S2 | Kiểm tra hiểu

1. Báo Đã lưu trước khi commit xong có đúng không?
- A. Có | Chưa đúng. Commit vẫn có thể thất bại.
- B. Không | Đúng. Đúng: phản hồi phải theo kết quả thực.
- C. Đúng khi mạng nhanh | Chưa đúng. Tốc độ không thay đổi tính đúng.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo form id/name/dob và Listbox mã; Add gọi repository rồi refresh.
 2. Chọn mã, sửa tên và bấm Rename; only dòng được chọn đổi.
 3. Delete cần xác nhận; hủy giữ dữ liệu, lỗi FK hiện riêng, không giả xóa dòng trong UI.
- Thử thay đổi: Mô phỏng mất kết nối: form giữ input, nút được bật lại và không hiện thành công.

S4 | Thử thách độc lập

Viết build(root, repository) có ba Entry id/name/dob, Listbox records, nút Add/Rename/Delete/Refresh, Label status. Dữ liệu fixture như A19. Callback validate id/name không rỗng và date.fromisoformat(dob); repository lỗi -> status lỗi và giữ Entry. Delete chỉ sau askyesno. Trả dict widget để QA invoke.

Môi trường: MANUAL_MYSQL_TK. Challenge answer: answers/A20/answer.md; test_criteria.json khóa A20.

Gợi ý mở theo nhu cầu

1. Callback gọi repository, không chép SQL rải trong từng widget. Trong ứng dụng thực với I/O lâu, áp dụng worker/queue và kết nối riêng theo worker.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: A20-K01 (end_to_end); A20-K02 (errors). Xem input/expected hoặc protocol trong test_criteria.json.

A21 | wxPython: cửa sổ và các nhóm control

Học phần nâng cao | 45 phút dự kiến | Học trước: T02, T03, C01

Nguồn (số trang PDF): GUI 409-454. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

wxPython là toolkit khác với Tkinter: cùng nhu cầu form nhưng tên API, bố cục và cách nối event khác. Mỗi ứng dụng wx có một wx.App điều phối vòng lặp.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
app=wx.App(False)
frame=wx.Frame(None,title="PE wx",size=(420,300))
button.Bind(wx.EVT_BUTTON,on_click)
frame.Show()
app.MainLoop()
```

Bind đăng ký xử lý event.Sizer phân chia không gian giống vai trò layout manager nhưng không dùng API grid của Tk.

S2 | Kiểm tra hiểu

1. Có thể thay ttk.Button bằng wx.Button mà giữ mọi API bố cục Tk không?
- A. Có | Chưa đúng. Hai toolkit có hợp đồng khác.
- B. Không, phải dùng parent/event/sizer của wx | Đúng. Đúng.
- C. Chỉ đổi import | Chưa đúng. Import không chuyển API.

S3 | Làm có hỗ trợ rồi thay đổi

1. Cài wxPython trong môi trường riêng bằng pip, ghi wx.version và thử frame tối thiểu; mainloop chạy trên main thread.
2. Thêm TextCtrl multiline, Button ghi log, CheckBox, RadioBox và StaticBox chứa label; dùng BoxSizer/StaticBoxSizer.
3. Thêm menu File/Exit, resize và kiểm tra TextCtrl giãn theo sizer.

Thử thay đổi: So form Tk với form wx cùng chức năng; chỉ rõ callback command so với Bind và Frame/LabelFrame so với Panel/StaticBox.

S4 | Thử thách độc lập

Viết Frame wx title PE wx với TextCtrl multiline, nút Print append Clicked+newline, CheckBox Reminder False, RadioBox Basic/Advanced mặc định Basic, StaticBox Info chứa label Ready, menu File/Exit. TextCtrl giãn khi resize. build_frame() trả frame và dict controls, không tự gọi MainLoop khi import.

Môi trường: MANUAL_WX. Challenge answer: answers/A21/answer.md; test_criteria.json khóa A21.

Gợi ý mở theo nhu cầu

1. Bind đăng ký xử lý event. Sizer phân chia không gian giống vai trò layout manager nhưng không dùng API grid của Tk.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: A21-K01 (native); A21-K02 (layout). Xem input/expected hoặc protocol trong test_criteria.json.

A23 | Bitmap lát nền trong wxPython

Học phần nâng cao | 30 phút dự kiến | Học trước: A21

Nguồn (số trang PDF): GUI 455-498. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Ảnh bitmap có thể được lặp thành nền. Khi cửa sổ bị che hoặc resize, nền phải được vẽ lại trong paint event, không chỉ vẽ một lần lúc khởi tạo.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
def on_paint(event):
    dc=wx.AutoBufferedPaintDC(panel)
    for y in range(0,height,tile_height):
        for x in range(0,width,tile_width): dc.DrawBitmap(tile,x,y)
```

EVT_PAINT là nơi vẽ; EVT_SIZE gọi Refresh rồi cho event tiếp tục. Đây là hiệu chỉnh từ ví dụ ClientDC vẽ trực tiếp trong sự kiện resize ở nguồn.

S2 | Kiểm tra hiểu

- Vẽ bằng ClientDC một lần có bảo đảm nền còn sau khi bị cửa sổ khác che không?
- Có | Chưa đúng. OS có thể yêu cầu repaint sau đó.
- Không, cần paint handler | Đúng. Đúng.
- Chỉ cần sleep | Chưa đúng. Sleep không lưu nội dung vẽ.

S3 | Làm có hỗ trợ rồi thay đổi

- Tạo tile8x8 hai màu bằng MemoryDC, chuyển thành bitmap độc lập.
 - Vẽ lặp theo hàng/cột trong EVT_PAINT; resize làm số tile thay đổi.
 - Đặt nút Quit trên panel; nền không được làm mất khả năng click control.
- Thử thay đổi: Che rồi mở lại cửa sổ và thu nhỏ/phóng lớn; nền phải được tái tạo, không để vùng trắng.

S4 | Thử thách độc lập

Viết TiledPanel(wx.Panel) tự tạo bitmap8x8 hai màu, SetBackgroundStyle(PAINT), EVT_PAINT dùng AutoBufferedPaintDC lát kín client area; EVT_SIZE gọi Refresh và Skip. Thêm nút Quit đóng frame. Không cần file ảnh bên ngoài.

Môi trường: MANUAL_WX. Challenge answer: answers/A23/answer.md; test_criteria.json khóa A23.

Gợi ý mở theo nhu cầu

- EVT_PAINT là nơi vẽ; EVT_SIZE gọi Refresh rồi cho event tiếp tục. Đây là hiệu chỉnh từ ví dụ ClientDC vẽ trực tiếp trong sự kiện resize ở nguồn.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: A23-K01 (paint); A23-K02 (resources). Xem input/expected hoặc protocol trong test_criteria.json.

A24 | OpenGL trong wx: cube và phép biến đổi 3D

Học phần nâng cao | 60 phút dự kiến | Học trước: A21, A23

Nguồn (số trang PDF): GUI 455-498. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

OpenGL cần context gắn với vùng vẽ. Dữ liệu đỉnh mô tả hình; phép chiếu và phép biến đổi quyết định hình xuất hiện ở đâu trên màn hình.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
canvas.SetCurrent(context)
glViewport(0,0,width,height)
glMatrixMode(GL_PROJECTION); glLoadIdentity(); gluPerspective(45,width/height,.1,100)
glMatrixMode(GL_MODELVIEW); glLoadIdentity(); glTranslatef(0,0,-6); glRotatef(angle,0,1,0)
```

Bài dùng fixed-function OpenGL compatibility context để giải thích mã cùng thể hệ với sách. Không khẳng định chạy trong core profile hiện đại; A25 dùng shader.

S2 | Kiểm tra hiểu

1. Có thể gọi GL trước khi SetCurrent context phù hợp không?
 - A. Có | Chưa đúng. GL state gắn với context.
 - B. Không bảo đảm đúng | Đúng. Đúng: cần context hiện hành trên thread vẽ.
 - C. Chỉ cần import OpenGL | Chưa đúng. Import không tạo context.

S3 | Làm có hỗ trợ rồi thay đổi

1. Tạo wx.GLCanvas và GLContext, SetCurrent trong paint; ghi glGetString(GL_VERSION).
 2. Vẽ sáu mặt cube, bật depth test; kéo chuột làm angle đổi.
 3. Resize cập nhật viewport và aspect; nhấn Reset đưa angle về 0; nếu context không hỗ trợ, báo environment unsupported.
- Thử thay đổi: Tắt depth test rồi xoay, giải thích mặt sau có thể che sai; bật lại để đối chiếu.

S4 | Thử thách độc lập

Viết CubeCanvas với cube 8 đỉnh/6 mặt, mỗi mặt màu riêng, projection perspective 45°, camera dịch -6 theo z; kéo chuột ngang/dọc đổi hai góc. SetCurrent trước GL, clear color/depth, SwapBuffers mỗi frame. Resize cập nhật aspect; không chia 0 khi height 0. Ghi compatibility requirement và phiên bản GL thực.

Môi trường: MANUAL_OPENGL. Challenge answer: answers/A24/answer.md; test_criteria.json khóa A24.

Gợi ý mở theo nhu cầu

1. Bài dùng fixed-function OpenGL compatibility context để giải thích mã cùng thể hệ với sách. Không khẳng định chạy trong core profile hiện đại; A25 dùng shader.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghị thu bài học

Các ca bắt buộc: A24-K01 (render); A24-K02 (lifecycle). Xem input/expected hoặc protocol trong test_criteria.json.

A25 | Pyglet, shader và cube nhiều màu

Học phần nâng cao | 60 phút dự kiến | Học trước: A24

Nguồn (số trang PDF): GUI 455-498. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Pyglet có cửa sổ và vòng lặp riêng. Shader mô tả cách biến đổi vị trí và màu từng đỉnh, thay cho các lời gọi fixed-function trong bài trước.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
window=pyglet.window.Window(width=640,height=480)
program=ShaderProgram(Shader(vertex_source,"vertex"),Shader(fragment_source,"fragment"))
vertices=program.vertex_list(count,GL_TRIANGLES,position=("f",positions),color=("f",colors))
```

Mỗi thuộc tính phải đủ số thành phần cho count đỉnh. Cube được tách thành 36 đỉnh tam giác để mỗi mặt có màu riêng.

S2 | Kiểm tra hiểu

- Pyglet có cần Tkinter để chạy event loop không?
 - Có | Chưa đúng. Pyglet đã có Window và app loop.
 - Không | Đúng. Đúng.
 - Chỉ cần khi có màu | Chưa đúng. Màu là dữ liệu render, không quyết định toolkit.

S3 | Làm có hỗ trợ rồi thay đổi

- Cài pyglet, chạy Window và label PE ở giữa như bước mở đầu của nguồn.
 - Tạo shader position/color, sáu mặt cube thành 12 tam giác; kiểm tra buffer 36 đỉnh.
 - Giữ phím trái/phải làm góc quay liên tục theo dt, không phụ thuộc số lần key press.
- Thử thay đổi: Đổi màu một mặt và resize cửa sổ; chỉ mặt đó đổi màu, aspect trong uniform cập nhật.

S4 | Thử thách độc lập

Viết pyglet cube dùng OpenGL 3.3 shader, vertex positions và colors 36 đỉnh. Uniform angle/aspect, depth test; giữ LEFT/RIGHT quay 90 độ/giây; label bài mở đầu chạy độc lập trước cube. Nộp log context và ảnh sáu mặt khi xoay. Không dùng API vertex_list cũ ngoài ShaderProgram.

Môi trường: MANUAL_PYGLET_GL. Challenge answer: answers/A25/answer.md; test_criteria.json khóa A25.

Gợi ý mở theo nhu cầu

- Mỗi thuộc tính phải đủ số thành phần cho count đỉnh. Cube được tách thành 36 đỉnh tam giác để mỗi mặt có màu riêng.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A25-K01 (shader); A25-K02 (input). Xem input/expected hoặc protocol trong test_criteria.json.

A26 | Hoạt ảnh OpenGL: chuyển động theo thời gian

Học phần nâng cao | 45 phút dự kiến | Học trước: A25, C10

Nguồn (số trang PDF): GUI 455-498. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Hoạt ảnh cập nhật trạng thái theo thời gian đã trôi, rồi render trạng thái đó. Gắn quãng đường với số frame làm tốc độ thay đổi khi máy chậm.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
x += velocity * dt
if x > right:
    x = 2 * right - x
    velocity = -velocity
```

Phản xạ giữ phần quãng đường vượt biên thay vì bỏ nó. Khi dt lớn có thể vượt nhiều lần, cần xử lý đủ.

S2 | Kiểm tra hiểu

1. Cập nhật $x += 1$ mỗi frame trên 30FPS và 60FPS có cùng tốc độ không?

- A. Có | Chưa đúng. Mỗi giây có số bước khác nhau.
- B. Không, 60FPS nhanh gấp đôi | Đúng. Đúng: dùng $velocity * dt$.
- C. Chỉ phụ thuộc màu | Chưa đúng. Màu không quyết định vận tốc.

S3 | Làm có hỗ trợ rồi thay đổi

- Mô phỏng square đỏ chuyển trên trục 0..100; hiển thị x, v, dt cạnh khung render.
 - Đến biên phản xạ và đổi dấu velocity; bật một frame dt lớn để thấy overshoot.
 - Dùng `pygame.clock.schedule_interval` gọi update và `rectangle.draw` trong `on_draw`; Pause unschedule, Resume chỉ tạo một lịch.
- Thử thay đổi: Chạy cùng 1 giây chia 30 hoặc 60 bước; trạng thái cuối gần bằng nhau trong sai số float.

S4 | Thử thách độc lập

Viết `advance(x, v, dt, limit)`: $0 \leq x \leq \text{limit}$, $\text{limit} > 0, dt \geq 0$; trả $(x_mới, v_mới)$ phản xạ ở 0 và limit, kể cả vượt nhiều lần. Sau đó gắn vào square đỏ trong cửa sổ pygame (render qua OpenGL), hai trục độc lập, kích thước square 20; giới hạn tâm theo client area. Pause/Resume không nhân đôi lịch.

Môi trường: PY_CORE. Challenge answer: `answers/A26/answer.md`; `test_criteria.json` khóa A26.

File/học liệu cấp kèm: `motion_app.py`.

Gợi ý mở theo nhu cầu

- Phản xạ giữ phần quãng đường vượt biên thay vì bỏ nó. Khi dt lớn có thể vượt nhiều lần, cần xử lý đủ.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A26-K01 (bounce); A26-K02 (dt). Xem input/expected hoặc protocol trong `test_criteria.json`.

Native integration cần square đỏ bounce cả x/y, dt đo từ clock, Pause giữ trạng thái và Resume tiếp; answer pure function PASS chưa là chứng cứ hoạt ảnh GPU PASS.

P21 | Thread và queue: nhận kết quả việc nền

Học phần nâng cao | 25 phút dự kiến | Học trước: P10, P13, P15

Nguồn (số trang PDF): SL 249-299; GUI 225-260. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một việc lưu dữ liệu có thể kéo dài. Worker làm việc và gửi kết quả qua queue; giao diện lấy kết quả rồi cập nhật trên luồng UI. Bài này học đường đi dữ liệu, không thi đua tốc độ.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
from threading import Thread
from queue import Queue
q = Queue()
def work():
    q.put("Đã lưu")
t = Thread(target=work)
t.start()
t.join()
message = q.get()
```

start tạo luồng chạy; gọi run trực tiếp không tạo luồng mới. join trong harness giúp test đợi xong; không đặt join chặn callback UI thật. Demo hiển thị timeline luồng học riêng, không khẳng định thứ tự scheduler thực.

S2 | Kiểm tra hiểu

- Lệnh nào khởi động thread mới?
 - t.run() | Chưa đúng. run trực tiếp chạy như lời gọi thường.
 - t.start() | Đúng. Đúng: start khởi động luồng.
 - q.get() | Chưa đúng. get lấy dữ liệu từ queue.
- Worker nên cập nhật UI của app theo thiết kế này bằng cách nào?
 - Gọi widget tùy ý từ worker | Chưa đúng. Không coi đây là cơ chế an toàn mặc định.
 - Dùng sleep thật lâu trong click | Chưa đúng. sleep trong click làm UI bị chặn.
 - Gửi message, UI thread nhận và cập nhật | Đúng. Đúng: tách kết quả worker và cập nhật UI.

S3 | Làm có hỗ trợ rồi thay đổi

- Đặt target=work chưa gọi; timeline có Main và Worker, worker chưa chạy.
 - Bấm Start: worker gửi "Đã lưu" vào q, queue0->1; trace minh họa gắn nhãn mô hình.
 - Trong harness join xong rồi get: queue1->0, message="Đã lưu"; trong UI dùng poll/after thay join chặn.
- Thử thay đổi: Đổi thông báo worker; dự đoán dữ liệu nhận, không dự đoán thời gian chính xác.

S4 | Thử thách độc lập

Viết run_job(values) dùng một Thread và Queue, worker tính bình phương từng số nguyên rồi gửi list; hàm chờ hoàn tất và trả list theo thứ tự đầu vào. Đây là harness đồng bộ kiểm tra trao đổi kết quả, chưa phải callback UI. Đầu vào tối đa 100 phần tử.

Môi trường: WORKER_LAB. Challenge answer: answers/P21/answer.md; test_criteria.json khóa P21.

Gợi ý mở theo nhu cầu

- Worker và nơi nhận cần một kênh truyền dữ liệu.
- Dùng Queue.put/get, Thread.start và join trong hàm harness.
- Worker gửi cả list vào queue; sau join, return q.get().

Phản hồi ngộ nhận: Nếu chỉ gọi run: kết quả có thể đúng nhưng chưa đạt mục tiêu tạo thread. Bộ chấm phải kiểm tra hành vi luồng riêng.

Nghịệm thu bài học

Ví dụ công khai P21-a: {} -> [4, 9]

TH21: instrument Thread.start và Queue.put/get; worker có thread id khác luồng gọi; test hành vi số học riêng không chứng minh điểm này.

TH21-UI: khi tích hợp GUI, công việc giả lập 2s không chặn click/hủy; xử lý exception worker và không cập nhật widget sau đóng cửa sổ.

P22 | Tiến trình con và mã kết thúc

Học phần nâng cao | 25 phút dự kiến | Học trước: P11, P13

Nguồn (số trang PDF): SL 193-248. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Một công cụ riêng có thể xuất báo cáo giúp chương trình chính. Tiến trình con có vùng chạy riêng; stdout mang dữ liệu, returncode báo trạng thái kết thúc. Hai giá trị không phải một.

Trước khi bấm Chạy, hãy dự đoán giá trị hoặc trạng thái cuối của ví dụ. Gửi dự đoán trước, rồi mới mở giải thích bên dưới mã.

```
import subprocess
import sys
completed = subprocess.run(
    [sys.executable, "-c", "print(7)"],
    capture_output=True, text=True, timeout=2
)
report = (completed.stdout.strip(), completed.returncode)
```

stdout chứa "7" còn returncode thường là 0 khi thành công. Bài dùng sys.executable để chọn đúng Python hiện chạy. Truyền danh sách đối số, không qua shell.

S2 | Kiểm tra hiểu

1. report là gì khi chạy thành công?

- A. ("7", 0) | Đúng. Đúng: dữ liệu và mã trạng thái khác nhau.
- B. (0, 7) | Chưa đúng. Bạn đã đổi ý nghĩa hai trường.
- C. ("7", 7) | Chưa đúng. Giá trị chương trình in không thành mã thoát.

2. Vì sao phải có timeout?

- A. Để luôn chạy nhanh hơn | Chưa đúng. Timeout giới hạn chờ, không làm thuật toán nhanh hơn.
- B. Để kiểm soát tiến trình không kết thúc | Đúng. Đúng: có giới hạn cho tình huống bị treo.
- C. Để tự sửa cú pháp | Chưa đúng. Nó không sửa code.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt danh sách argv vào run; sơ đồ Parent tạo Child, không lẫn namespace hai phía.

2. Child print7: stdout buffer rỗng->"7\n"; parent chưa coi 7 là trạng thái.

3. Child exit0: completed.returncode=0, parent tạo tuple("7", 0); timeout là nhánh lỗi riêng.

Thử thay đổi: Fixture child đổi thành sys.exit(3); stdout rỗng, returncode 3; giải thích đây không phải Python in số 3.

S4 | Thử thách độc lập

Viết launch_report() chạy đúng file report_child.py do hệ thống cấp bằng sys.executable, capture_output=True, text=True, timeout=2, shell=False. Trả dict stdout đã strip và returncode. Không nhận chuỗi lệnh tùy ý. TimeoutExpired được giữ nguyên để tăng UI báo timeout.

Môi trường: WORKER_LAB. Challenge answer: answers/P22/answer.md; test_criteria.json khóa P22.

Gợi ý mở theo nhu cầu

- 1. Dữ liệu và trạng thái kết thúc là hai thuộc tính khác nhau.
- 2. Dùng sys.executable và danh sách đối số cố định.
- 3. Trả completed.stdout.strip() và completed.returncode sau subprocess.run.

Phản hồi ngộ nhận: Nếu returncode bằng chữ READY hoặc stdout: bạn đang lấy sai trường kết quả.

Nghiệm thu bài học

Ví dụ công khai P22-a: {} -> {"stdout": "READY", "returncode": 0}

PR22-timeout: thay child bằng fixture chờ 5s, giới hạn 2s phải báo TimeoutExpired và không để tiến trình mở còi.

PR22-safety: runner chặn truy cập host/network/lệnh ngoài vùng bài; yêu cầu shell=False không tự tạo sandbox.

A06 | Tiến trình, trạng thái và PCB

Học phần nâng cao | 30 phút dự kiến | Học trước: P22

Nguồn (số trang PDF): SL 196-206; SL 207-208; SL 209-210; SL 226-228. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Chương trình là mã; tiến trình là một lần chạy với trạng thái và tài nguyên. Bộ lập lịch chọn tiến trình được dùng CPU; chờ I/O khác với sẵn sàng chạy.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
new -> ready -> running -> waiting
waiting -> ready -> running -> terminated
# Context switch: lưu PC/register của P1, nạp PC/register của P2
```

PCB lưu thông tin quản lý tiến trình. Cây cha/con cho biết quan hệ tạo tiến trình, không đồng nghĩa cha và con dùng chung mọi biến.

S2 | Kiểm tra hiểu

1. Tiến trình chờ dữ liệu từ đĩa nằm ở ready hay waiting?
 - A. ready | Chưa đúng. Ready nghĩa là chỉ còn chờ CPU.
 - B. waiting | Đúng. Đúng: chờ sự kiện I/O.
 - C. terminated | Chưa đúng. Chờ I/O chưa phải kết thúc.

S3 | Làm có hỗ trợ rồi thay đổi

1. Đặt các thẻ new/ready/running/waiting/terminated vào trace nạp -> dispatch -> I/O -> hoàn tất.
 2. Chuyển CPU P1 sang P2: lưu pc=12 của P1 và khôi phục pc=7 của P2; không reset cả hai về đầu.
 3. Vẽ cây parent 10 có child 11,12, child11 tạo13; tô vùng bộ nhớ riêng từng process.
- Thử thay đổi: Thêm preempt khi timer hết: running->ready, khác I/O running->waiting.

S4 | Thử thách độc lập

Viết next_state(state,event) theo bảng: new/admit->ready, ready/dispatch->running, running/io->waiting, waiting/complete->ready, running/preempt->ready, running/exit->terminated; cặp khác ValueError. Nộp trace PCB P1(pc12)->P2(pc7)->P1(pc12) và cây 10->{11,12},11->{13}.

Môi trường: PY_CORE. Challenge answer: answers/A06/answer.md; test_criteria.json khóa A06.

Gợi ý mở theo nhu cầu

1. PCB lưu thông tin quản lý tiến trình. Cây cha/con cho biết quan hệ tạo tiến trình, không đồng nghĩa cha và con dùng chung mọi biến.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: A06-K01 (trace); A06-K02 (invalid). Xem input/expected hoặc protocol trong test_criteria.json.

Rubric PCB: lưu PC/register khi rời CPU, khôi phục đúng khi chạy lại; cây có bốn process và vùng memory tách biệt.

A07 | Mô phỏng FCFS, SJF, priority và round-robin

Học phần nâng cao | 45 phút dự kiến | Học trước: A06, C11

Nguồn (số trang PDF): SL 222-225; SL 229-235. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Các cách lập lịch đánh đổi thời gian chờ, phản hồi và công bằng. Mô phỏng giúp thấy khác biệt trước khi đo một hệ điều hành thật.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

P1: burst=5, priority=2

P2: burst=2, priority=1

FCFS: P1[0,5], P2[5,7]

SJF: P2[0,2], P1[2,7]

RR q=2: P1[0,2], P2[2,4], P1[4,6], P1[6,7]

Mọi process tới lúc 0; priority số nhỏ là ưu tiên cao. Trace là mô hình đơn giản, không dự báo chính xác scheduler thật.

S2 | Kiểm tra hiểu

1. RR khác FCFS ở điểm nào trong mô hình này?

A. Có thể thu hồi CPU khi hết quantum | Đúng. Đúng: job chưa xong về cuối ready queue.

B. Luôn chạy job ngắn nhất | Chưa đúng. Đó là ý tưởng SJF.

C. Không có hàng đợi | Chưa đúng. RR dùng hàng đợi.

S3 | Làm có hỗ trợ rồi thay đổi

1. Kéo P1/P2 vào ready queue; chọn FCFS rồi SJF và tính thời gian hoàn tất.

2. Chọn priority, so số priority trước, thứ tự tới khi hòa.

3. Chạy RR từng quantum; job còn thời gian được đưa về cuối hàng đợi.

Thử thay đổi: Đổi quantum từ 2 thành 10; dự đoán RR giống FCFS cho fixture này. Nêu nguy cơ starvation của ưu tiên cố định.

S4 | Thử thách độc lập

Viết `schedule(jobs,policy,quantum=2)`, jobs list (id,burst,priority), burst int>0, tất cả arrival=0. policy fcfs/sjf/priority/rr; SJF và priority không preempt; hòa giữ thứ tự input. Trả list (id,start,end), mô hình context switch cost=0. Policy lạ hoặc quantum<=0 raise ValueError.

Môi trường: PY_CORE. Challenge answer: answers/A07/answer.md; test_criteria.json khóa A07.

Gợi ý mở theo nhu cầu

1. Mọi process tới lúc 0; priority số nhỏ là ưu tiên cao. Trace là mô hình đơn giản, không dự báo chính xác scheduler thật.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: A07-K01 (policies); A07-K02 (empty_tie). Xem input/expected hoặc protocol trong test_criteria.json.

Tính waiting: FCFS P1=0,P2=5; SJF P2=0,P1=2; RR P1=2,P2=2. Giải thích quantum nhỏ tăng số lần chuyển và priority có thể gây starvation.

A08 | Tạo tiến trình trên Windows và fork/exec trên UNIX

Học phần nâng cao | 45 phút dự kiến | Học trước: P22, A06

Nguồn (số trang PDF): SL 211-213; SL 214-221. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Windows và UNIX cung cấp cơ chế tạo tiến trình khác nhau. Python subprocess cung cấp giao diện chung; fork/exec là kiến thức riêng của UNIX.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
pid=os.fork()
if pid==0:
    os.execv(sys.executable,[sys.executable,"-c","print(123)"])
else:
    os.waitpid(pid,0)
```

fork trả 0 ở child và PID child ở parent; exec thay image tiến trình hiện tại, không tạo thêm một child mới. os.fork không có trên Windows.

S2 | Kiểm tra hiểu

1. exec thành công có quay lại dòng ngay sau exec trong chương trình cũ không?

A. Có | Chưa đúng. Image cũ đã được thay.

B. Không | Đúng. Đúng.

C. Chỉ ở parent | Chưa đúng. Nhánh nào gọi exec thành công đều bị thay.

S3 | Làm có hỗ trợ rồi thay đổi

1. Vẽ fork thành hai luồng điều khiển với hai giá trị trả về; gán biến ở child không sửa biến parent.

2. Trong Linux lab chạy fork rồi exec Python in123, parent waitpid; đếm đúng hai process.

3. Trong Windows lab chạy subprocess.Popen; đối chiếu khái niệm CreateProcess gồm image mới và process/thread handles; phân biệt WinExec API cũ với CreateProcess.

Thử thay đổi: Cho exec đường dẫn không tồn tại: child phải báo lỗi rồi exit khác0, parent thu mã kết thúc.

S4 | Thử thách độc lập

Nộp hai script: unix_spawn.py dùng fork/execv/waitpid và windows_spawn.py dùng Popen(...,shell=False). Child in123 và exit0, parent báo mã0. Nộp sơ đồ fork/exec so với CreateProcess; giải thích WinExec là API lịch sử trong slide, không dùng làm mẫu API Python mới. Chỉ chạy script đúng OS.

Môi trường: MANUAL_OS_PAIR. Challenge answer: answers/A08/answer.md; test_criteria.json khóa A08.

Gợi ý mở theo nhu cầu

1. fork trả 0 ở child và PID child ở parent; exec thay image tiến trình hiện tại, không tạo thêm một child mới. os.fork không có trên Windows.

2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghệ thu bài học

Các ca bắt buộc: A08-K01 (POSIX); A08-K02 (Windows). Xem input/expected hoặc protocol trong test_criteria.json.

A09 | Luồng chuẩn, pipe và vòng đời process

Học phần nâng cao | 40 phút dự kiến | Học trước: P22, A06

Nguồn (số trang PDF): SL 236-240; SL 245. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

stdin đưa dữ liệu vào, stdout đưa kết quả ra, stderr dành cho thông báo lỗi. Pipe nối luồng; timeout cần thu dọn process để không để việc chạy mồ côi.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
p=subprocess.Popen(args,stdin=subprocess.PIPE,stdout=subprocess.PIPE,stderr=subprocess.PIPE,te
xt=True)
try:
    out,err=p.communicate("An",timeout=2)
except subprocess.TimeoutExpired:
    p.kill(); out,err=p.communicate()
```

communicate vừa ghi vừa đọc để giảm nguy cơ tắc pipe. Sau kill vẫn cần communicate/wait để thu process.

S2 | Kiểm tra hiểu

1. Sau TimeoutExpired, process đã chắc bị dừng chưa?
 - A. Chưa với Popen.communicate | Đúng. Đúng: caller phải xử lý vòng đời.
 - B. Luôn tự dừng | Chưa đúng. Không được giả định vậy.
 - C. Luôn exit0 | Chưa đúng. Timeout không là thành công.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chạy child đọc stdin, in upper vào stdout và note vào stderr; hiện ba vùng riêng.
2. Redirect stdout vào file, stderr vẫn riêng; đọc file kiểm tra kết quả.
3. Chạy child ngủ 10 giây với timeout 0.2; kill + communicate, UI báo timed_out=True.

Thử thay đổi: Child exit3 nhưng stdout có dữ liệu: giữ cả stdout và returncode, không coi có output là thành công.

S4 | Thử thách độc lập

Viết run_child(args,text,timeout): Popen shell=False, pipes text=True; communicate; khi timeout kill và communicate lần nữa. Trả dict stdout,stderr,returncode,timed_out. args list do lab cấp; không ghép chuỗi shell.

Môi trường: PY_CORE. Challenge answer: answers/A09/answer.md; test_criteria.json khóa A09.

Gợi ý mở theo nhu cầu

1. communicate vừa ghi vừa đọc để giảm nguy cơ tắc pipe. Sau kill vẫn cần communicate/wait để thu process.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: A09-K01 (streams); A09-K02 (timeout). Xem input/expected hoặc protocol trong test_criteria.json.

A10 | Bộ nhớ dùng chung, GIL và chọn concurrency

Học phần nâng cao | 40 phút dự kiến | Học trước: P21, A06

Nguồn (số trang PDF): SL 251-264; SL 265-272; SL 274-276; SL 278-286. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Thread trong một process chia sẻ heap nhưng có luồng thực thi và stack riêng. Concurrency giúp tổ chức công việc; không bảo đảm mọi chương trình chạy nhanh hơn.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

I/O chờ mạng -> nhiều thread có thể chờ xen kẽ

CPU tính Python thuần, CPython có GIL -> cân nhắc process

GUI -> main thread xử lý widget, worker gửi kết quả

Bài giả định CPython có GIL. Có build free-threaded từ Python3.13; luôn ghi phiên bản/build và loại tác vụ khi so sánh.

S2 | Kiểm tra hiểu

1. Bốn thread tính Python thuần trên CPython có GIL có chắc nhanh gấp4 không?

A. Có | Chưa đúng. Còn GIL và chi phí điều phối.

B. Không | Đúng. Đúng: phải đo, không suy từ số core.

C. GIL cấm mọi I/O song song | Chưa đúng. Thread vẫn hữu ích khi chờ I/O.

S3 | Làm có hỗ trợ rồi thay đổi

1. Vẽ hai thread cùng trở heap và stack riêng; sửa shared list ở worker, parent thấy thay đổi sau join.

2. So hai process có memory riêng, muốn trao đổi cần IPC.

3. Dựa mô hình Apache/Chromium trong slide, đánh dấu nơi dùng nhiều process để cô lập và nơi dùng thread để xử lý đồng thời; không xem sơ đồ lịch sử là mô tả phiên bản hiện tại.

Thử thay đổi: Thêm overhead khởi tạo process vào mô hình một tác vụ rất nhỏ; giải thích vì sao tuần tự có thể nhanh hơn.

S4 | Thử thách độc lập

Với ba case tải10URL, tính nhiều số nguyên tố bằng Python thuần, GUI lưu file lớn: chọn mô hình, vẽ memory/stack và kênh kết quả; nêu responsiveness, throughput, scalability khác nhau. So kiến trúc Apache/Chromium theo trang274-276 và ghi rõ giới hạn phiên bản/build.

Môi trường: CONCEPT_REVIEW. Challenge answer: answers/A10/answer.md; test_criteria.json khóa A10.

Gợi ý mở theo nhu cầu

1. Bài giả định CPython có GIL. Có build free-threaded từ Python3.13; luôn ghi phiên bản/build và loại tác vụ khi so sánh.

2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: A10-K01 (models); A10-K02 (architecture). Xem input/expected hoặc protocol trong test_criteria.json.

A11 | Race condition và Lock

Học phần nâng cao | 30 phút dự kiến | Học trước: P21, A10

Nguồn (số trang PDF): SL 273; SL 296-297. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

S1 | Hiểu tình huống và dự đoán

Hai worker cùng đọc rồi ghi một bộ đếm có thể làm mất cập nhật. Lock bảo vệ cả chuỗi read-modify-write, không chỉ riêng lệnh ghi.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
A read0; B read0; A write1; B write1
# Muốn tổng2 nhưng nhận1
with lock:
    counter += 1
```

Trace phân rõ minh họa lỗi; không dựa vào việc một lần chạy ngẫu nhiên có tình cờ xuất hiện race hay không.

S2 | Kiểm tra hiểu

- Chỉ khóa bước write nhưng read nằm ngoài lock có đủ không?
- Đủ | Chưa đúng. Hai worker vẫn có thể đọc cùng giá trị cũ.
- Không | Đúng. Đúng: bảo vệ toàn bộ thao tác logic.
- GIL luôn giải quyết | Chưa đúng. GIL không thay thế hợp đồng đồng bộ.

S3 | Làm có hỗ trợ rồi thay đổi

- Kéo bốn thẻ read/write tạo trace lost update như ví dụ; counter hiển thị 1.
- Gộp read+increment+write vào critical section; worker B chờ A nhả khóa.
- Chạy nhiều worker tăng counter; join hết trước khi kiểm tra tổng.

Thử thay đổi: Giữ lock trong khi sleep mô phỏng I/O; quan sát các worker bị tuần tự hóa và rút ngắn critical section.

S4 | Thử thách độc lập

Viết `count_parallel(workers, iterations)`, hai `int` ≥ 0 , dùng `threading.Thread` và `Lock` để tổng cuối đúng `workers*iterations`. Không dùng `sleep` làm cơ chế đồng bộ. Nộp thêm trace hai worker giải thích lost update và bản sửa khóa cả read-modify-write.

Môi trường: PY_CORE. Challenge answer: `answers/A11/answer.md`; `test_criteria.json` khóa A11.

Gợi ý mở theo nhu cầu

- Trace phân rõ minh họa lỗi; không dựa vào việc một lần chạy ngẫu nhiên có tình cờ xuất hiện race hay không.
- Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: A11-K01 (counter). Xem input/expected hoặc protocol trong `test_criteria.json`.

Không chấm hiểu race chỉ từ stress test PASS: trace phải có hai `read0` trước hai `write1` và giải thích phạm vi Lock.

A12 | Nhiều worker, dùng hợp tác và queue qua module

Học phần nâng cao | 45 phút dự kiến | Học trước: P21, A11, C19

Nguồn (số trang PDF): SL 287-291; GUI 225-283. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Thread không có nút kill an toàn cho mọi tác vụ. Worker cần kiểm tra tín hiệu dừng, gửi kết quả hoặc lỗi qua queue và thoát có kiểm soát.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
class Worker(Thread):
    def run(self):
        if not self.stop_event.is_set():
            self.output.put((self.job_id, self.value**2))
```

start chạy run ở thread mới; gọi run trực tiếp vẫn chạy trong caller. Event là tín hiệu phối hợp; Queue truyền dữ liệu giữa module được cấp chung instance.

S2 | Kiểm tra hiểu

- Muốn worker nhận queue của controller, nên làm gì?
 - Tạo queue riêng không trả ra | Chưa đúng. Controller không đọc được queue riêng.
 - Truyền cùng queue qua constructor | Đúng. Đúng: dependency được truyền tường minh.
 - Dùng sleep để đọc biến global | Chưa đúng. Sleep không là giao thức kết quả.

S3 | Làm có hỗ trợ rồi thay đổi

- Tạo workers.py chứa subclass, controller.py tạo Queue/Event và truyền vào hai worker.
- Start cả hai, thu kết quả theo job_id thay vì thứ tự hoàn tất.
- Set stop_event trước start: worker không làm việc; join rồi kiểm tra queue rỗng.

Thử thay đổi: Cho hai worker hoàn thành đảo thứ tự, báo cáo vẫn giữ thứ tự input bằng job_id.

S4 | Thử thách độc lập

Viết SquareWorker(Thread) constructor(job_id, value, output, stop_event); run kiểm tra event trước khi tính, nếu chưa dừng put (id, value**2). Viết run_many(values) tạo một worker/mục, start tất cả rồi join, đọc queue và trả list theo id. Tách class vào workers.py khi nộp dự án. Đây là job ngắn; job nhiều bước phải kiểm tra Event mỗi bước.

Môi trường: PY_CORE. Challenge answer: answers/A12/answer.md; test_criteria.json khóa A12.

Gợi ý mở theo nhu cầu

- start chạy run ở thread mới; gọi run trực tiếp vẫn chạy trong caller. Event là tín hiệu phối hợp; Queue truyền dữ liệu giữa module được cấp chung instance.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: A12-K01 (multi); A12-K02 (stop). Xem input/expected hoặc protocol trong test_criteria.json.

Nộp workers.py/controller.py không circular import; đúng một Queue được tạo ở controller và truyền vào mọi worker. Thêm job vòng lặp 10 bước: set Event sau bước 2, worker xác nhận dừng trước bước 10, không dùng terminate giả.

A13 | TCP và HTTP qua dịch vụ lab cục bộ

Học phần nâng cao | 45 phút dự kiến | Học trước: A09, A12

Nguồn (số trang PDF): GUI 225-283. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

TCP truyền một luồng bytes nên cần quy tắc đóng khung thông điệp. HTTP đã có giao thức request/response; urlopen đọc response với timeout.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
with socket.create_connection(("127.0.0.1",port),timeout=2) as sock:
    sock.sendall(b"hello\n")
    reply=sock.makefile("rb").readline()
with urlopen(url,timeout=2) as response:
    text=response.read().decode("utf8")
```

Lab dùng loopback và cổng do OS cấp để không phụ thuộc website công cộng. Một recv không bảo đảm nhận đủ cả thông điệp.

S2 | Kiểm tra hiểu

1. Một lần send tương ứng chính xác một lần recv không?
- A. Có | Chưa đúng. Không có ranh giới message ở TCP.
- B. Không, TCP là byte stream | Đúng. Đúng: dùng newline, length prefix hoặc protocol khác.
- C. Chỉ khi cùng máy | Chưa đúng. Loopback vẫn là stream.

S3 | Làm có hỗ trợ rồi thay đổi

1. Server bind 127.0.0.1:0, lấy port thật; client gửi dòng hello, server trả HELLO với newline.
 2. Chia message thành hai lần send; reader vẫn chờ newline và nhận một dòng hoàn chỉnh.
 3. Mở HTTP server lab trả UTF-8 PE; urlopen đọc status/body; thử endpoint 404 và timeout.
- Thử thay đổi: Response TCP đến từng byte: code phải vẫn đúng; giới hạn lab message <=1024byte và timeout 2 giây.

S4 | Thử thách độc lập

Viết tcp_line(host,port,message): message không chứa newline, gửi UTF-8 + newline, đọc một dòng tối đa 1025byte với timeout 2; thiếu newline hoặc >1024byte ValueError; trả text không newline. Viết fetch_text(url) urlopen timeout 2, read tối đa 4097byte, >4096 ValueError, UTF-8; HTTPError giữ nguyên. Nộp server loopback để kiểm tra.

Môi trường: MANUAL_NETWORK. Challenge answer: answers/A13/answer.md; test_criteria.json khóa A13.

Gợi ý mở theo nhu cầu

1. Lab dùng loopback và cổng do OS cấp để không phụ thuộc website công cộng. Một recv không bảo đảm nhận đủ cả thông điệp.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghệ thuật bài học

Các ca bắt buộc: A13-K01 (TCP); A13-K02 (HTTP). Xem input/expected hoặc protocol trong test_criteria.json.

A22 | Hai toolkit: chặn đoán mainloop và trao đổi dữ liệu

Học phần nâng cao | 60 phút dự kiến | Học trước: A21, A12, A13, T07

Nguồn (số trang PDF): GUI 409-454. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Hai toolkit đều cần điều khiển event loop. Gọi loop thứ hai trong callback có thể chặn loop thứ nhất; chạy GUI tùy tiện trong worker không phải cách sửa di động.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
# Trace thử nghiệm trong sách:
# Tk callback -> wx.App.MainLoop -> chưa quay về Tk
# wx callback -> Tk.mainloop -> chưa quay về wx
# Thiết kế lab: mỗi process có một GUI loop ở main thread; JSON line nối hai process.
```

Hai recipe embedding trong nguồn là thử nghiệm, có giới hạn và lỗi được tác giả nêu. Bài giữ phần phân tích đó và triển khai giải pháp tách process như một thiết kế bổ sung, không gọi là những widget native.

S2 | Kiểm tra hiểu

1. Chuyển cả wx GUI sang một thread nên có bảo đảm sửa được trên mọi OS không?

A. Có | Chưa đúng. GUI toolkit thường yêu cầu main thread.

B. Không | Đúng. Đúng: cần tôn trọng loop/thread của từng toolkit.

C. Chỉ cần daemon=True | Chưa đúng. Daemon chỉ ảnh hưởng vòng đời thread.

S3 | Làm có hỗ trợ rồi thay đổi

1. Vẽ hai trace Tk gọi wx loop và wx gọi Tk loop; đánh dấu callback nào không trả về và cửa sổ nào ngừng xử lý sự kiện.

2. Đọc ví dụ thử nghiệm worker trong nguồn; chỉ ra lỗi lifecycle/main thread, không thực thi như mẫu production.

3. Chạy hai process, mỗi GUI ở main thread; reader thread chỉ put message vào Queue, Tk after hoặc wx.Timer nhận và cập nhật widget.

Thử thay đổi: Đổi vai host từ Tk sang wx; giao thức name/quit giữ nguyên. Đóng peer trước, host phải báo peer closed thay vì treo.

S4 | Thử thách độc lập

Viết dual_gui.py nhận tk/wx và --peer. Host mở toolkit còn lại bằng subprocess; hai form có Entry/TextCtrl và nút Send. Gửi JSON line type=name,value=...; form bên kia hiển thị Received: tên. Worker reader không chạm widget. Tk poll bằng after, wx poll bằng Timer. Đóng host gửi quit, chờ tối đa 1 giây rồi thu dọn child. Nộp trace giải thích cả hai hướng embedding thất bại và giải pháp này là phối hợp process, không embedding widget.

Môi trường: MANUAL_DUAL_GUI. Challenge answer: answers/A22/answer.md; test_criteria.json khóa A22.

Gợi ý mở theo nhu cầu

1. Hai recipe embedding trong nguồn là thử nghiệm, có giới hạn và lỗi được tác giả nêu. Bài giữ phần phân tích đó và triển khai giải pháp tách process như một thiết kế bổ sung, không gọi là những widget native.

2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.

3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: A22-K01 (both_hosts); A22-K02 (lifecycle). Xem input/expected hoặc protocol trong test_criteria.json.

X02 | PW2-PW4: domain objects, GPA và chia package

Học phần nâng cao | 120 phút dự kiến | Học trước: X01, C20, A01, A02, A03, C19

Nguồn (số trang PDF): SL 107-113; SL 118-120; SL 125-127; SL 136-139,151; SL 143-148; SL 150; SL 150-151. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 4 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Sau khi dữ liệu và hành vi đã rõ, chuyển hệ thống sang object để giữ quy tắc ở domain. Chia module theo input, output và domain giúp UI thay đổi mà không viết lại tính điểm.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
pw4/
main.py
input.py
output.py
domains/
__init__.py
student.py
course.py
registry.py
```

Student giữ id/name/dob, Course giữ id/name/credits, Registry liên kết điểm. Các hàm nhập/list có cùng ý nghĩa ở hai loại bản ghi là cơ hội dùng đa hình.

S2 | Kiểm tra hiểu

1. Cần chuyển cả thuật toán GPA vào callback curses không?
- A. Có | Chưa đúng. Sẽ trộn trách nhiệm và khó đổi UI.
- B. Không, giữ trong domain/service | Đúng. Đúng.
- C. Curses tự tính GPA | Chưa đúng. Curses chỉ hiển thị.

S3 | Làm có hỗ trợ rồi thay đổi

1. Chuyển collection X01 thành Student/Course, giữ toàn bộ input/list/marks; so output trước/sau.
 2. Áp dụng floor(score*10)/10 khi nhập điểm; dùng NumPy cho GPA và xếp giảm dần.
 3. Tách package đúng cây, output.py hiển thị qua curses; main phối hợp, không có công thức GPA trong UI.
- Thử thay đổi: Thay output curses bằng print để test; Registry phải không phụ thuộc curses.

S4 | Thử thách độc lập

Hoàn thành PW2/3/4: Student và Course có input/list theo giao diện chung; Registry quản lý domain objects, điểm đã floor 1 chữ số, GPA bằng NumPy, ranking giảm dần. Môn có credits int>0; chỉ tính GPA trên các môn đã có điểm, chưa có điểm trả None (quy ước bổ sung). File input.py đọc số lượng và trường như X01; output.py curses liệt kê; main.py menu. Nộp package và commit/push từng checkpoint.

Môi trường: LIBS:numpy. Challenge answer: answers/X02/answer.md; test_criteria.json khóa X02.

File/học liệu cấp kèm: PACKAGE_LAYOUT.md.

Gợi ý mở theo nhu cầu

1. Student giữ id/name/dob, Course giữ id/name/credits, Registry liên kết điểm. Các hàm nhập/list có cùng ý nghĩa ở hai loại bản ghi là cơ hội dùng đa hình.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiem thu bài học

Các ca bắt buộc: X02-K01 (domain); X02-K02 (polymorphism). Xem input/expected hoặc protocol trong test_criteria.json.

Ba checkpoint 40 phút: domain; math/ranking; package+curses. Nghiệm thu không chỉ chạy domain tests: main nhập đủ số lượng, output curses chọn môn/xem điểm, cây file đúng và import không mở UI.

X03 | PW5-PW6: ba file, archive và pickle có nén

Học phần nâng cao | 90 phút dự kiến | Học trước: X02, A04, A05

Nguồn (số trang PDF): SL 155-157; SL 173-178,191; SL 179-181,192; SL 182. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 3 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Dữ liệu phải sống qua lần đóng ứng dụng. Ghi ba file để quan sát trước, rồi thay định dạng lưu bằng pickle có nén mà giữ cùng hành vi nạp lại.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
students.txt -> thông tin sinh viên
courses.txt -> thông tin môn
marks.txt -> điểm
students.dat -> archive của ba file ở PW5
students.dat -> gzip pickle ở PW6, schema/version khác
```

Hai phiên bản students.dat không được đọc lẫn mà không nhận diện format. Bài dùng thư mục pw5 và pw6 riêng.

S2 | Kiểm tra hiểu

1. Đọc file nén hỏng rồi coi như lớp rỗng có phù hợp không?
- A. Có | Chưa đúng. Có thể ghi đè lên dữ liệu cần phục hồi.
- B. Không, phải báo lỗi để tránh mất dữ liệu | Đúng. Đúng.
- C. Chỉ khi có UI | Chưa đúng. Nguyên tắc áp dụng cả CLI và GUI.

S3 | Làm có hỗ trợ rồi thay đổi

1. Sau nhập sinh viên ghi students.txt, sau nhập môn ghi courses.txt, sau nhập điểm ghi marks.txt; dùng UTF-8 JSON trong từng file text.
 2. Trước thoát ZIP ba file vào students.dat; khi khởi động có archive thì đọc/nạp, thiếu archive tạo state rỗng.
 3. Sao chép checkpoint sang pw6, đổi sang gzip pickle object Registry và kiểm tra lại cả kiểu, DoB, credits, điểm.
- Thử thay đổi: Chạy lại bằng process mới để phát hiện pickle class nằm sai module `__main__`; đặt class ở domains như X02.

S4 | Thử thách độc lập

Viết `save_text_bundle(folder,state)`, `load_text_bundle(folder)`: state có students/courses/marks dạng JSON, lưu đúng ba file .txt và ZIP students.dat; load thiếu archive trả ba dict rỗng, archive hỏng giữ lỗi. Viết `save_pickle(path,registry)`, `load_pickle(path)`: gzip pickle; file thiếu trả None, hỏng giữ lỗi. Tích hợp lưu sau mỗi nhóm nhập và nạp khi mở app. Dữ liệu pickle chỉ từ lab tin cậy.

Môi trường: PY_CORE. Challenge answer: answers/X03/answer.md; test_criteria.json khóa X03.

Gợi ý mở theo nhu cầu

1. Hai phiên bản students.dat không được đọc lẫn mà không nhận diện format. Bài dùng thư mục pw5 và pw6 riêng.
2. Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghiệm thu bài học

Các ca bắt buộc: X03-K01 (bundle); X03-K02 (pickle). Xem input/expected hoặc protocol trong test_criteria.json.

Nạp object Registry thật từ process mới với domains import được; DoB/credits/marks và kiểu Student/Course giữ nguyên. Bản PW5 phải thấy ba file cập nhật ngay sau từng nhóm input, không chỉ lúc thoát.

X04 | PW7: shell có chuyển hướng và pipeline

Học phần nâng cao | 90 phút dự kiến | Học trước: A09, C11, C21

Nguồn (số trang PDF): SL 236-240; SL 241-245; SL 245; SL 247-248. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 3 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Shell đọc lệnh, tách argument và nối các luồng. Một pipeline chạy các process đồng thời và đóng các pipe không còn dùng để bên đọc nhận EOF.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
producer | consumer
command < input.txt
command > output.txt
# POSIX ví dụ nguồn: ls -la; ls -la > out.txt; bc < input.txt; ps aux | grep term
```

Bài định nghĩa grammar nhỏ: argument có quote, nhiều stage phân cách |, < chỉ ở đầu và > chỉ ở cuối. Không yêu cầu biến môi trường/glob/ background job.

S2 | Kiểm tra hiểu

- Parent giữ đầu ghi pipe sau khi producer kết thúc có thể gây gì?
 - Consumer chưa thấy EOF | Đúng. Đúng: mọi đầu ghi phải đóng mới có EOF.
 - Luôn nhanh hơn | Chưa đúng. Giữ handle thừa có thể làm treo.
 - Tự xóa file | Chưa đúng. Pipe không là thao tác xóa file.

S3 | Làm có hỗ trợ rồi thay đổi

- Parse lệnh có quoted argument thành argv, không split bằng khoảng trắng đơn giản.
- Mở file input/output trong context; stdout producer nối stdin consumer; parent đóng bản sao pipe đã chuyển.
- Chạy pipeline hai stage, thu returncode; timeout thu dọn mọi child.

Thử thay đổi: Thử lệnh không tồn tại và pipe trống như cmd |; báo lỗi có ngữ cảnh, không thoát cả shell tương tác.

S4 | Thử thách độc lập

Viết run_line(line, timeout=5) cho grammar trên, trả {stdout, returncodes}. stderr của mỗi stage chuyển về terminal (không capture để tránh deadlock); shell=False. Có <, > thì chỉ nhận một tên file sau mỗi toán tử; syntax khác raise ValueError. main lập input, exit thoát, lỗi hiển thị rồi tiếp tục. Lab POSIX chạy đủ bốn lệnh nguồn; test portable dùng Python fixture.

Môi trường: PY_CORE. Challenge answer: answers/X04/answer.md; test_criteria.json khóa X04.

Gợi ý mở theo nhu cầu

- Bài định nghĩa grammar nhỏ: argument có quote, nhiều stage phân cách |, < chỉ ở đầu và > chỉ ở cuối. Không yêu cầu biến môi trường/glob/ background job.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: X04-K01 (portable); X04-K02 (syntax). Xem input/expected hoặc protocol trong test_criteria.json.

POSIX gate: ls -la; output redirect; bc input (cài bc trong lab); ps aux | grep term. Cú pháp giới hạn đã công bố; không gọi bài này là shell đầy đủ của OS. Chạy lệnh child ngủ với timeout, kiểm tra không còn process con.

X05 | PW8: Lưu pickle có nén trong background

Học phần nâng cao | 60 phút dự kiến | Học trước: X03, A11, A12, T07

Nguồn (số trang PDF): SL 299. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 2 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Lưu nền giữ giao diện phản hồi, nhưng dữ liệu phải là một snapshot ổn định. Không được báo đã lưu khi worker còn đang ghi.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

```
snapshot=copy.deepcopy(state)
worker=Thread(target=save_snapshot,args=(snapshot,path,results))
worker.start()
# UI poll queue, không join blocking trong callback
```

Snapshot ngăn dữ liệu đang sửa bị ghi dở thành trạng thái pha trộn. Worker chỉ ghi file và gửi kết quả; UI nhận rồi đổi trạng thái.

S2 | Kiểm tra hiểu

- Sao chép nông có đủ nếu state có list/dict lồng nhau đang thay đổi không?
 - Luôn đủ | Chưa đúng. Các phần bên trong vẫn dùng chung tham chiếu.
 - Không | Đúng. Đúng: cần snapshot nhất quán theo mô hình dữ liệu.
 - Chỉ cần daemon | Chưa đúng. Daemon không bảo vệ dữ liệu.

S3 | Làm có hỗ trợ rồi thay đổi

- Chụp deepcopy ở main thread khi không có mutator khác; đổi tên sau đó, file phải chứa tên thời điểm bắt đầu lưu.
 - Worker ghi file tạm cùng thư mục rồi os.replace; thành công put("ok",path), lỗi put("error",message).
 - UI poll bằng after; ngăn hai job lưu cùng path chồng nhau và yêu cầu chờ/hủy đóng cho tới khi job kết thúc.
- Thử thay đổi: Giả lập lỗi ghi: giữ file cũ, UI báo error và vẫn cho chỉnh sửa; không ghi đè bằng file nửa chừng.

S4 | Thử thách độc lập

Viết start_save(state,path) chụp deepcopy rồi khởi chạy Thread, trả (thread,Queue). Worker ghi gzip pickle snapshot vào file tạm cùng folder và os.replace khi hoàn tất; lỗi gửi error, xóa file tạm do job tạo. File cũ còn nguyên nếu bước ghi thất bại. UI tích hợp không gọi join trong callback, chỉ cho một job lưu mỗi path.

Môi trường: PY_CORE. Challenge answer: answers/X05/answer.md; test_criteria.json khóa X05.

Gợi ý mở theo nhu cầu

- Snapshot ngăn dữ liệu đang sửa bị ghi dở thành trạng thái pha trộn. Worker chỉ ghi file và gửi kết quả; UI nhận rồi đổi trạng thái.
- Quay lại ví dụ và đổi chiều đầu vào, kiểu dữ liệu, trạng thái trước/sau.
- Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nhiệm vụ bài học

Các ca bắt buộc: X05-K01 (snapshot); X05-K02 (failure). Xem input/expected hoặc protocol trong test_criteria.json.

Native UI vẫn đếm nút phụ trong khi job lớn; một path chỉ có một worker; WM_DELETE_WINDOW hỏi chờ, poll tới xong rồi destroy. Kiểm thử injection lỗi trước replace xác nhận file cũ nguyên vẹn.

X06 | PW9: ứng dụng quản lý điểm hoàn chỉnh bằng Tkinter

Học phần nâng cao | 120 phút dự kiến | Học trước: X02, X03, X05, T02, T04, T08, T10

Nguồn (số trang PDF): SL 155-157; SL 299; SL 312-313; SL 322-324; SL 325; GUI 156-187. SL: slide; HF: Head First bản Early Release; GUI: Cookbook 2e.

Chia thành 4 phiên; lưu checkpoint sau từng nhiệm vụ nhỏ.

S1 | Hiểu tình huống và dự đoán

Bài cuối nối domain, lưu trữ và giao diện thành một sản phẩm. Người học phải chứng minh một luồng công việc hoàn chỉnh sau khi khởi động lại.

Dự đoán kết quả hoặc trạng thái của ví dụ trước khi bấm Chạy. Sau đó chỉ ra bước khiến dự đoán thay đổi.

Tab Sinh viên: ID, Ten, DoB, Them, Danh sach

Tab Môn học: ID, Ten, Tín chỉ, Them

Tab Điểm: Chon mon, Chon sinh vien, Diem, Luu

Tab Báo cáo: GPA giam dan

Thanh trạng thái: Chưa lưu -> Đang lưu -> Đã lưu / Lỗi

Mỗi tab gọi cùng Registry; không tạo bản sao dữ liệu nghiệp vụ riêng cho mỗi form. Persist là checkpoint sau một thay đổi hợp lệ.

S2 | Kiểm tra hiểu

1. UI hiện điểm8 nhưng mở lại thành7 có được coi là hoàn thành không?
- A. Có vì UI đúng | Chưa đúng. Ứng dụng phải giữ dữ liệu qua phiên.
- B. Không, luồng lưu/ nạp chưa đúng | Đúng. Đúng: kiểm tra end-to-end gồm khởi động lại.
- C. Chỉ cần thêm animation | Chưa đúng. Animation không sửa persistence.

S3 | Làm có hỗ trợ rồi thay đổi

1. Dựng bốn tab, bind lựa chọn theo mã; dữ liệu hai sinh viên trùng tên không gán nhầm điểm.
 2. Mọi thay đổi hợp lệ cập nhật Registry, refresh list/report rồi đánh dấu dirty; Save dùng X05.
 3. Mở app: nạp gzip pickle tin cậy; file thiếu tạo Registry mới, file hỏng báo lỗi và không tự ghi đè. Đóng khi đang lưu đợi job xong.
- Thử thay đổi: Thử điểm0, điểm10,11,nan; chỉ0 và10 được lưu; thay đổi điểm cập nhật GPA và vị trí ranking.

S4 | Thử thách độc lập

Hoàn thành pw9 kế thừa toàn bộ PW1-PW8 liên quan quản lý điểm: hồ sơ DoB, môn credits, input/list/marks, floor1chữ số, NumPy GPA/ranking, package domain, lưu pickle+nén nền và Tkinter. Shell PW7 là dự án riêng. Hai học viên cùng tên phải độc lập. Nộp source package, video luồng test bên dưới và commit/push. Đáp án kiến trúc và prototype tham chiếu được đóng gói trong capstone_pw9.

Môi trường: MANUAL_CAPSTONE. Challenge answer: answers/X06/answer.md; test_criteria.json khóa X06.

Gợi ý mở theo nhu cầu

1. Mỗi tab gọi cùng Registry; không tạo bản sao dữ liệu nghiệp vụ riêng cho mỗi form. Persist là checkpoint sau một thay đổi hợp lệ.
2. Quay lại ví dụ và đối chiếu đầu vào, kiểu dữ liệu, trạng thái trước/sau.
3. Mở lời giải từng phần sau khi đã nộp một lần; ghi lượt này là có hỗ trợ.

Nghịệm thu bài học

Các ca bắt buộc: X06-K01 (journey); X06-K02 (boundaries); X06-K03 (responsive). Xem input/expected hoặc protocol trong test_criteria.json.